Table of Contents

Frontmatter

Secure all the things with the Spring Authorization Server by Josh Long

ISBN: {isbn} Version 1.0.

without limitation, responsibility for damages resulting from the use of or reliance on this work.

The use of the information and instructions contained in this work is at your own risk.

If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights. :code: ..

Conventions

In this book I'll do certain things that are for the purposes of this book and not necessarily the best configuration for code. There's a fundamental tension in the way books are written and the way code is written that I wrestle with on every attempt. I've long tried to find ways to write that avoid errors in the reproduced code. I want all the code printed to be compiled and valid. It is for this reason that the vast majority of the code reprinted in this book, and especially of the code reprinted in this book that comes from code I've written for this book, is actually in include of some actual source code. It's *not* code I copied and pasted into some Microsoft Word document or something, and that has long since diverged from the state of the "actual" code.

Code

As I work through these examples, I'll be showing lots of code. You can follow along at home, either by typing things out or simply referencing the code here [https://github.com/secure-all-the-things-book]. It's Apache 2 licensed and yours for the taking.

Docker

I assume the availability of Docker or some sort of container orchestrator on your machines. Where appropriate, I've provided Docker Compose `compose.yml` files in each of the chapters' code.

If you're using the Docker tooling, you can spin up the container images thusly:

```
docker compose up
```

That will run the Docker images and block, keeping the shell on that process in the foreground. If you want to run things and the background them, do:

```
docker compose up -d
```

Formatting

I wrote some tooling to ensure that every example in the book is using the same version of Spring Boot, assumes the same version of Java (or Kotlin), and is using the same version of any projects that aren't managed by Spring Boot's parent `pom.xml`.

I also wrote some tooling to format the `pom.xml` files exactly the same and to format the Java source code exactly the same, too. I'm using the Spring Java Format Maven plugin [https://github.com/spring-io/spring-javaformat]. There's also a variant for Gradle.

The Spring Boot team created this plugin to consistently format the code in Spring Boot itself. This way, it doesn't matter if a contributor to the project is using emacs, vim, IntelliJ IDEA, Visual Studio Code, Eclipse, etc.: their code can be quickly and consistently formatted to match the project's style. Less quibbling about tabs vs. spaces, and more closed issues. This is a *good thing*!

If you want to format the source code:

```
./mvnw spring-javaformat:apply
```

Sometimes the formatter, which internally is based on the Eclipse IDE's formatter machinery, might format things in a way that reads poorly on a printed page. I sometimes force its hand by add empty line comment delimiters (//) at the end of a line, which forces the formatter to wrap to the next line.

Noisy Configuration Classes

Often, you'll see me introduce new beans in new configuration classes when they could've just as easily gone in a previous configuration class. This is so that I might avoid reprinting the original beans while keeping code working.

Spring Profiles

Similarly, I might introduce beans that I don't intend for us to land on, but that are interesting to know about. These beans are useful, but not the intended final result. In these cases, I'll use a Spring *profile*, which is a marker annotation that makes the bean inert - Spring doesn't "see" it - until you explicitly *activate* the profile.

```
@Configuration
@Profile("example")
class MyConfig {

 @Bean
 Bar bar(){
   // ...
 }

 @Bean
 @Profile("foo")
 Foo foo(){
   // ...
 }
}
```

Usually, you activate the profile (or profiles) by passing in a comma-delimited command line switch, e.g.:

```
./mvnw -Dspring.profiles.active=example,foo spring-boot:run
```

In this example, both the profile for the configuration class, `example`, and the profile for the nested bean, `foo`, are active and thus both the configuration class and the specific beans inside of it will be

active. If the profile `example` is active, but not the `foo` profile, then the result would be that *only* the `bar` bean would be active, since it has no other profile.

As you work through the book, either remove the profile or activate it to ensure the relevant code runs when you run the example.

Generally, however, I try to avoid profiles for "real" code intended for production, as they make it easy to violate the 12-Factor principle of one build for multiple environments. And, there are some sticky issues with the use of profiles when building Spring AOT-powered applications. All in all, it's worth avoiding.

Nullability

Starting from Spring Framework 7 and Spring Boot 4, the entire Spring portfolio adopted the JSpecify.dev annotations [https://jspecify.dev]. These annotations define null restriction on references in the codebase. They give build tools like Apache Maven and Gradle, IDEs like IntelliJ IDEA, and compilers like Java and Kotlin feedback about when a `null` reference is being passed to a place that it should not be.

Sometimes, in the course of this book, I'll present interfaces or types from Spring that feature these annotations. Spring Security is a very good example of this. It's an old project whose origins assumed a world where all authentication attempts would include a password. Later, new ways to authenticate (and arguably more secure ones) developed and the password field in the original `Authentication` became *nullable*. That is, the password wasn't required because there might be other things in the request - a token, a certificate, etc. - that did the work of identifying the user and so the password wasn't needed.

Adopting these JSpecify.dev annotations has been helpful for even the Spring team because it helped us to see places where our suppositions around null references didn't universally hold true. Considering that Spring Security is more than 20 years old, it's not surprising that these occasional inconsistencies sometimes read their ugly head!

Often, in the Spring codebases, you'll see a `package-info.java` class that contains a single package declaration and `@NullMarked`, like this one from `org.springframework.context`:

```
@NullMarked
package org.springframework.context;

import org.jspecify.annotations.NullMarked;
```

`@NullMarked` says that unless shown otherwise, every reference in this package is meant to be non-`null`. Occasionally, you'll see such exceptions marked with `@Nullable` on specific fields.

Spring Initializr Project Configuration

In this book we'll look at a *lot* of code, and I'll always go to the Spring Initializr and tell you how to consturct a project of similar makeup. I will basically always choose Apache Maven and I will basically always choose the latest and greatest version of Java or the latest and greatest long-term

support (LTS) version of Java or Kotlin. At the time of this book's writing, the latest LTS is Java 25.

I may forget to stipulate the artifact ID. Use whatever you want there in this case.

If you're using the Spring Tools [https://spring.io/tools] for Visual Studio Code or Eclipse, or if you're using IntelliJ IDEA, you will have wizards inside of the IDE that'll guide you through the same steps as you would go through on the Spring Initializr. Indeed, those in-IDE wizards are frontends to the same webservice that powers the Spring Initializr, so the results are exactly the same. Feel free to use those, but for consistency, I'll refer to the Spring Initializr experience.

The Spring Initializr gives you a `.zip` file that you have to unzip. Inside you'll find a `pom.xml` (or `build.gradle` or `build.gradle.kts` if you go those routes). I usually then run IntelliJ IDEA: `idea pom.xml` from the command line.

I do this *so often* that I've even got a little Java script you can use.

```
// java ~/josh-env/bin/uao.java $HOME/Downloads/demo.zip
import java.io.*;
import java.util.function.*;
import java.util.* ;
void main(String [] args) throws Exception {
    var zipFile = new File(args[0]);
    var zipFileAbsolutePath = zipFile.getAbsolutePath();
    var folder = new File(zipFileAbsolutePath.substring(0,
zipFileAbsolutePath.lastIndexOf(".")));
    new ProcessBuilder().command("unzip", "-a",
zipFileAbsolutePath).inheritIO().start().waitFor();
    for (var k : Set.of("build.gradle", "build.gradle.kts", "pom.xml")) {
        var buildFile = new File(folder, k);
        if (buildFile.exists()) {
            new ProcessBuilder().command("idea",
buildFile.getAbsolutePath()).inheritIO().start().waitFor();
        }
    }
}
```

I've created an alias to that script in my shell called `uao`.

Your First Cup of Java

Are you using the latest version of Java yet?

No? That's a real pity, because it's basically a different language from the one you might've known. As of this writing, the latest Long-Term Support (LTS) release is Java 25. There are new releases of Java coming out every six months. By the time you read this, we could easily be on many versions beyond that. This book was written with Java 25 in mind, but you should be using whatever the latest-and-greatest version of Java available to you happens to be.

That said, I can appreciate that some of you may not have seen all that is new and nifty in the latest and greatest installments of Java, so we'll review some of my favorite features here. If you're all caught up, then please move on to the next chapters - there's entirely too much to look at, anyway.

I don't know what the reason is, but I hope you're able to move up and over soon because Spring Framework 6 and Spring Boot 3 assume a Java 17 baseline. That means that not only do those generations of Spring support Java 17 features (as did the Spring Framework 5 and Spring Boot generations), they also use some Java 17 features in their source code. This book uses Spring Framework 6 and Spring Boot 3.

Whatever the cause of your hesitation, this chapter contains *spoilers*! If you don't mind spoilers, let's dive right in!

First things first: you need an OpenJDK distribution that works for you. The list of valid and viable distributions scrolls down to the floor and out the door, so I'll refer you to Foojay.io's handy version almanac for Java 17 [https://foojay.io/almanac/java-17/]. One of my favorite tools for managing my Java installations on UNIX-like operating systems is `sdkman` [https://sdkman.io/]. It works on macOS, Windows Subsystem for Linux (WSL), Linux, and, I'm sure, plenty of other places besides. I use the GraalVM [https://www.graalvm.org/] distribution. GraalVM is an OpenJDK distribution with some extra features, including ahead-of-time native image compilation. To get that distribution, you might say:

```
sdk install java 25-graalce ①
sdk default java 25-graalce ②
```

① the first command installs the latest version of the GraalVM distribution of Java 25

② the second makes it the default installed distribution on my box so that all my interactions with Java are going to that distribution

With that done, let's look at some neat features in Java's latest and greatest versions up until Java 25.

Operations Improvements

The newer versions of Java now support something called CDS Archives. CDS Archives essentially capture some of the invariant (but freshly computed) state from a given application and cache it for easy reuse on subsequent runs. I consistently shave 0.1 or 0.2 seconds from startup time when using CDS Archives.

The newer versions of Java are also container-aware. So, let's suppose you are running Docker images on your host machine, which has 32GB of RAM. You might configure the JRE with 2GB of RAM and errantly configure the Docker container with only 1GB of RAM. Java would see the 32GB and think it could allocate 2GB, and then fail to start up. In newer versions, Java is aware of the container's limited RAM and won't exceed it.

Java Flight Recorder (JEP 328) monitors and profiles running Java applications. Even better, it does so with a low runtime footprint. Java Mission Control allows ingesting and visualizing Java Flight Recorder data. Java Mission Control takes Java's already stellar support for observability to the next level.

Performance

Java 25 is *fast* and reliable.

Java's garbage collector is the stuff of legend. It's fast, lightweight, and minimally invasive. It's also one of those things where, when it's improved, your application's runtime improves with it, for free. No recompilation is required.

G1 has been the default garbage collector since Java 9, replacing the Parallel garbage collector that had been the default in Java 8. G1 reduces pause times compared to Parallel, though its overall throughput may be lower. Next, Java 11 introduced the ZGC garbage collector to reduce pauses further. Finally, Java 14 introduced the Shenandoah GC, which keeps pause times low and does so in a manner independent of the heap's size.

And it's fast. I can't give you specific numbers or anything because it is so highly workload-sensitive, but large organizations like Netflix have talked about how more recent iterations in Java have helped them improve their workload performance merely by upgrading. Upgrading the JVM is one of the easiest ways to gain performance and reduce memory utilization, and it's one of the easiest, comparably, too.

AutoCloseable

Java manages most memory for you, but it can't be responsible for the state outside the humble confines of the JVM. So, for example, you wouldn't like it if Java *garbage-collected* your connection to the database, and you wouldn't like it if Java garbage-collected your open socket connections without warning. Therefore, the interfaces you use to interact with these external resources typically have a `close()` method that you, the client, need to call when your work with the external resource is finished.

You mustn't neglect to call that method! Don't be greedy! You're not greedy, are ya? You want to leave the JVM as clean as you found it. So, you write the boilerplate. We all know the boilerplate: open the resource; work with it; surround that work with a `try/catch/finally` block; catch any `Throwable` instances if (*when*?) something goes wrong; `close()` the resource if something goes wrong. Add the `close()` call in a `finally` block for extra robustness. All the while, handle the exceptions that might arise when you try to do anything, including calling `close()` on the resource. You'll end up with one or more `try/catch` blocks embedded within the outer `try/catch` blocks.

Let's look at some examples. I'll read a file on the filesystem to demonstrate this new feature. But something needs to ensure that there's a file to read in the first place, so I've extracted all that out into a separate class, `Utils`:

```
package com.example.java.closeable;

import java.io.File;
import java.nio.file.Files;
import java.time.Instant;

public class Utils {

①
    static String CONTENTS = String.format("""
            <html>
            <body><h1> Hello, world, @ %s !</h1> </body>
            </html>
            """, Instant.now().toString()).trim();

②
    static File setup() {
        try {
            var path = Files.createTempFile("bootiful", ".txt");
            var file = path.toFile();
            file.deleteOnExit();
            Files.writeString(path, CONTENTS);
            return file;
        } //
        catch (Throwable t) {
            error(t);
            throw new RuntimeException("could not produce a new file");
        }

    }

③
    static void error(Throwable throwable) {②
        System.err.println("there's been an exception processing the read! " +
throwable.getMessage());
    }

}
```

① the contents of the file, using a handy multiline Java `String`

② this method returns the `java.io.File` for the newly created temporary file

③ a convenience method to log errors. Do not rewrite this code! Otherwise, if you're doing things the old-fashioned way, you'll find yourself rewriting this a *lot*.

Now, let's look at an example of what *not* to do.

```
package com.example.java.closeable;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.io.*;

import static com.example.java.closeable.Utils.error;

class TraditionalResourceHandlingTest {

    private final File file = Utils.setup();①

    private static void close(Reader reader) { ③
        if (reader != null) {
            try {
                reader.close();
            } //
            catch (IOException e) {
                error(e);
            }
        }
    }

    @Test
    void read() {
        var bufferedReader = (BufferedReader) null; ②
        try {
            bufferedReader = new BufferedReader(new FileReader(this.file));
            var stringBuilder = new StringBuilder();
            var line = (String) null;
            while ((line = bufferedReader.readLine()) != null) {
                stringBuilder.append(line);
                stringBuilder.append(System.lineSeparator());
            }
            var contents = stringBuilder.toString().trim();
            Assertions.assertEquals(contents, Utils.CONTENTS);
        } //
        catch (IOException e) { ③
            error(e);
        } //
        finally { ④
            close(bufferedReader);
        }
    }

}
```

① from which file are we reading?

② We need a reference to the `BufferedReader` outside of the scope of the `try/catch` block, but we don't want to initialize that reference until we're inside the `try/catch` block because it might incur an exception.

③ handle any errors. Bear in mind that this solution doesn't even attempt to field the errors and somehow recover. If there is *any* error anywhere, then we abort.

④ also, make sure to close that `Reader`! Err, close that reader *if* it's not `null`! Sorry, close that reader *if* it's not `null`, and don't forget to handle any errors that arise while doing so, either! Be kind, rewind!

Yuck. We can do better with Java 7's `try-with-resources` construct. Let's rework the example accordingly.

```
package com.example.java.closeable;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;

class TryWithResourcesTest {

    private final File file = Utils.setup();

    @Test
    void tryWithResources() {
        try (var fileReader = new FileReader(this.file); //
                var bufferedReader = new BufferedReader(fileReader)) {
            var stringBuilder = new StringBuilder();
            var line = (String) null;
            while ((line = bufferedReader.readLine()) != null) {
                stringBuilder.append(line);
                stringBuilder.append(System.lineSeparator());
            }
            var contents = stringBuilder.toString().trim();
            Assertions.assertEquals(contents, Utils.CONTENTS);
        } //
        catch (IOException e) {
            Utils.error(e);
        }
    }

}
```

Type Inference

Java 10 introduced type inference for variables given enough context. As a result, Java is a strongly typed language with less typing - pressing of keys - required.

```
package com.example.java.typeinference;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.Map;

class TypeInferenceTest {

    @Test
    void infer() throws Exception {
        var map1 = Map.of("key", "value"); ①
        Map<String, String> map2 = Map.of("key", "value");
        Assertions.assertEquals(map2, map1); ①
        var anonymousSubclass = new Object() {
            final String name = "Peanut the Poodle";

            int age = 7;

        };
②
        Assertions.assertEquals(7, anonymousSubclass.age);
        Assertions.assertEquals("Peanut the Poodle", anonymousSubclass.name);
    }

}
```

① both variable definitions contain a type of `Customer`. The compiler knows it, and you know it because the right side of the expression makes it clear. So, all things being equal, why not use the more concise version?

② Indeed, in this case, the compiler knows *more* about the type than it would before, particularly in the case of anonymous subclasses. Suppose you need a throwaway object in which to stash some data temporarily. If you created an anonymous subclass of `Object` and assigned it to a variable of type `Object`, there'd be no way to dereference fields defined on the anonymous subclass. Traditionally, there was no way to reify *anonymous* subclass types because they were *anonymous*. But with `var`, you don't need to account for the type; you can let the compiler infer it.

This last bit - reifying anonymous subclasses - comes in particularly handy when you're working with Java 8 streams abstractions and want to avoid having a host of throwaway classes created as side effects to conduct your data across the transformations.

The new `var` keyword can come in handy in a whole host of other smaller scenarios. You can use the `var` keyword for parameters in a lambda definition. It doesn't buy you much over just omitting

`var` (or the type itself), except that you can now decorate the parameter with annotations. Neat!

Lambdas are the one fly in the proverbial ointment, however. Java doesn't have structural lambdas like those in Scala, Kotlin, and other languages. Instead, you must assign the lambda to an instance of a functional interface like `java.util.function.Consumer<T>`. If the literal lambda syntax doesn't clarify what type that is, then the variable type definition itself must. So you can use `var` for every variable definition *except* lambdas. It's so dissatisfying! The agony of having a column of nice, clean `var`s punctuated occasionally by standard variable definitions just because those variables happen to be lambdas! There is a way around it with casts, but I admit it isn't much better. Let's look at some examples.

```
package com.example.java.typeinference;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.HashSet;
import java.util.List;

class LambdasAndTypeInferenceTest {

    private String delegate(String s, Integer two) {
        return "Hello " + s + ":" + two;
    }

    @Test
    void lambdas() {
        MyHandler defaultHandler = this::delegate; ①
        var withVar = new MyHandler() { ②

            @Override
            public String handle(String one, int two) {
                return delegate(one, two);
            }
        };
        var withCast = (MyHandler) this::delegate; ③
        var string = "hello";
        var integer = 2;
        var set = new HashSet<>( //
                List.of(withCast.handle(string, integer), //
                        withVar.handle(string, integer), //
                        defaultHandler.handle(string, integer)));
        Assertions.assertEquals(1, set.size(), "the 3 entries should all be the same,
and thus deduplicated out");
    }

    @FunctionalInterface
    interface MyHandler {

        String handle(String one, int two);
```

```
    }

}
```

① I can't use `var` here because the compiler doesn't know to which functional interface type the method reference, `delegate(String, Integer)`, should be assigned

② I can use `var` here, but I've lost all the brevity of lambdas!

③ the only way I know around it is to cast the type like this. Ugh.

I tend to use the cast form a lot. Your sensibilities may vary, and given how new this feature is, I wouldn't be surprised if my sensibilities change in the future with respect to this feature, as well.

Enhanced Switch Expressions

Java has a new *enhanced* switch, which forms the basis of the gradual addition of *pattern matching* to the Java language. It simplifies things considerably:

```
package com.example.java.switches;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

class TraditionalSwitchExpressionTest {

    @Test
    void switchExpression() {
        Assertions.assertEquals(respondToEmotionalState(Emotion.HAPPY), "that's
wonderful.");
        Assertions.assertEquals(respondToEmotionalState(Emotion.SAD), "I'm so sorry to
hear that.");
    }

    public String respondToEmotionalState(Emotion emotion) {
        var response = ""; ②
        switch (emotion) {
            case HAPPY:
                response = "that's wonderful.";
                break; ③
            case SAD:
                response = "I'm so sorry to hear that.";
                break;
        }

        return response;
    }

    enum Emotion {
```

```
①
        HAPPY, SAD

    }

}
```

① `Emotion` is an `enum`. There are only two possible values (for this simple example that doesn't at all reflect the rich gradient of human emotions) for a variable of type `Emotion`: `HAPPY` and `SAD`. We have branches for all known states in this `switch` statement. The compiler is satisfied that we have covered every possible value, so it doesn't insist on a `default` branch to handle any unforeseen values. There are no unforeseen values. We say that we've *exhausted* the range of values.

② we define a variable to which we assign the results of our processing.

③ take care to `break` for each branch. Otherwise, the execution flow will drop down to other branches, producing undesirable side effects.

The first example is a classic `switch` statement. There's nothing wrong, *per se*, but it's tedious, and I tend to avoid writing them.

```
package com.example.java.switches;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

class EnhancedSwitchExpressionTest {

    @Test
    void switchExpression() {
        Assertions.assertEquals("that's wonderful.",
respondToEmotionalState(Emotion.HAPPY));
        Assertions.assertEquals("I'm so sorry to hear that.",
respondToEmotionalState(Emotion.SAD));
    }

    private String respondToEmotionalState(Emotion emotion) { ①
        return switch (emotion) {
            case HAPPY -> "that's wonderful.";
            case SAD -> "I'm so sorry to hear that.";
        };
    }

    enum Emotion {

        HAPPY, SAD;

    }
```

```
}
```

① there's no intermediate variable! Each branch produces a value, and that value is the result of the `switch` expression and can be assigned to a variable or, as I do here, returned in one fell swoop from the method as any other expression would.

Multiline Strings

This one is a super-small feature that punches well above its weight: Java supports multiline `String`s. Hooray! There are a million opportunities for multiline `String`s: SQL queries, HTML, Markdown, Asciidoctor, Velocity templating (such as for emails), unit testing, etc.

```
package com.example.java;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.time.Instant;

class MultilineStringsTest {

    private final String instant = Instant.now().toString();

    ①
    private final String multilines = String.format("""
            <html>
            <body>
            <h1> Hello, world, @ %s!</h1> </body>
            </html>
            """, instant).trim();

    ②
    private final String concatenated = "<html>\n<body>\n" + "<h1> Hello, world, @ " +
instant + "!</h1> </body>\n"
            + "</html>";

    @Test
    void stringTheory() {
        Assertions.assertEquals(this.multilines, this.concatenated);
    }

}
```

① The first example uses a multiline `String` to represent some HTML markup

② The second variable recreates the same HTML markup, down to the padding and the newlines, but uses string concatenation and manually encodes newlines, as we used to have to do.

In the example, the two variables `multilines` and `concatenated` are identical, but the multiline `String` is much easier to wrangle.

Short and simple.

Records

Records are perhaps my second-favorite new feature in Java. If you've ever used case classes in Scala, or data classes in Kotlin, you'll feel *almost* right at home.

Java introduced records, which are a new kind of type. Records, like enums, are a restricted form of a class. As a result, they're ideal for "plain data carriers": classes containing data not meant to be altered, with only the most fundamental methods, such as constructors and accessors. We'll use (and sometimes abuse) them *all* the time in this book. They're wonderful time savers. Need to model a read-only entity in your database that has accessors for all the constituent fields, a constructor, a functional `toString` implementation, a valid `equals` implementation, and a valid `hashCode` implementation? You'd better get codin'! That'll take a while. Or, you could use something like Lombok to code-generate all of that for you based on the presence of a handful of annotations. Or, you could use records.

Here's a trivial example. Suppose you want to return information about `Customer` entities *and* their associated `Order` data. Unfortunately, Java doesn't support multiple return types or provide suitable tuple types, so you need to create something to hold both types. Records to the rescue!

```
package com.example.java.records;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.List;

class SimpleRecordsTest {

    @Test
    void records() {
        var customer = new Customer(253, "Tammie");
        var order1 = new Order(2232, 74.023);
        var order2 = new Order(9593, 23.44);
        var cos = new CustomerOrders(customer, List.of(order1, order2));
        Assertions.assertEquals(2232, order1.id());
        Assertions.assertEquals(74.023, order1.total());
        Assertions.assertEquals("Tammie", customer.name());
        Assertions.assertEquals(2, cos.orders().size());
        IO.println("order components  " + order1.id() + ':' + order1.total()); ②
    }

    ①
    record Customer(Integer id, String name) {
    }

    record Order(Integer id, double total) {
    }
```

```
    record CustomerOrders(Customer customer, List<Order> orders) {
    }

}
```

① these three `record` definitions define three new types, each with accessors, constructors, and internal storage. Scarcely more than three lines of code for three brand new types!

② records also automatically expose accessor methods for the fields defined in the constructor. For example, if you want to read the `name` field of the `Customer` type, use `Customer#name()`. If you wish to read the `orders` field of the `CustomerOrders` type, use `orders()`. It took a while to accept that they're in the form `x()`, and not in the form `getX()`, but I've come to love it. Mercifully, all the interesting libraries that need to know about this convention - JSON serializers, for example - already work well with it.

Records make perfect sense for immutable, data-centric types. They alleviate a whole host of boilerplate code. In the first edition of this book, I used the Lombok project [https://projectlombok.org] (which is brilliant) to synthesize the getters, setters, no-args, and all-arg constructors with just a few annotations. It worked, but it was still a handful of lines instead of the one-liners enabled by Java records. I love records!

I still occasionally use Lombok for other things, but it's nice to reduce my reliance on it further.

More controversially, I also sometimes use records to implement services and components quickly. After all, you can have methods on a record. Of course, record implementations can't extend classes, but one doesn't need to do that a lot. There is the undesirable side effect of having accessor methods that expose the state - `dataSource()` or whatever - but, for whatever reason, I don't care. It doesn't cost me anything when I use it. If my code grows large enough to need hierarchies or interface implementations, I'll change it. If the code grows enough that I need to worry about leaking state, I'll change it. But the immediate, short-term effect of having more concise, approachable, readable code seems to make sense to me. Maybe one day that'll change?

Behind the scenes, a record creates a default constructor whose parameters match the types and arity of the record header. Records can have other constructors, but they need to delegate to the default constructor.

```
package com.example.java.records;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.springframework.util.Assert;
import org.springframework.util.StringUtils;

class RecordConstructorsTest {

    @Test
    void multipleConstructors() {
        var customer1 = new Customer("test@email.com");
        var customer2 = new Customer(2, "test2@gmail.com");
```

```
        Assertions.assertEquals(customer1.id(), -1);
        Assertions.assertEquals(customer2.id(), 2);
    }

    record Customer(Integer id, String email) { ①

        Customer { ②
            Assert.notNull(id, () -> "the id must never be null!");
            Assert.isTrue(StringUtils.hasText(email), () -> "the email is invalid");
        }

        Customer(String email) {
            this(-1, email);
        }
    }

}
```

① the default constructor is the one defined in the record header

② If you want to act on the fields passed in the default constructor, create a constructor with no parameters and do the work there. This no-parameter constructor and the record header taken together are the default constructor. If you want to have an alternative constructor, you can do that so long as you forward to the default constructor. I use the default constructor to initialize the `id` to -1. This way, it's impossible to initialize the record with a `null` `id` field.

Sealed Types

Sealed types are a novel feature that hasn't reached its full potential yet. The basic idea is to constrain the number of subtypes for a given type. Why would you do this? Well, it's all a bit *exhausting*! Or should I say, it's all about exhausting the extent to which the runtime needs to support virtual (polymorphic) dispatch?

Sound complicated? It's not really. Ever have a method in a parent type that the child type overrides? The ability to call that method on an instance of that child type, in terms of the parent type's interface, is called polymorphism.

Polymorphic, or *virtual*, dispatch requires a lookup in the virtual function table, which in theory takes time. The runtime wouldn't have to do that if it could say conclusively that a given type can never be subclassed, such as with a final type. The `final` keyword is a bit of a sledgehammer, however. It forecloses entirely on the possibility of any kind of subclassing whatsoever. You might have a hierarchy but wish to keep it shallow and well known. Alternatively, you could make everything package-private (simply omit the `public` modifier on the class), which means that only types in the same package could subclass the type. This would probably work, but of course, somebody else could come along and create types in the same package in their `.jar`. There haven't traditionally been many good ways to keep a hierarchy shallow - until now. Sealed types can help. They let you constrain the number of subclasses to a known set.

Let's look at an example.

```
package com.example.java;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.springframework.util.Assert;

class SealedTypesTest {

④
    private String describeShape(Shape shape) {
        Assert.notNull(shape, () -> "the shape should never be null!");
        if (shape instanceof Oval)
            return "round";
        if (shape instanceof Polygon)
            return "straight";
        throw new RuntimeException("we should never get to this point!");
    }

    @Test
    void disjointedUnionTypes() {
        Assertions.assertEquals(describeShape(new Oval()), "round");
        Assertions.assertEquals(describeShape(new Polygon()), "straight");
    }

①
    sealed interface Shape permits Oval, Polygon {

    }

②
    static sealed class Oval implements Shape permits Circle {

    }

③
    // static final class Rhombus implements Shape {}

    static final class Circle extends Oval {

    }

    static final class Polygon implements Shape {

    }

}
```

① we'll start with a `Shape` and permit two direct subclasses, `Oval` and `Polygon`.

② a subclass of a sealed type must be `final` or `sealed` and explicitly name its subclasses. A sealed

type's subclasses may themselves be sealed types, permitting further subtypes.

③ the following declaration does not compile, as it is not one of the explicitly permitted subclasses

④ the `describeShape` method is written so that we can exhaustively handle every subclass.

Sealed types help the compiler, too. The compiler can exhaustively determine every case of every possible subtype, which has implications for the future, as Java looks to better incorporate simple pattern matching into the language. Imagine the possibilities here, and you can kind of see how sealed types might play with the new `switch` expressions, too.

Right now, I don't recommend using `sealed` types. I tend to think types should be open by default. You just don't know what scenario will arise in the future that changes your fundamental assumptions. Furthermore, sealed subtypes are `final`, inhibiting tools like Spring and Hibernate, which must subclass your types to proxy them.

Smart Casts

This feature is one of those things that I don't use all that often in application code, but it's pretty useful when writing infrastructure code that deals with polymorphism. Essentially, the feature spares you from having to create an intermediate variable and casting after you've already done an `instanceof` check.

```
package com.example.java;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.HashSet;

public class SmartCastsTest {

    @Test
    void casts() {
        interface Animal {

            String speak();

        }

        class Cat implements Animal {

            @Override
            public String speak() {
                return "meow!";
            }

        }

        class Dog implements Animal {
```

```
            @Override
            public String speak() {
                return "woof!";
            }

        }

        var newPet = Math.random() < .5 ? new Cat() : new Dog();
        var messages = new HashSet<String>();

①
        if (newPet instanceof Cat) {
            var cat = (Cat) newPet;
            messages.add("the cat says " + cat.speak());
        }
②
        if (newPet instanceof Cat cat) {
            messages.add("the cat says " + cat.speak());
        }

        if (newPet instanceof Dog dog) {
            messages.add("the dog says " + dog.speak());
        }

        Assertions.assertEquals(messages.size(), 1);
        Assertions.assertTrue(messages.contains("the dog says woof!") || messages
.contains("the cat says meow!"));

    }

}
```

① this is a classic example, where we determine the subtype and then create a variable cast to the appropriate type. If ever we change the type definition, we have to replace it both in the cast and in the `instanceof` check.

② the newer, more elegant alternative uses a smart-cast, sparing us the extra variable and cast.

This feature looks almost exactly like a similar feature in Kotlin, and I love it.

Functional Interfaces, Lambdas, and Method References

Java 8 introduced lambdas. They let us treat functions as first-class citizens that can be assigned and passed around. Well, almost. In Java, lambdas are slightly limited; they're *not* first-class citizens. But, close enough! They *must* have return values and input parameters compatible with the method signature of the sole abstract method of an interface. This interface, one with a single abstract method intended for use as a lambda, is called a *functional interface*. (Interfaces may also have *default*, non-abstract methods where the interface provides an implementation.)

There are some very convenient and oft-used functional interfaces in the JDK itself. Here are some of my favorites.

- `java.util.function.Function<I, O>`: an instance of this interface accepts a type `I` and returns a type `O`. This is useful as a general-purpose function. Every other function could, in theory, be generalized from this. Thankfully, there's no need for that as several other handy functional interfaces exist.
- `java.util.function.Predicate<T>`: an instance of this interface accepts a type `T` and returns a `boolean`. This is an ideal filter when you're trying to sift through a stream of values.
- `java.util.function.Supplier<T>`: A supplier accepts no input and returns an output.
- `java.util.function.Consumer<T>`: A consumer is the mirror image of a `Supplier<T>`; it takes an input and returns no output.

You can also create and use your own custom functional interfaces. Let's look at some examples.

```
package com.example.java;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.function.Function;

class LambdasTest {

    @Test
    void lambdas() {
        Function<String, Integer> stringIntegerFunction = str -> 2;①
        interface MyHandler {

            String handle(String one, Integer two);

        }
        MyHandler withExplicit = (one, two) -> one + ':' + two;②
        Assertions.assertEquals(stringIntegerFunction.apply(""), 2);
        Assertions.assertEquals(withExplicit.handle("one", 2), "one:2");
        var withVar = (MyHandler) (one, two) -> one + ':' + two;③
        Assertions.assertEquals(withVar.handle("one", 2), "one:2");
        MyHandler delegate = this::doHandle; ④
        Assertions.assertEquals(delegate.handle("one", 2), "one:2");

    }

    private String doHandle(String one, Integer two) {
        return one + ':' + two;
    }

}
```

① You can use existing functional interfaces in the JDK…

② you can define your own interfaces and use them as functional interfaces

③ You may use `var` and lambdas together with this (admittedly unsightly) cast.

④ and if you have existing methods whose return types and input parameters line up with the single abstract method of a functional interface, then you can create a method reference and assign it to an instance of that type.

Pattern Matching

We've seen the smart-casting `if` statement, and we've seen the switch expression. At the heart of both of these new things is *pattern matching*, wherein we match a value and then extract the value for use. We've only really seen that it's possible to easily match values thus far, so let's revisit that a bit.

```
package com.example.java.patternmatching;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertTrue;

class PatternMatchingTest {

①
    final String authenticateMessage = """
                please authenticate, so that we can
                get to know you a little better.
            """.stripIndent().stripLeading().stripTrailing();

    final String customerMessage = "the customer's name is %s";

②
    String showMessageWithIf(BrowserClient browserClient) {
        var message = "";
        if (browserClient instanceof Customer(var id, var name)) {
            message = this.customerMessage.formatted(name);
        }
        else if (browserClient instanceof UnknownUser) {
            message = this.authenticateMessage;
        }
        return message;
    }

③
    String showMessageWithSwitch(BrowserClient browserClient) {
        return switch (browserClient) {
            case Customer(var id, var name) -> this.customerMessage.formatted(name);
            case UnknownUser() -> this.authenticateMessage;
        };
```

```
    }

    @Test
    void patternMatching() throws Exception {
        assertTrue(showMessageWithIf(new Customer(1, "Josh")).contains(this
.customerMessage.formatted("Josh")));
        assertTrue(showMessageWithSwitch(new UnknownUser()).contains(this
.authenticateMessage));
    }

④
    sealed interface BrowserClient permits UnknownUser, Customer {

    }

    record UnknownUser() implements BrowserClient {
    }

    record Customer(Integer id, String name) implements BrowserClient {
    }

}
```

① for our examples, we're going to work with a shallow hierarchy rooted in a type called `BrowserClient`. Imagine that when an HTTP request comes in, we adapt the request into one of either a known `Customer` or an `UnknownUser`.

② we'll do some analysis of the incoming user type and display a message as appropriate. Since we're going to prepare these same messages two different ways, I've extracted them out into separate variables.

③ let's use the smart `if` statement to deduce which message to present. Note that we're now switching on an instance of `BrowserClient`. In the first line, we do two things at once: we check that the variable is an instance of `Customer`, and if so, we hoist out - we *extract* - not just the `Customer` variable, but the constituent component fields of the record itself - `id` and `name` - into their own new variables. Super concise. In the second test, we don't need a pointer to the `UnknownUser` or its field, so we do nothing with it save match it.

④ the `if/else` structure worked fine, but it meant that we had to stash a variable and then make sure that only one branch of the test was ever evaluated. The structure as written also gives us nothing in the way of comfort. What happens when we add a new type to the hierarchy? We need to handle that branch, but the compiler will let us ignore it. Let's use our new friend the `switch` expression. The switch expression has access to the same pattern-matching superpowers as the `if`, but it does us two new favors. The compiler will complain if we add a new type to the hierarchy, because it knows that the types are `sealed` and therefore there is an exhaustive set of types, and our `switch` hasn't defined a `default` branch. Thanks, compiler! And, to make things just that much cleaner, the `switch` is an expression, so we can assign the result of the `switch` to a variable or just return the result of the expression directly.

I bet you never thought you'd see the day that the `switch` was more elegant than an `if`, did you?

Java language architect Brian Goetz talks about the combination of some of the features that we have just covered - pattern matching, sealed types, the smart switch expression - as *data-oriented programming*. Java has reigned supreme in large monolithic codebases, he explains, because it provides good object-oriented support and strong enforcement of privacy, security, and encapsulation. In such a codebase, the boundary on which different parts of the program align is typically an object interface, which the magic of polymorphism incentivizes. But programs aren't usually large, sprawling, deeply rooted graphs of objects these days. More often than not, they're smaller programs dealing with ad hoc messages coming in over the wire and being handled in terms of dumb, but typed, carriers. There's no hierarchy at all! In this context, Java 8 was starting to feel mighty clunky. But no more. With Java 25, it's trivial to define small ad hoc objects and to vary behavior based on them. In a sense, Java 25 introduces a new kind of dispatch: not the dynamic dispatch of yore, but a dispatch for small ad hoc objects.

And just like that, without our ever realizing it, Java 25 manages to introduce support for a completely new style of programming: data-oriented programming. And I bet you were only just getting used to the possibilities of the rudimentary functional programming support in Java 8!

Java 25, Project Loom, and Virtual Threads

Project Loom brings transparent fibers to the JVM. As things stand today, in Java 20 or earlier, IO is blocking. Call `int InputStream#read()`, and you might have to wait for the next byte to finally arrive. In `java.io.File`-centric IO, there's very rarely much of a delay. On the network, however, you just can't really ever know. Clients might disconnect. Clients are people, driving through a tunnel and getting spotty signal coverage. During this time, the program flow is said to be *blocked* from proceeding on the thread of execution. In the following snippet, we have no way of knowing when we'll see the word "after" printed. Might be a nanosecond from now. Might be a week from now. It's *blocking*.

```
Executor executor = Executors.newCachedThreadPool();
executor.submit(() -> {
    InputStream in = ...
    IO.println("before");
    int next = in.read();
    IO.println("after");
});
```

This means that we can't address any other requests with this thread until it's finished doing whatever it was doing. We'll need more threads.

This is bad enough, but it's made worse by the architecture of threading in Java prior to Java 25, where each thread maps, more or less, to a native operating system thread. It's expensive to create more threads, too, taking about two megabytes of RAM. We're going to need a *lot* of threads to handle requests if our existing threads are blocked long enough. Many, many more threads. Or, we're going to need to find a way to avoid *blocking* on the precious few threads we do have.

We could use non-blocking IO, as enabled by Java NIO (java.nio.*). In this model, you ask for bytes and register a callback that the runtime executes only when there are actually bytes available. No waiting, no blocking. This approach has the significant benefit of keeping us off threads when

there's nothing to be done, allowing others to use those threads in the meantime. It's a bit tedious, however, and low-level. Spring has amazing support for reactive programming, which offers a functional-style programming model on top of non-blocking IO. It works well. But, it requires changing the way you write code. If you're not used to it, it can be a bit daunting, too. Wouldn't it be nice if you could just take that existing code, as demonstrated above, and have it do the right thing, transparently moving the flow of execution off the thread when there's nothing happening, and then resuming the flow of execution when there is? This way, we could keep our simpler code and still get the benefits of non-blocking IO. Absolutely it would. And now you can. Project Loom is straightforward at the high levels: if you run your code in a *virtual thread* (just another type of `java.lang.Thread`), then the runtime will detect *blocking* operations like `InputStream#read()`, `Thread.sleep(long)`, etc., and automatically move the code off the thread it's on if it's in a state where nothing is happening. It'll move the code back onto the thread once there's something to be done, once the blocking operation has stopped blocking. In the case of reading, that means it'll resume once there are bytes to read. In the case of `Thread.sleep`, it'll resume once the sleep time has elapsed. And you don't have to do anything to get it to work. Let's look at an example that I shamelessly stole from Oracle Java advocate José Paumard.

```
package com.example.java.loom;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;
import java.util.concurrent.ConcurrentSkipListSet;
import java.util.stream.IntStream;

class LoomTest {

    private static Set<String> observe(int index) {
        var before = Thread.currentThread().toString();
        try {
            Thread.sleep(100);
        } //
        catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        var after = Thread.currentThread().toString();
        return index == 0 ? new HashSet<>(Arrays.asList(before, after)) : Set.of();
    }

    @Test
    void threads() throws Exception {
        var switches = 5;
        var observed = new ConcurrentSkipListSet<String>();
        var threads = IntStream.range(0, 1000)①
            .mapToObj(index -> Thread.ofVirtual()②
                .unstarted(() -> {
```

```
                    for (var i = 0; i < switches; i++) ③
                        observed.addAll(observe(index));
                }))
            .toList();
        for (var t : threads)
            t.start();
        for (var t : threads)
            t.join();
        Assertions.assertTrue(observed.size() > 1); ④
    }

}
```

① This program launches 1,000 virtual threads. This is an arbitrary number, but it's also enough that it'll create contention - a need to share the precious few resources available to the machine.

② the program sleeps five times in each thread, for about 100 milliseconds each. This is not a long time, but it's enough that it forces the runtime to have to interleave, to *share*, the actual operating system threads.

③ In each thread, we call the `observe` function, which notes the current thread, sleeps, and then notes the current thread again. We don't want all the threads, however, as that would be too many. So we sample: if we're on the first thread launched (whose index would be `0`), note the information; otherwise, do nothing with it and move on.

④ by the end of the run, we've captured the threads on which the first of our thousand virtual threads executed, and stored them in a `Set<String>`, where the values will be deduplicated. Interestingly, there is *probably* more than one value in the set! Remember, we only noted the thread of execution for *one* thread - the *same* thread! - and somehow, without our asking or doing anything, it got moved around. Print the values out, and you'll see the code ran on the same *virtual* thread, but on multiple carrier threads.

The result is that while we were `Thread.sleep`-ing, the carrier thread on which we were executing was made available to whatever else required it in the JVM. This efficient time-sharing means that now we can handle a *lot* more requests with the same codebase. All we had to do was make sure to run the code on a *virtual* thread.

You don't have to use the `Thread.ofVirtual()` construction, either. There's an `Executor` implementation, too: `Executors.newVirtualThreadPerTaskExecutor()`. It's not a pool, however. Threads are no longer expensive, after all.

That's easy enough. In a typical Spring Boot application, however, there are countless places where you'll need to override the default configured `Executor`, but in Spring Boot 3.2 we can do that for you. Specify `spring.threads.virtual.enabled=true`, and you're all set.

Loom is especially important in the context of the Spring Authorization Server, because it uses traditional blocking IO. If you want to easily scale it up, consider using virtual threads and Java 25 when running the Spring Authorization Server. Who knows, you may not need to add new instances. You'll get to have your cake and eat it too: scalable, but simpler, code.

Improving Startup with Project Leyden

Project Leyden is a set of initiatives from the OpenJDK team to improve Java's performance by evaluating more and more about a program's structure earlier, at compilation or as early as possible. This effort is ongoing, but has already borne fruit. You can use Project Leyden with a Spring application to great effect. It provides a dramatic improvement on startup time for *all* Java application workloads.

Improving Startup and RAM footprint with GraalVM Native Images

We're using the GraalVM distribution of OpenJDK. It is not strictly speaking the same as OpenJDK. It releases more slowly than OpenJDK, for example. GraalVM offers some alluring capabilities, including a polyglot language runtime called Truffle, and the capability to turn Java programs into lightweight, rapid, architecture- and operating system-specific native images. The startup time is akin to or better than Project Leyden, and the RAM reductions could be quite considerable. GraalVM does require some compromises. The resulting binaries are *not* strictly speaking Java programs anymore and must give up compatibility in certain cases.

Next Steps

I didn't intend for this chapter to be a thorough introduction to all things Java. Just an amuse-bouche for all the cool stuff you might see me do in my code independent of Spring. Java 25 is a very compelling way to write code, and it's no coincidence that it's starting to look more and more like languages like Kotlin, which is also a lovely way to write Spring Boot-based applications. Hopefully, this chapter has been persuasive, and you're ready to turn the page. :code: ..

From Beans to Boot

I am assuming that by the time you're reading this, you have some understanding of Spring. I assumed the same thing about one of my earlier books, *Reactive Spring*, too, but I included a *very* simple discussion of some of the core principles of Spring in that book and—to my utter shock and chagrin—it was one of the most loved aspects of the book! I couldn't believe it!

So, here we go. We're going to do the same sort of thing, but of course greatly updated to reflect the amazing moment in which we find ourselves.

When Spring first came out, we talked about the "Spring triangle," the three pillars that made the Spring Framework so useful for people:

- portable service abstractions
- dependency injection
- aspect-oriented programming (AOP)

More recently, with the introduction of Spring Boot, we've added a new dimension of utility to the framework:

- autoconfiguration

So I guess it's more like a "Spring square" now, come to think of it.

At the core of Spring as a framework is the fact that it needs to understand how your objects are wired together. But why? Why bother telling it these things? Why not just wire things yourself?

At the micro level, this isn't an unreasonable question. Spring is just Java, and I can write Java without Spring, so why bother with Spring? Let's learn what Spring can help you with by way of an example that we'll start from scratch, iteratively letting Spring do more and more to reduce code and elevate results.

But first, we'll need a project and a database…

The Spring Initializr

We will need to incept a new project. It's just Java and some libraries, so in theory you could create your own with no issues. But I like to use the Spring Initializr (https://start.spring.io) [https://start.spring.io]. Visit the Spring Initializr and make the following dependency selections: `Spring Boot Actuator`, `Thymeleaf`, `Spring Data JDBC`, `PostgreSQL Driver`, `Spring Web`, `GraalVM Native Support`, `OpenTelemetry`, and `Spring Boot DevTools`. Select Java 25 (or later), and Apache Maven as the build tool. You could use Gradle if you prefer, though I'll be showing Apache Maven.

Give the application whatever group name and artifact name you like. Click `Generate`. You'll download an application archive (`.zip`). Unzip it and then point your IDE—IntelliJ IDEA, Visual Studio Code, Eclipse, etc.—at the `pom.xml` in the root of the project.

You should have a project with the usual folder structure: `pom.xml` and `src/{test,main}/{java,resources}`. You'll also have a new class in the `src/main/java` folder with a

`public static void main` method.

Rename the file `Main.java`, and gut the code inside. It should look something like this when you're done:

```
package com.example.beans_to_boot;

public class Main {

    public static void main(String[] args) {
    ①
    }
}
```

1. A *tabula rasa*!

It is on this humble foundation that we build.

The Database

Let's introduce a *trivial* example to reinforce the idea. We're going to read `Dog` data from a SQL database, PostgreSQL, running on our machine. We'll use the following Docker Compose file to stand up the instance of our database.

```
services:
  ①
  postgres:
    image: 'postgres:latest'
    environment:
      - 'POSTGRES_DB=mydatabase'
      - 'POSTGRES_PASSWORD=secret'
      - 'POSTGRES_USER=myuser'
    ports:
      - '5432:5432'

  ②
  grafana-lgtm:
    image: 'grafana/otel-lgtm:latest'
    ports:
      - '3000:3000'
      - '4317:4317'
      - '4318:4318'
```

1. This will stand up a PostgreSQL database.
2. ...and a Grafana instance. We'll return to this later.

Run this in the usual way: `docker compose up -d`

Put the following SQL files in `src/main/resources/schema.sql` and `src/main/resources/data.sql` relative to your project root.

Here's `schema.sql`:

```
create table if not exists dog
(
    id          serial primary key,
    name        text not null,
    description text not null
);
```

and here's `data.sql`:

```
delete from dog;
insert into dog (name, description)
values ('Prancer', 'A cute dog');
insert into dog (name, description)
values ('Duke', 'Oh the adventures you will have!');
insert into dog (name, description)
values ('White Dog', 'Oh the adventures you will have!');
```

Run `docker compose up -d`, then make sure to pass that SQL to PostgreSQL:

```
cat schema.sql | PGPASSWORD=secret psql -Umyuser -hlocalhost mydatabase
cat data.sql | PGPASSWORD=secret psql -Umyuser -hlocalhost mydatabase
```

A Repository of Knowledge… About Dogs

We're going to be working with data about dogs, so we'll need a layer to read that data and make it available to us. Here's a strawman first cut at this code, with several less-than-ideal design decisions. Let's look at the moving parts. We'll introduce all of them now and only the things that change as we progress.

First, the holder for data coming from our `dog` table.

```
package com.example.beans_to_boot.raw;

record Dog(int id, String name, String description) {
}
```

1. A Java record is a marvelous thing.

We'll be implementing a repository. Here's the interface for that repository.

```
package com.example.beans_to_boot.raw;

import java.util.Collection;

interface DogRepository {

    Collection<Dog> findAll();

}
```

It seems simple enough...

And here's the default implementation of that interface.

```
package com.example.beans_to_boot.raw;

import org.springframework.jdbc.datasource.DriverManagerDataSource;

import javax.sql.DataSource;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Collection;

class DefaultDogRepository implements DogRepository {

    private final DataSource db;

    DefaultDogRepository() {
        ①
        this.db = new
DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase", "myuser", "secret");
    }

    @Override
    public Collection<Dog> findAll() {
        ②
        try (var connection = this.db.getConnection(); //
                var ps = connection.prepareStatement("select * from dog") //
        ) {
            var list = new ArrayList<Dog>();
            try (var rs = ps.executeQuery()) {
                while (rs.next())
                    list.add(new Dog(rs.getInt("id"), rs.getString("name"),
rs.getString("description")));
            }
            return list;
        } //
        catch (SQLException throwable) {
            throw new RuntimeException(throwable);
        }
```

```
    }

}
```

1. Everything about this arrangement makes me itch: `DataSource` objects are expensive and should be pooled, centralized, and reused for many objects. I don't love credentials hardcoded into the source code. And I don't love that we're instantiating the expensive resource in the constructor instead of centralizing its definition and having it passed in via injection.
2. Believe it or not, this demo used to be a *lot* worse when I first started teaching Spring to folks (even if just others on my team) 20+ years ago. The ability to use Java 7's `AutoCloseable` is a game-changer for this sort of code. Just hope you remember to use it! You definitely don't want these things leaking.

And of course here's the entry point into this example. All `Main` classes will have the same basic formula: use an instance of `DogRepository` to read data and iterate over the data.

```
package com.example.beans_to_boot.raw;

public class Main {

    static void main(String[] args) throws Exception {
        var dogRepository = new DefaultDogRepository();
        test(dogRepository);
    }

    static void test(DogRepository repository) {
        repository.findAll().forEach(IO::println);
    }

}
```

1. First we create an instance…
2. …then we pass it to a `test` method, which expects an instance of an interface and doesn't care about how we got it.

Dependency Injection

You probably already saw this coming. We should centralize that `DataSource` definition. Here's the updated `Main` and `DefaultDogRepository`.

```
package com.example.beans_to_boot.di;

import org.springframework.jdbc.datasource.DriverManagerDataSource;

public class Main {

    static void main(String[] args) throws Exception {
```

```
        var db = new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
        var dogRepository = new DefaultDogRepository(db);
        test(dogRepository);
    }

    static void test(DogRepository repository) {
        repository.findAll().forEach(IO::println);
    }

}
```

And here's the `DogRepository` implementation. There shouldn't be much to say. This is, after all, just using constructors in an object-oriented language. But I'll say a few things anyway, because there was a time when that kind of code used to be fairly common.

When I first started using Java, I'd see a lot of code written like the first example, with the `DataSource` initialized *inside* the thing that used it. This was because a lot of the programmers who first adopted Java when I started using it came from C++ and C backgrounds. In those paradigms, developers were forced to manage memory, and if any of the memory got away from the programmer, you'd get an insidious memory leak! So we had a pattern: RAII. Resource Acquisition Is Initialization. The idea was that we bound things that needed to be managed—memory, sockets, etc.—to the lifecycle of the smallest scope, usually that of the object that used it. Put another way, if a `Foo` needed a `Bar`, it'd create it in the `Foo` constructor and it'd destroy it in the `Foo` *destructor*, a concept we don't really have in Java. Because Java has a garbage collector, it's fine if objects have lifetimes longer than the things that use them.

`DataSource`s tend to be expensive, especially once you have connection pooling. You'll want to keep tabs on where the `DataSource` is built, what its pool size is, and so on. There are several benefits here.

One, it's easier to maintain, as you only need to change things in one place. Two, we're writing our code…

Anyway, here's the updated repository with the constructor. One of the benefits of the constructor is that it's written against the lowest common type, `javax.sql.DataSource`. We're using Spring's built-in (and *not* production-worthy) `DriverManagerDataSource`. It does *not* do connection pooling or anything interesting. It just implements the `DataSource`. In a production environment, you'd likely replace it with a more appropriately configured HikariCP connection pool. In a test environment, you'd replace it with a mock. Either way, you only need to change one thing in one place and pass it into constructors as you go.

```
package com.example.beans_to_boot.di;

import javax.sql.DataSource;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Collection;
```

```
class DefaultDogRepository implements DogRepository {

    private final DataSource db;

    DefaultDogRepository(DataSource db) {
        this.db = db;
    }

    @Override
    public Collection<Dog> findAll() {
        try (var connection = this.db.getConnection(); //
                var ps = connection.prepareStatement("select * from dog") //
        ) {
            var list = new ArrayList<Dog>();
            try (var rs = ps.executeQuery()) {
                while (rs.next())
                    list.add(new Dog(rs.getInt("id"), rs.getString("name"),
rs.getString("description")));
            }
            return list;
        } //
        catch (SQLException throwable) {
            throw new RuntimeException(throwable);
        }
    }

}
```

Nothing fancy. Just object-oriented programming, constructors, and leveraging the power of polymorphism. Our constructors expect a `DataSource`, and we can give them any implementation we want without violating the contract.

Portable Service Abstractions

Looking at the code for the repository, it is a bit… much. I've written this sort of low-level `DataSource` manipulation code a million times in my career. Trust me when I say that as terrible as it is, it could be *much worse*. Thanks to the elegance of `AutoCloseable`, most of the resource management code and the tedious wrapping of `try` / `catch` go away. But it's still a low-level approach to data access. This isn't even controversial. JDBC is a driver, not an abstraction. It's the lowest level of infrastructure for working with data in a SQL database. I bet if you asked, you'd find that most of the folks working on JDBC drivers never expected people to use them directly. And a good thing, too, because it hurts.

Look at the code again. There are two things that are *essential*: the SQL query and the mapping logic. Everything else—creating the connection, prepared statements, and result sets; iterating over the result sets; extracting out to lists; and so on—is boilerplate code that is far removed from whatever we're trying to achieve by writing this code in the first place. (Surely there's *some* business justification? Right?)

Let's rework this code to leverage one of Spring's many amazing *portable service abstractions*, the `JdbcClient`.

```
package com.example.beans_to_boot.psb;

import org.springframework.jdbc.core.simple.JdbcClient;

import java.util.Collection;

class DefaultDogRepository implements DogRepository {

    private final JdbcClient db;

    DefaultDogRepository(JdbcClient db) {
        this.db = db;
    }

    @Override
    public Collection<Dog> findAll() {
        return db ①
            .sql("select * from dog")//
            ②
            .query((rs, _) -> new Dog(//
                    rs.getInt("id"), //
                    rs.getString("name"), //
                    rs.getString("description"))//
            ) //
            .list();
    }

}
```

① Look! There's the query we were talking about!

② And there's the mapper logic. Clear as a bell!

I think we can agree, this is *much* better.

Unified Transaction Handling with `PlatformTransactionManager`

This repository has just *one* method, and that method only executes *one* read query. But what happens if that method has two queries? What if they're both writes, modifying database state?

Suppose we're writing to the database and we want to add two records. The first record will go to a new `customers` table that contains a list of all the people who have registered with our pet shop. The second will go to a table containing the customer's expressed preferences for pets (small enough for condo or apartment living, needs more aggressive routine grooming, etc.). These two records are related to each other. Now imagine that, as soon as we write the first row, the power goes out and we don't manage to write the second row. Now we have a record in an inconsistent state in the

database. We can't help the customer until we sort out their preferences. Or imagine, by some contrivance, that the first write succeeds, and then somebody comes along and deletes the customer before the second gets a chance to commit. Now we've got preferences, but no customer!

The data is in an inconsistent state. The normal way to handle this in a database system is to use transactions. Transactions are, broadly speaking, a way of batching and then either committing or rolling back work. It looks conceptually like this:

- start tx
  - write 1
  - write 2
  - write 3
- if everything is good: commit *everything* all at once
- else: rollback

This oversimplifies things a bit, but the main takeaway is that from the perspective of other SQL clients to the database, the system is never in an in-between state. It's all or nothing. They won't ever take action based on inconsistent or invalid database state.

It's a *truly* powerful concept, and it has mirrors in all sorts of technologies. Here's an abbreviated list:

- JMS
- JDBC
- JPA (based on JDBC)
- JDO (based on JDBC)
- JDB (based on JDBC)
- JOOQ (based on JDBC)
- Kafka
- AMQP (the protocol under RabbitMQ)
- Neo4J
- MongoDB
- Redis (to be fair: here transactions are more about batching than rollbacks)

And as you might imagine, things can become disastrously complicated trying to work with transactions in a consistent way across all these technologies. Some might reach for JTA, but this sort of reminds me of that old XKCD cartoon in which, in introducing the new standard, we've only introduced yet another incompatible variant of the thing being standardized. It doesn't help. Here, Spring has yet another very powerful portable service abstraction called `PlatformTransactionManager`.

There's also a reactive variant, but let's stick to blocking semantics for this example.

Here's the basic shape of Spring's `PlatformTransactionManager`.

```
package org.springframework.transaction;

import org.jspecify.annotations.Nullable;

public interface PlatformTransactionManager extends TransactionManager {

    ①
    TransactionStatus getTransaction(@Nullable TransactionDefinition definition)
throws TransactionException;

    ②
    void commit(TransactionStatus status) throws TransactionException;

    ③
    void rollback(TransactionStatus status) throws TransactionException;
}
```

1. Create a transaction (and something to which we can ascribe subsequent work).
2. Commit the transaction and the batch of work therein.
3. Or, if something's gone wrong, roll back the work.

We can write code in terms of implementations—the `DataSourceTransactionManager`, for one—of this interface and handle transactions in a consistent way across our code. If we decide to work with Kafka tomorrow, we need only swap out the instance of the `PlatformTransactionManager` in a single place.

So, where to put this code? Remember, we're using Java, a good object-oriented language. We've got to follow the single-responsibility principle; each class should do one thing. Transaction handling feels like a different concern. I suppose we could extract it into another class that implements the same `DogRepository` interface, sets up and tears down the transactions, and then calls our actual target `DogRepository`.

Here's our new `TransactionalDogRepository`.

```
package com.example.beans_to_boot.tx1;

import org.springframework.transaction.PlatformTransactionManager;

import java.util.Collection;

class TransactionalDogRepository implements DogRepository {

    private final DogRepository target;

    private final PlatformTransactionManager txm;
```

```
    TransactionalDogRepository(PlatformTransactionManager txm, DogRepository target) {
        this.target = target;
        this.txm = txm;
    }

    @Override
    public Collection<Dog> findAll() {
        IO.println("starting transaction");
        var status = this.txm.getTransaction(null);
        try {
            var results = this.target.findAll();
            this.txm.commit(status);
            IO.println("committing transaction");
            return results;
        } //
        catch (Throwable throwable) {
            IO.println("rolling transaction back");
            this.txm.rollback(status);
            throw new RuntimeException(throwable);
        }
    }

}
```

It's wrapped the core functionality, finding records, in a transaction.

In order for this arrangement to work, we have to spin up a `PlatformTransactionManager` and rework our `Main` class.

```
package com.example.beans_to_boot.tx1;

import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;

public class Main {

    static void main(String[] args) {
        var db = new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
        var jdbc = JdbcClient.create(db);
        var dogRepository = new DefaultDogRepository(jdbc);
        var dbPlatformTransactionManager = new DataSourceTransactionManager(db);
        var transactionalDogRepository = new
TransactionalDogRepository(dbPlatformTransactionManager, dogRepository);
        test(transactionalDogRepository);
    }

    static void test(DogRepository repository) {
        repository.findAll().forEach(IO::println);
```

```
    }

}
```

① The instance given to the `test` method is still a `DogRepository`, so it is none the wiser.

Prefer Composition to Inheritance

This new wrapper class is a *composite*. In object-oriented design there are two ways to introduce new features to a type system: specialization (subclassing) and composition.

Prefer composition to inheritance. It buys you lots of *optionality*. Optionality is wiggle room, the freedom to make different decisions later on with little cost or disadvantage. Optionality is the bedrock of architecture: given the constraints, what choices should I make that most advantage me now and tomorrow?

- Flexibility at runtime. Composition lets an object hold references to other objects and swap them out, so behavior can change dynamically. Inheritance fixes the relationship at compile time.
- Avoids the fragile base class problem. With inheritance, changes to a parent class can silently break subclasses that depended on its internals. Composition isolates components behind their interfaces, so changes propagate less unpredictably.
- Escapes deep hierarchy rigidity. Inheritance tends to grow tall, tangled class trees. When you need behavior from multiple sources, single-inheritance languages force awkward workarounds; composition lets you assemble capabilities from independent pieces.
- Better encapsulation. Subclasses often need to know how the parent works internally (protected members, method call order). Composition only exposes the public interface of the parts you use, keeping coupling loose.
- Models "has-a" more honestly than forced "is-a". Inheritance implies a true subtype relationship. Developers frequently abuse it just to reuse code, creating relationships that aren't logically sound (a `Stack` is not really a `List`, even if it reuses one). Composition expresses "uses" or "has" without that false claim.
- Easier testing. Composed dependencies can be mocked or injected; inherited behavior is baked in and harder to isolate.

The tradeoff is a bit more boilerplate—you delegate calls to the contained objects rather than getting them "for free." Inheritance still fits genuine is-a relationships and cases in which you want polymorphism through a shared base type. The guidance is a default, not an absolute.

If you run `Main`, you'll see that we've got printouts and that we've started and stopped a transaction for every call.

Simplified Transaction Management with `TransactionTemplate`

I'm happy with what we've done, but the result is certainly more verbose. Imagine having to write

all that boilerplate transaction code in a new composite method for every method we introduce in the original implementation. It wouldn't be so bad if we could do away with the fragile resource management and error-handling logic.

The core arrangement is similar from one use to another. Start transaction, do work. Commit. Or catch an exception and roll back. The only thing we really care about is the stuff in the middle—the work itself. Another very interesting design pattern we care about is the *template* pattern. Here, we extract that which changes from that which doesn't, and we hide that which doesn't change in a *template method*. It's a template, you see, because you only need to fill out the parts that are unique to your situation.

Spring's `TransactionTemplate` and `JdbcTemplate`—the precursor to Spring's `JdbcClient`—were gateway drugs for me. I deleted thousands of lines of code when I first found these gems.

Let's refactor the `TransactionalDogRepository` to build on the `TransactionTemplate` instead.

```
package com.example.beans_to_boot.tx2;

import org.springframework.transaction.support.TransactionTemplate;

import java.util.Collection;

class TransactionalDogRepository implements DogRepository {

    private final DogRepository target;

    private final TransactionTemplate txm;

    TransactionalDogRepository(TransactionTemplate txm, DogRepository target) {
        this.target = target;
        this.txm = txm;
    }

    @Override
    public Collection<Dog> findAll() {
        return this.txm.execute(_ -> target.findAll());
    }

}
```

So much better!

Here's the refactored `Main` class, too.

```
package com.example.beans_to_boot.tx2;

import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
```

```
import org.springframework.transaction.support.TransactionTemplate;

public class Main {

    static void main(String[] args) {
        var db = new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
        var jdbc = JdbcClient.create(db);
        var dogRepository = new DefaultDogRepository(jdbc);
        var dbPlatformTransactionManager = new DataSourceTransactionManager(db);
        ①
        var transactionTemplate = new
TransactionTemplate(dbPlatformTransactionManager);
        var transactionalDogRepository = new
TransactionalDogRepository(transactionTemplate, dogRepository);
        test(transactionalDogRepository);
    }

    static void test(DogRepository repository) {
        repository.findAll().forEach(IO::println);
    }

}
```

① All we've done is introduce the `TransactionTemplate`, which of course takes a pointer to the `PlatformTransactionManager`.

Cross-Cutting Concerns

The approach is conceptually clean enough, but it's obviously going to start becoming tedious as we add more methods, or more concerns. Suppose we wanted to add logging. Or to add metrics. Or to add security checks. We'd have to write more and more composites, and for every new method in the core `DogRepository` interface, we'd have to add new methods in each of the composites.

What about ordering? Which composite wraps which? What if we wanted to support auditing, so that for every write operation we also write an access log to a database? That would need to happen inside the transactional composite so that both the underlying write and the write to the audit table would be in the same transaction.

This is where the object-oriented approach starts to fall apart. We've got a common concern that'll be common to *all* repositories. These are *cross-cutting* concerns.

Hold My Proxy!

The plan of attack for an object-oriented language is to use composition or inheritance to introduce new concerns. But we can't scale that. We'd get too many permutations of too many types in no time at all. So what if we could do this automatically? What if we could, at runtime, *programmatically* create a class that implements the interface? Java has had a way to do this since Java 1.4.

I've extracted the real metaprogramming magic into a separate new class called `Transactions`.

Fair warning: this level of high-wire metaprogramming can be a little intense for folks not prepared for it. You don't need to retain all of it. Let's just understand conceptually what's happening. We're going to ask Java to do two things at once: create a class that implements the interface we want it to implement (`DogRepository`) and then create a single instance of that new class and return it to us.

We provide a callback (here it's shown as a lambda, but it's actually an instance of the JDK's `InvocationHandler` type) that will be invoked whenever anybody invokes a method on the newly generated instance.

```
package com.example.beans_to_boot.jdkproxy;

import org.springframework.transaction.support.TransactionTemplate;

import java.lang.reflect.Method;
import java.lang.reflect.Proxy;

class Transactions {

    static Object createProxy(TransactionTemplate tt, Object target) {
        var classLoader = target.getClass().getClassLoader();
        var interfaces = target.getClass().getInterfaces();
        ①
        return Proxy.newProxyInstance(classLoader, interfaces, (_, method, args) ->
doInTx(tt, target, method, args));
    }

    ②
    private static Object doInTx(TransactionTemplate tt,
            ③
            Object target,
            ④
            Method method,
            ⑤
            Object[] args) {
        return tt.execute(_ -> {
            try {
                try {
                    IO.println("before tx");
                    ⑥
                    return method.invoke(target, args);
                } //
                finally {
                    IO.println("after tx");
                }
            } //
            catch (Exception e) {
                throw new RuntimeException(e);
            }
```

```
        });
    }

}
```

① Call `java.lang.reflect.Proxy.newProxyInstance(ClassLoader, Class[] interfaces, InvocationHandler)`.

② In the `InvocationHandler` lambda, we're given a pointer to `this`, the instance that `newProxyInstance` hands back to us. We don't need it here.

③ We're given a pointer to the `java.lang.reflect.Method` that is being invoked. Did the user call `toString` on the proxy instance? Then this will be the `Method` for the `toString` method.

④ We're given a pointer to the `Object[]` arguments that have been passed into the method when it was called. In our case, we're going to be calling `findAll`, which takes no arguments, so this `Object[] args` will be empty.

Weird, isn't it? But it's a force multiplier that we need. It'll allow us to achieve the same sort of runtime metaprogramming shenanigans you'd expect with a duck-typed language like JavaScript or Ruby, without giving up the type system for most of our code.

Let's rework our `Main` class, jettisoning the old, artisanal, hand-crafted `TransactionalDogRepository` and instead using this new shortcut to simplify things.

```
package com.example.beans_to_boot.jdkproxy;

import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.support.TransactionTemplate;

public class Main {

    static void main(String[] args) {
        var db = new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
        var jdbc = JdbcClient.create(db);
        var dogRepository = new DefaultDogRepository(jdbc);
        var dbPlatformTransactionManager = new DataSourceTransactionManager(db);
        var transactionTemplate = new
TransactionTemplate(dbPlatformTransactionManager);
        var transactionalDogRepository = (DogRepository)
Transactions.createProxy(transactionTemplate, dogRepository);
        test(transactionalDogRepository);
    }

    static void test(DogRepository repository) {
        repository.findAll().forEach(IO::println);
    }
```

```
}
```

Again, from the perspective of our "test," nothing has changed. It expects a `DogRepository`, and a `DogRepository` it got.

What About Concrete Types?

OK, we got this far. Let's talk about the elephants in the room. First: we now have a single concrete implementation of the interface and a programmatically generated implementation. But, really, since the programmatically generated one is not *our* work product, it feels like we've got a single interface for a single concrete type. Opinions vary here, but I would wager that the consensus is that if you have just one concrete implementation, then you don't need the interface. So how do we get rid of it? Remember, the JDK's `Proxy` class only works on *interfaces*. The JDK does not have a way to automatically create new implementations of concrete classes. Conceptually, it's simple. Subclass a concrete type and then use that subclass as a sort of composite, forwarding requests to the actual target. In this way the subclass has the same shape as the concrete type, and can wrap the target just as before. But how? The easiest way to do this these days is to do bytecode weaving. We can programmatically write—with bytecode—a new class file that we add to the classloader and then reflectively construct a new instance of. There are many libraries that can help with this sort of thing. ASM. The JDK itself in more recent versions can do this. ByteBuddy. And, a perennial favorite, CGLIB.

Spring vendors CGLIB. That is to say, it actually has a copy of CGLIB in the framework itself. We use it to generate classes for all sorts of reasons. And there's even a super convenient type, the `ProxyFactoryBean`, which we can use to achieve the same results as with the JDK's `Proxy` class.

Here's the rewritten `Transactions` class. No other changes are required. You can see we've even kept the same `doInTx` method. The concepts are the same; both the JDK implementation and the CGLIB implementation need a pointer to the current executing method, the arguments of the method, and the method itself.

```
package com.example.beans_to_boot.cglibproxy;

import org.aopalliance.intercept.MethodInterceptor;
import org.springframework.aop.framework.ProxyFactoryBean;
import org.springframework.transaction.support.TransactionTemplate;

import java.lang.reflect.Method;

class Transactions {

    static Object createProxy(TransactionTemplate tt, Object target) {
        var pfb = new ProxyFactoryBean();
        pfb.setProxyTargetClass(true);
        pfb.setTarget(target);
        for (var i : target.getClass().getInterfaces())
            pfb.addInterface(i);
        pfb.addAdvice((MethodInterceptor) invocation -> doInTx(tt, target,
```

```
invocation.getMethod(),
                invocation.getArguments()));
        return pfb.getObject();
    }

    private static Object doInTx(TransactionTemplate tt, Object target, Method method,
Object[] args) {
        return tt.execute(_ -> {
            try {
                try {
                    IO.println("before tx");
                    return method.invoke(target, args);
                } //
                finally {
                    IO.println("after tx");
                }
            } //
            catch (Exception e) {
                throw new RuntimeException(e);
            }
        });
    }

}
```

In this iteration, we've deleted the interface and renamed our `DefaultDogRepository` to just `DogRepository`. Less is more.

Better Configuration with Spring

Right now the code is approachable enough, but things are getting out of hand. What if we wanted to write any kind of test? We'd have to duplicate all these object definitions by hand. And this is just for a single database query in a single repository—we haven't even begun to think about web layers, messaging, or observability. Let's see if Spring can improve things. The idea is to turn object instantiation into *beans* (basically an object that Spring manages) and let the Spring `ApplicationContext` wire them together. Inside an `ApplicationContext` is a `BeanFactory`. The context is what glues a `BeanFactory` into the surrounding application environment. It's the context, after all. The `BeanFactory` underneath does most of the real work: it's the thing that performs dependency injection. Conceptually, the flow is simple: Spring ingests your configuration, produces `BeanDefinitions`, and from those definitions instantiates the actual beans.

Here's our rewritten `Main` class, this time using Spring Framework to wire things together.

```
package com.example.beans_to_boot.springframework;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
```

```
    static void main(String[] args) {
        ①
        var applicationContext = new
AnnotationConfigApplicationContext(DogConfiguration.class);
        var dogRepository = applicationContext.getBean(DogRepository.class);
        ②
        test(dogRepository);
    }

    static void test(DogRepository repository) {
        repository.findAll().forEach(IO::println);
    }

}
```

1. We create the `ApplicationContext` and tell it to source the wiring of these objects from a configuration class. Spring needs to know how your objects are wired together. There are multiple ways. You can use all or just one of them to get the job done. We're starting with one way, Java configuration. In this case it's a Java configuration class called `DogConfiguration`.

Here's that class.

```
package com.example.beans_to_boot.springframework;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.support.TransactionTemplate;

import javax.sql.DataSource;

@Configuration
class DogConfiguration {

    ①
    @Bean
    TransactionTemplate transactionTemplate(PlatformTransactionManager
platformTransactionManager) {
        return new TransactionTemplate(platformTransactionManager);
    }

    @Bean
    DriverManagerDataSource driverManagerDataSource() {
        return new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
    }
```

```
    @Bean
    JdbcClient jdbcClient(DataSource dataSource) {
        return JdbcClient.create(dataSource);
    }

    @Bean
    DogRepository dogRepository(TransactionTemplate tt, JdbcClient jdbcClient) {
        var target = new DogRepository(jdbcClient);
        return (DogRepository) Transactions.createProxy(tt, target);
    }

    @Bean
    DataSourceTransactionManager dataSourceTransactionManager(DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }

}
```

1. The wiring should look familiar. Each method constructs and returns one fully formed object. If the object requires any dependencies, it declares them as parameters in the method. Spring will attempt to resolve those dependencies for you.

We've simply retooled our `Main` method into an admittedly longer Java configuration class. In Spring, you want the return values of each method to be as specific as possible. Spring can do better work when it knows exactly what's been returned. For the dependencies, you can declare as imprecise a dependency as you want. This maxim appears all over the computer science ecosystem and is called *Postel's Law*. It says: *Be conservative in what you send, and liberal in what you accept*. The maxim is more about network services and API design, but it helps here when thinking about object wiring, too.

Bean Lifecycles

Earlier we spoke about C++ and RAII. That particular pattern isn't great, but it does imply an interesting point: objects often have interesting and delicate state. They're created. They're put into service. They're destroyed. Java handles memory management, but more things than just RAM can leak. Network sockets, for one. So it's important to have a concept of lifecycle. Spring provides *many* different lifecycle-related mechanisms that allow interested beans to participate in lifecycle events. These are essentially callbacks based on when the objects are created or destroyed.

In a typical Spring application, objects are created when the `ApplicationContext` starts up and destroyed when the `ApplicationContext` shuts down. There's a one-to-one mapping. But objects can have *scope*. They could be created anew for each and every HTTP request, for example. Or for each new unique HTTP session. Or for each new run of a *step* in a Spring Batch *job*. By default, Spring beans are *singleton*—they exist once and only once per application.

There are lots of ways to specify dependencies—*dependency injection*—for a bean. Thus far we've been relying on constructors. But some objects are designed in such a way that they have a constructor and *setter* methods that might be called with *optional* dependencies after the object's been created. In this case, the constructor wouldn't be the best place for initialization code because

the initialization would need to happen necessarily *after* the object's dependencies (constructor and setter alike) have been specified. Spring provides two ways of participating in this lifecycle phase: `InitializingBean` and `@PostConstruct`. Similarly, when the application shuts down, there are two equivalent approaches for cleaning up state: `DisposableBean` and `@PreDestroy`.

Here's our reworked `DogRepository` to demonstrate this.

```
package com.example.beans_to_boot.lifecycle;

import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.springframework.beans.factory.DisposableBean;
import org.springframework.beans.factory.InitializingBean;
import org.springframework.jdbc.core.simple.JdbcClient;

import java.util.Collection;

class DogRepository implements InitializingBean, DisposableBean {

    private final JdbcClient db;

    DogRepository(JdbcClient db) {
        this.db = db;
    }

    public Collection<Dog> findAll() {
        return db.sql("select * from dog")
            .query((rs, i) -> new Dog(rs.getInt("id"), rs.getString("name"),
rs.getString("description")))
            .list();
    }

    @PostConstruct
    void postInit() {
        IO.println("postInit");
    }

    @PreDestroy
    void preDestroy() {
        IO.println("preDestroy");
    }

    @Override
    public void destroy() throws Exception {
        IO.println("destroy");
    }

    @Override
    public void afterPropertiesSet() throws Exception {
        IO.println("afterPropertiesSet");
```

```
    }

}
```

Most of these lifecycle events are, in this particular example, pointless. But hopefully illustrative.

There are tons of interesting events that happen in a typical application. Somebody has authenticated in a Spring Security application. A Spring Batch `Job` has finished executing. A new HTTP request has been made. A customer has added an item to a cart. *Whatever*. Spring provides an event bus for components within the framework to make this work even easier. Any Spring bean can inject a reference to an `ApplicationEventPublisher` and publish an event—any object, but commonly a subclass of Spring's `ApplicationEvent`. And any interested bean can listen for this event. In the old days, beans did this by implementing the `ApplicationListener` interface; they'd get *every* event and would have to sift through the types to determine whether they were interested. Then we got generics-capable `ApplicationListener<T>`, where listeners would only be delivered events of the type they were interested in. Then, finally, we got `@EventListener`, which is my go-to approach these days. Here's an example showing two different listeners for Spring's `ContextRefreshedEvent`, which is published right after Spring has updated the bean graph with the input configuration.

```
package com.example.beans_to_boot.lifecycle;

import org.springframework.context.ApplicationEvent;
import org.springframework.context.ApplicationListener;
import org.springframework.context.event.ContextClosedEvent;
import org.springframework.context.event.ContextRefreshedEvent;
import org.springframework.context.event.EventListener;

class Listener implements ApplicationListener<ApplicationEvent> {

    @Override
    public void onApplicationEvent(ApplicationEvent event) {
        IO.println("got [" + event.getClass().getName() + "]");
        if (event instanceof ContextRefreshedEvent contextRefreshedEvent) {
            IO.println("got contextRefreshedEvent [" + contextRefreshedEvent + "]");
        }
    }

    @EventListener
    void onClose(ContextClosedEvent contextClosedEvent) {
        IO.println("context closed event [" + contextClosedEvent.getTimestamp() +
"]");
    }

}
```

And, of course, we've wired that up in the `DogConfiguration` class.

```
package com.example.beans_to_boot.lifecycle;
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.support.TransactionTemplate;

import javax.sql.DataSource;

@Configuration
class DogConfiguration {

    ①
    @Bean
    Listener listener() {
        return new Listener();
    }

    @Bean
    TransactionTemplate transactionTemplate(PlatformTransactionManager
platformTransactionManager) {
        return new TransactionTemplate(platformTransactionManager);
    }

    @Bean
    DriverManagerDataSource driverManagerDataSource() {
        return new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
    }

    @Bean
    JdbcClient jdbcClient(DataSource dataSource) {
        return JdbcClient.create(dataSource);
    }

    @Bean
    DogRepository dogRepository(TransactionTemplate tt, JdbcClient jdbcClient) {
        var target = new DogRepository(jdbcClient);
        return (DogRepository) Transactions.createProxy(tt, target);
    }

    @Bean
    DataSourceTransactionManager dataSourceTransactionManager(DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }

}
```

1. The newly registered `Listener` class.

A typical Spring application will publish dozens of events.

Post Processors

As you might've surmised, every Spring application goes through a well-known set of phases as it starts up. There's a lot more to it than this, but conceptually I find it helps to talk about three distinct phases.

- ingest, where Spring reads the definitions of how you want your objects wired. So far we've seen Java configuration. But there are others, including component scanning, XML definitions, and `BeanRegistrar`s.
- `BeanDefinition` - after reading all the configuration, Spring turns all configuration into a `BeanDefinition`, which is a metamodel object that contains the definition of how an object should be created, its lifecycle, its dependencies, etc., inside a `BeanFactory`. At this stage nothing has been created yet, but we know enough to validate how objects relate to each other, any missing dependencies, etc. Think of `BeanDefinitions` as the primordial soup for your application. No strong types at this point. Just designs and aspirations.
- beans - at this point we have live-fire objects. Spring has called constructors, passed in dependencies, and turned the `BeanDefinitions` into a valid graph of objects that taken together are your application.

Spring has callback interfaces that allow you to be involved at different phases during this three-phase lifecycle.

The earliest one is a `BeanFactoryPostProcessor`. Implement that and Spring will let you visit all the `BeanDefinition` instances in the `BeanFactory`, potentially deleting them, replacing them, or adding new ones.

Once the `BeanFactory` is full of viable `BeanDefinition` objects, Spring creates actual *beans* out of them by calling the constructors and setting the dependencies. Here we have one interface that lets us be involved *before* all the setters are called and *after*: `BeanPostProcessor`. A `BeanPostProcessor` is one of the last lines to cross before the program is up and running. A `BeanPostProcessor` may replace a bean of a given type with another bean of the same polymorphic type, but not much more. It is, however, an ideal place in which to introduce generic, cross-cutting concerns like transaction demarcation.

We can see this in action by creating a `BeanFactoryPostProcessor` called `MyBeanFactoryPostProcessor`.

```
package com.example.beans_to_boot.pp;

import org.springframework.beans.BeansException;
import org.springframework.beans.factory.config.BeanFactoryPostProcessor;
import org.springframework.beans.factory.config.ConfigurableListableBeanFactory;

class MyBeanFactoryPostProcessor implements BeanFactoryPostProcessor {
```

```
    @Override
    public void postProcessBeanFactory(ConfigurableListableBeanFactory beanFactory)
throws BeansException {

        for (var beanName : beanFactory.getBeanDefinitionNames()) {
            var beanDefinition = beanFactory.getBeanDefinition(beanName);
            var clzz = beanFactory.getType(beanName);
            IO.println("got the bean definition for [" + clzz.getName() + "] with bean
name [" + beanName + "]: "
                    + beanDefinition.getDescription());
        }

    }

}
```

This example registers two beans, our `BeanFactoryPostProcessor` *and* a single bean that implements `InitializingBean`. We don't really care about the bean itself, but we can inspect the resulting `BeanDefinition` in the `BeanFactoryPostProcessor` and inspect the resulting `description` on the `BeanDefinition` that results from the use of the Spring Framework `@Description` annotation on the bean method.

Make sure to register the `BeanFactoryPostProcessor` accordingly. It needs to be invoked very early on in the lifecycle of the application, so use `static` on the bean method.

```
package com.example.beans_to_boot.pp;

import org.springframework.beans.factory.InitializingBean;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Description;

@Configuration
class BppConfiguration {

    @Bean
    static MyBeanFactoryPostProcessor myBeanFactoryPostProcessor() {
        return new MyBeanFactoryPostProcessor();
    }

    @Bean
    @Description("A simple demo that has a description")
    InitializingBean simpleDemoThatHasADescription() {
        return () -> IO.println("a demo with a description ");
    }

}
```

So far we've been manually using the `Transactions` class to create transactional proxies. We'll have

to remember to do this for all objects. But suppose we had some generic mechanism by which to identify that we wanted to participate in transactions, like a marker interface. Let's call it `Tx`.

```
package com.example.beans_to_boot.pp;

interface Tx {

}
```

Add that to the implementation of `DogRepository`:

```
package com.example.beans_to_boot.pp;

import org.springframework.beans.factory.DisposableBean;
import org.springframework.beans.factory.InitializingBean;
import org.springframework.jdbc.core.simple.JdbcClient;

import java.util.Collection;

class DogRepository implements Tx, InitializingBean, DisposableBean {

    private final JdbcClient db;

    DogRepository(JdbcClient db) {
        this.db = db;
    }

    public Collection<Dog> findAll() {
        return db.sql("select * from dog")
            .query((rs, i) -> new Dog(rs.getInt("id"), rs.getString("name"),
rs.getString("description")))
            .list();
    }

    @Override
    public void destroy() throws Exception {
        IO.println("destroy");
    }

    @Override
    public void afterPropertiesSet() throws Exception {
        IO.println("afterPropertiesSet");
    }

}
```

What we need now is some code to sift through all the objects right as they're about to be pressed into service and, if they have this marker interface, wrap them in a transactional composite.

```
package com.example.beans_to_boot.pp;

import org.jspecify.annotations.Nullable;
import org.springframework.beans.BeansException;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.config.BeanPostProcessor;
import org.springframework.transaction.support.TransactionTemplate;

class TxBeanPostProcessor implements BeanPostProcessor {

    private final BeanFactory beanFactory;

    ①
    TxBeanPostProcessor(BeanFactory beanFactory) {
        this.beanFactory = beanFactory;
    }

    @Override
    public @Nullable Object postProcessAfterInitialization(Object bean, String
beanName) throws BeansException {
        if (bean instanceof Tx tx) {
            IO.println("making a transactional proxy for [" + tx.getClass().getName()
+ "] with bean name [" + beanName
                    + "]");
            var transactionTemplate =
this.beanFactory.getBean(TransactionTemplate.class);
            return Transactions.createProxy(transactionTemplate, tx);
        }
        return bean;
    }

}
```

1. Inject the `BeanFactory` so that we can ask it for whatever user-configured instance of `TransactionTemplate` is available. Note that we look up the bean only at the very last moment, at the time of the transactional method's invocation. We don't want to force eager initialization of the `TransactionTemplate` or any of its dependencies by looking up the object in the constructor.

With this done, we no longer need to manually call `Transactions` for any particular bean. The framework will do this automatically for any bean that implements `Tx`. Register the new `TxBeanPostProcessor` using a `static @Bean` method so Spring is able to invoke the bean much earlier in the lifecycle.

```
package com.example.beans_to_boot.pp;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Import;
```

```
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.support.TransactionTemplate;

import javax.sql.DataSource;

@Configuration
@Import(BppConfiguration.class)
class DogConfiguration {

    @Bean
    static TxBeanPostProcessor txBeanFactoryPostProcessor(BeanFactory beanFactory) {
        return new TxBeanPostProcessor(beanFactory);
    }

    @Bean
    Listener listener() {
        return new Listener();
    }

    @Bean
    TransactionTemplate transactionTemplate(PlatformTransactionManager
platformTransactionManager) {
        return new TransactionTemplate(platformTransactionManager);
    }

    @Bean
    DriverManagerDataSource driverManagerDataSource() {
        return new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
    }

    @Bean
    JdbcClient jdbcClient(DataSource dataSource) {
        return JdbcClient.create(dataSource);
    }

    @Bean
    DogRepository dogRepository(JdbcClient jdbcClient) {
        return new DogRepository(jdbcClient);
    }

    @Bean
    DataSourceTransactionManager dataSourceTransactionManager(DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }

}
```

This is where a good framework like Spring really comes into its own. Spring has a bird's-eye view of everything you're doing and can dramatically simplify things accordingly. You can centralize cross-cutting concerns in a single place. This is the essence of aspect-oriented programming (AOP).

But `@Transactional` Exists so Let's Use That Instead

If you have used Spring at all before, then you probably know where this is going: Spring has already done this exact declarative transaction demarcation magic trick. It's called `@Transactional`, and it's been in the framework for 20+ years. Let's refactor to use `@Transactional`. We just need to enable it with `@EnableTransactionManagement`.

```
package com.example.beans_to_boot.transactional;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;
import org.springframework.transaction.support.TransactionTemplate;

import javax.sql.DataSource;

@Configuration
@EnableTransactionManagement ①
class DogConfiguration {

    @Bean
    Listener listener() {
        return new Listener();
    }

    @Bean
    TransactionTemplate transactionTemplate(PlatformTransactionManager
platformTransactionManager) {
        return new TransactionTemplate(platformTransactionManager);
    }

    @Bean
    DriverManagerDataSource driverManagerDataSource() {
        return new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
    }

    @Bean
    JdbcClient jdbcClient(DataSource dataSource) {
        return JdbcClient.create(dataSource);
    }
```

```
    @Bean
    DogRepository dogRepository(JdbcClient jdbcClient) {
        return new DogRepository(jdbcClient);
    }

    @Bean
    DataSourceTransactionManager dataSourceTransactionManager(DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }

}
```

1. Here's the most important bit. We can replace everything we wrote with this one annotation, plus the `@Transactional` annotation.

And now we can delete `Tx` and remove its use from `Main`. We can also delete the `Transactions` class and the `TxBeanPostProcessor`.

Ahh. Much better.

Spring won't know to do this for us unless we ask it to with the `@EnableTransactionManagement` annotation on a `@Configuration` class.

The `@Transactional` annotation is more granular and capable than our homebrewed `Tx` marker interface. You can place it on a class, in which case every method in the class will be wrapped in a transaction, or you can place it on individual methods. You can also stipulate things like propagation, isolation, and timeouts. Or you could place it on the class for some sensible defaults and then override those at the method level. *Much* better.

This exercise of implementing a `BeanPostProcessor` and using proxies to centrally implement change is the core of how Spring works. Spring does this sort of misdirection *everywhere*. You put annotations on a class, Spring discovers it and replaces them with proxies that do the thing you asked for. And, thanks to the magic of polymorphism, consumers of these beans are none the wiser. They can interface with your code in terms of its plain interface. Everybody is better for it.

Component Scanning

We mentioned earlier that Spring can ingest your beans in a number of different ways, and that Java configuration was just one. Another one is *component scanning*, where Spring can scan my packages and identify beans to manage by the presence of marker annotations, which we call *stereotype* annotations, from the Unified Modeling Language (UML).

The root stereotype annotation is `@Component`. Place it on a class and instruct Spring to discover classes in the package(s) in which that class lives and Spring will apply some useful heuristics to create the bean for you. It'll attempt to satisfy any of the constructor's dependencies by type. It'll look for interfaces like `InitializingBean` and `DisposableBean`, and annotations like `@PostConstruct`, and honor them just as before. It'll create a singleton, just as before. Assuming Spring can construct the object without any intervention from you, this is a great way to make short work of registering

beans for Spring to manage. Indeed, it's easier to think about the places where Spring *couldn't* do the work for you.

- If you have custom factory methods (e.g., `JdbcClient.create(DataSource)`).
- If you need to pass in parameters that Spring can't satisfy by looking at other beans in the `BeanFactory`.
- If you don't have access to the source code. Obviously. From this arises an OK rule of thumb: use component scanning for your own code, use Java configuration for other people's code.

Spring also supports *meta-annotations*. You can put `@Component` on other annotations and then put those meta-annotations on your code and Spring will pick it up as well.

Spring does this quite famously with a handful of other annotations. These annotations are documentary *and* technical in nature. They tell the framework that the thing should be treated a certain way *and* they tell the programmer the intended role of the component in the codebase.

- `@Component` - a generic, managed bean
- `@Repository` - indicates that the component handles data-access logic. Reading, writing, etc.
- `@Service` - indicates that the component handles coarse-grained business logic.
- `@Controller` - indicates that the component is a *controller*, adapting requests of a particular protocol to server-side types, routing them to handler methods, and forwarding the output model onto a handler to produce a response.
- `@RestController` - indicates that the component is a *controller*, specifically adapted to HTTP and the tenets of the REST constraint on HTTP.

Interestingly, `@RestController` is *not* annotated with `@Component`. It's annotated with `@Controller`, so there you have *two* levels of meta-annotations!

Best part? You can create your own meta-annotations. Just add `@Component` to it and Spring will discover it just like it does any of the other meta-annotations.

Here's our reworked `DogConfiguration` supporting component scanning, and with no explicit Java bean configuration method for the `DogRepository`.

```
package com.example.beans_to_boot.componentscan;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;
import org.springframework.transaction.support.TransactionTemplate;

import javax.sql.DataSource;
```

```
@Configuration
@ComponentScan ①
@EnableTransactionManagement
class DogConfiguration {

    @Bean
    Listener listener() {
        return new Listener();
    }

    @Bean
    TransactionTemplate transactionTemplate(PlatformTransactionManager
platformTransactionManager) {
        return new TransactionTemplate(platformTransactionManager);
    }

    @Bean
    DriverManagerDataSource driverManagerDataSource() {
        return new DriverManagerDataSource("jdbc:postgresql://localhost/mydatabase",
"myuser", "secret");
    }

    @Bean
    JdbcClient jdbcClient(DataSource dataSource) {
        return JdbcClient.create(dataSource);
    }

    @Bean
    DataSourceTransactionManager dataSourceTransactionManager(DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }

}
```

1. We have enabled component-scanning with `@ComponentScan`. By default this will look in the current package (`com.example.beans_to_boot`) and below (e.g., `com.example.beans_to_boot.a.b.c`) and find any class marked with `@Component`.

The `DogRepository` no longer appears in the `DogConfiguration` class. We've simply annotated it with a stereotype annotation, `@Repository`.

```
package com.example.beans_to_boot.componentscan;

import org.springframework.beans.factory.DisposableBean;
import org.springframework.beans.factory.InitializingBean;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;
```

```
import java.util.Collection;

@Repository ①
@Transactional
class DogRepository implements InitializingBean, DisposableBean {

    private final JdbcClient db;

    DogRepository(JdbcClient db) {
        this.db = db;
    }

    public Collection<Dog> findAll() {
        return db.sql("select * from dog")
            .query((rs, i) -> new Dog(rs.getInt("id"), rs.getString("name"),
rs.getString("description")))
            .list();
    }

    @Override
    public void destroy() throws Exception {
        IO.println("destroy");
    }

    @Override
    public void afterPropertiesSet() throws Exception {
        IO.println("afterPropertiesSet");
    }

}
```

1. It's the little things that make everything so much nicer.

Component scanning supports a kind of *implicit* configuration. Spring discovers the class and applies heuristics to configure and construct a valid instance of your type. Java configuration supports a kind of *explicit* configuration. You must tell Spring almost everything, but you also have full control. It's a bit more verbose, but sometimes it's the best option.

Spring Framework 7's `BeanRegistrar`: The Best of Both Worlds

I love Java configuration and component scanning. (I certainly don't miss XML!) Component scanning was introduced as a first-class feature in Spring Framework 2.5 in 2007. Java configuration was introduced as a first-class citizen in Spring Framework 3.0 in 2009.

But they both have some limitations, too. Java configuration is very verbose. A whole method plus an annotation for one bean registration. They're also inflexible. How do I register beans dynamically, subject to some ambient thing like environment variables? What if I wanted to register five beans based on some property value? One does not simply programmatically add

methods to a class in a for-loop. The same limitation exists with stereotype annotations.

Spring Framework 7 (in 2026) introduced a new mechanism that nicely addresses some limitations of the other two models. If you need the dynamism it affords, it's the best way to get it.

I will confess, there *are* some existing ways to access that runtime dynamism that long predate `BeanRegistrar`, but they come with their own set of limitations and 99% of people won't use or need them, especially with `BeanRegistrar` now available.

Let's look at an example where we rewire *almost* the entire application using `BeanRegistrar`. We'll retain one bean with component scanning (the `DogRepository`) and we'll retain one bean (the `DriverManagerDataSource`) using Java configuration.

```
package com.example.beans_to_boot.beanregistrar;

import org.springframework.beans.factory.BeanRegistrar;
import org.springframework.beans.factory.BeanRegistry;
import org.springframework.core.env.Environment;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.support.TransactionTemplate;

import javax.sql.DataSource;

class DogBeanRegistrar implements BeanRegistrar {

    ①
    @Override
    public void register(BeanRegistry registry, Environment env) {

        ②
        registry.registerBean(Listener.class);
        registry.registerBean(MyBeanFactoryPostProcessor.class);
        ③
        registry.registerBean(TransactionTemplate.class,
                a -> a.supplier(ctx -> new
TransactionTemplate(ctx.bean(DataSourceTransactionManager.class))));
        registry.registerBean(JdbcClient.class, a -> a.supplier(ctx ->
JdbcClient.create(ctx.bean(DataSource.class))));
        registry.registerBean(DataSourceTransactionManager.class, a -> a
            .supplier(supplierContext -> new
DataSourceTransactionManager(supplierContext.bean(DataSource.class))));
    }

}
```

1. You do your wiring in a method. You can do iteration, load things, do `if`/`else`, etc., all from within a method. What a time to be alive!

2. If you want Spring to apply the same heuristics and automatically configure a bean, as it would when doing component scanning, then you can simply call `registerBean(Class)` and Spring will take it from there.
3. If you want to exert more control over how things are wired up, or look up other beans for reference, pass in the second parameter and specify a `Supplier<T>` on the `spec` parameter.

All in all, this is a huge improvement in total lines of code. I don't know how I feel about readability, though. I think it'd be pretty natural to factor those lambda suppliers out to separate methods. But then, I might as well just use Java bean configuration. That said, in Kotlin, this DSL is even more succinct and elegant and inarguably readable while taking less code.

This style grows on me more and more. You can use it. Or don't. You've got options depending on your style and use case.

The `Environment`

Let's revisit the one bean we left as Java configuration: the `DriverManagerDataSource`. This entire time we've left that bean configured with values hardcoded in the Java source code. This is terribad for a number of reasons.

First, in a book about security, it is *woefully* insecure! Obviously you should store your passwords in a more secure medium than in plaintext buried in the source code for a program developers interact with every day. Especially if it's the production database password. Heaven forbid!

Second, this is super fragile. It doesn't conform to the principles behind *The 12 Factor Manifesto* [https://12factor.net], which offers a ton of good advice for how to structure and deploy cloud-native applications. Factor III (oh, excuse me, I meant "three") is *Config*. This factor says that "an app's config is everything that is likely to vary between deploys (staging, production, developer environments, etc)." This includes things like database connection credentials, backing services like Amazon S3, a cache, etc. It poses the litmus test for whether your application is well enough configured as follows: if the codebase were open-sourced right now, would any credentials be compromised? If so, you've not done a good enough job with your configuration.

Let's use the Spring Framework `Environment` abstraction. It's a layer of indirection between the application and the outside world. Think: property files, YAML files, environment variables, JNDI keys, HashiCorp Vault, etc. The `Environment` is an object that any Spring bean might inject and use to load configuration values. It in turn delegates to a chain of `PropertySource` instances that know how to load values from various sources. Spring comes loaded with some useful out-of-the-box defaults. As you move up the abstraction stack, you get even more sophisticated alternatives for things like HashiCorp Vault, the Spring Cloud Config Server, etc.

Let's load our configuration values from a local configuration file called `application.properties`.

```
package com.example.beans_to_boot.environment;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.*;
import org.springframework.core.env.Environment;
```

```
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.annotation.EnableTransactionManagement;

import java.net.URI;
import java.util.Objects;

@Configuration
@ComponentScan
①
@PropertySource("classpath:application.properties")
@EnableTransactionManagement
@Import(DogBeanRegistrar.class)
class DogConfiguration {

    @Bean
    DriverManagerDataSource driverManagerDataSource(Environment environment,
            ②
            @Value("${spring.datasource.username}") String usernameValue) {
        IO.println("configuring the DataSource username: [" + usernameValue + "]");
        ③
        var username = environment.getProperty("spring.datasource.username");
        ④
        var url = environment.getProperty("spring.datasource.url", URI.class);
        var pw = environment.getProperty("spring.datasource.password");
        return new
DriverManagerDataSource(Objects.requireNonNull(url).toASCIIString(), username, pw);
    }

}
```

1. The `@PropertySource` annotation is a convenient way to add the contents of a `.properties` file to the `Environment`.
2. You can inject a reference to the `Environment` directly *or* you can have Spring provide the value for you in Java configuration methods by annotating parameters with `@Value` and then using a `\${…}` syntax.
3. Here we're just loading a value and getting it back as a `String`, the default.
4. Spring's built-in `ConversionService` can translate all manner of different String values into useful domain types. You only need ask, in this case by passing in the preferred target conversion type. I'm asking that the URI be converted to a `java.net.URI`. Why not? If it's not a valid URI, the conversion will fail and the program will stop just that much earlier.

We've added a new `.properties` file to the example.

```
spring.application.name=beans-to-boot
#
①
spring.datasource.url=jdbc:postgresql://localhost/mydatabase
spring.datasource.password=secret
```

```
spring.datasource.username=myuser
#
②
spring.threads.virtual.enabled=true
#
③
spring.sql.init.mode=always
④
management.endpoint.health.show-details=always
management.endpoints.web.exposure.include=*
management.tracing.sampling.probability=1.0
```

1. I've chosen to use the keys `spring.datasource.url`, `spring.datasource.password`, and `spring.datasource.username` as the keys for my DataSource's URL, password, and username for, um, no reason at all…
2. We'll return to this soon.
3. We'll return to this soon.

I've got the password for the PostgreSQL server as it will be configured when we use the Docker Compose `compose.yml` file that ships with this codebase. This isn't the end of the world for local-only development. But obviously in production environments, you'd never have the production database username and password and URL in plaintext. You'd pass them in through environment variables or use something like HashiCorp Vault.

This is why Spring's `Environment` has the notion of precedence. External contributions—those further away from the codebase—override contributions closer to the codebase, conceptually. So, if you specify an environment variable, it'll override the values provided in `application.properties`. There's even some goodness in there to normalize environment variables. So, `SPRING_DATASOURCE_URL` will be normalized to `spring.datasource.url`. In this way, developers can keep the hardcoded development-only environment variable, but at runtime in production the real connection credentials can be specified with environment variables.

Next Steps

We've built up quite the application, and in the next chapter we'll see how much we can delete with the introduction of Spring Boot! :code: ..

Spring Boot

We've come a long way, but done very, very little. For all our troubles, we've only got one repository with one method. Now imagine we were trying to build a real application. We'd need security; we'd need to stand up a web server. We'd need observability. We'd need to package the application up for production.

Let's leverage Spring Boot. It's already on the CLASSPATH. Spring Boot builds on Spring Framework (and the rest of the Spring ecosystem). It's just more Spring, taken a step further thanks to the power of something called *autoconfiguration*. But first, let's refactor.

Here's the updated `Main` class.

```
package com.example.beans_to_boot.boot;

import org.springframework.boot.ApplicationRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Import;

①
@SpringBootApplication
②
@Import(DogBeanRegistrar.class)
public class Main {

    static void main(String[] args) {
        ③
        SpringApplication.run(Main.class, args);
    }

    ④
    @Bean
    ApplicationRunner testRunner(DogRepository repository) {
        return _ -> repository.findAll().forEach(IO::println);
    }

}
```

1. Spring Boot's `@SpringBootApplication` annotation is itself a *meta-annotation*. It is meta-annotated with `@Configuration`, which means this `Main` class is also a `@Configuration` class. It's meta-annotated with `@ComponentScan`, so we don't need to worry about that either.
2. because it's a `@Configuration` class, we can use `@Import` on it, just like before.
3. through most of the code we've looked at, we've created a Spring `ApplicationContext`. Here, we're doing the same thing. The return value of `SpringApplication.run` is an `ApplicationContext`. But we don't need it, because we don't need to explicitly look up the `DogRepository` to pass to our

`test` method. Instead…

4. …we can define a bean of type `ApplicationRunner`. `ApplicationRunner` beans are given a chance to run as close as possible to the moment when the application is fully initialized and right before it's put into service. So, here, rather than squatting on the `main` method as we did in every earlier example, we simply register this bean. We can register as many as we want, for whatever purpose, and know that this logic will run as the service initializes.

Spring Data JDBC

We're using Spring Boot now. We've got an object mapper (not quite a *full* ORM, but close enough) framework on the CLASSPATH. Spring Data JDBC is capable, and it's already been configured. Let's rework our repository to leverage it, rather than hand-rolling SQL ourselves.

```
package com.example.beans_to_boot.boot;

import org.springframework.data.repository.ListCrudRepository;

import java.util.Collection;

interface DogRepository extends ListCrudRepository<Dog, Integer> {

    Collection<Dog> findByName(String name);

}
```

I know, you're thinking: "Didn't we get rid of the interface ages ago?" But fear not! The interface is back, but this is *way* better: we got rid of the implementation! This interface, which extends Spring Data's `ListCrudRepository`, already has a method called `findAll`. It also has methods to `save`, `delete`, `findById`, etc. Spring will create a proxy implementation of this interface at startup time and provide implementations for all those methods by default. You can even define custom finder methods. In this example, I've got a finder method called `findByName` that, when invoked, will execute a SQL query of the shape `select * from dog where name = ?`. It does this by convention. You can, of course, annotate the method in the interface with `@Query`, and Spring will use your query instead. *Much better!*

Conventions over Configuration

Spring Boot has a ton of convenient conventions. For example, it'll create a *real* `DataSource` backed by the HikariCP connection pool, with properties loaded by convention from the Spring `Environment`. Spring Boot will, by convention, load any `application.properties` or `application.yml` it finds in `src/main/resources`. So we no longer need our `@PropertySource` annotation.

AutoConfiguration

We imported the `DogBeanRegistrar`, so we might as well look at it.

```
package com.example.beans_to_boot.boot;

import org.springframework.boot.ApplicationRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Import;

①
@SpringBootApplication
②
@Import(DogBeanRegistrar.class)
public class Main {

    static void main(String[] args) {
        ③
        SpringApplication.run(Main.class, args);
    }

    ④
    @Bean
    ApplicationRunner testRunner(DogRepository repository) {
        return _ -> repository.findAll().forEach(IO::println);
    }

}
```

What happened? Where are all our beans?! They're gone. (Almost) all gone. Spring Boot provides those beans for us, thanks to its marvelous *autoconfiguration*. So all that remains are our beans.

When Spring Boot starts up, it scans the CLASSPATH for any `.jar` containing a file called `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`. That file, in turn, contains a newline-delimited list of Java configuration classes' fully qualified names. Functionally, these classes are the same as the other Java configuration classes we've seen so far, with some small refinements. First, keep in mind that Spring Boot will *try* to load every single one of these configuration classes on startup. Sometimes the beans in these configuration classes are not important or useful in a particular situation. It would be grossly inefficient if Spring Boot registered beans that we didn't need. So, autoconfiguration classes and the `@Bean` methods within are guarded with `@Conditional` annotations. These annotations point to implementations of the `Condition` type. These `Condition` instances test something about the application or its environment, and if the test comes back positive, the beans are contributed to the `BeanFactory`. If not, they are not contributed.

There are many such conditions. Conditions to test for the presence or specific value of a property in the `Environment`. Conditions to see whether you and I, the users, have or have not already defined beans of a particular type. Conditions to see whether certain classes are present in the CLASSPATH. These conditions let Spring Boot avoid defining beans that aren't needed, and degrade gracefully. If we'd kept our `DriverManagerDataSource` from earlier, Spring Boot would've backed off on contributing its `DataSource`, because it would infer that we want to exert control here. In this way, we get the best of both worlds: Spring Boot has sensible defaults that drop away whenever we want

to override something.

We can (and should) create our own autoconfiguration if we want to leverage the same convention-over-configuration experience that Spring Boot promotes.

Create an autoconfiguration class of your own. Be sure to put the code in a package that is *not* scanned by Spring's component scanning mechanism. In my application, my root package is `com.example.beans_to_boot`, so Spring will discover all classes in that package or deeper. So, I'll put the autoconfiguration in a separate, adjacent, package called `com.example.autoconfigure`, not because it's required but because it confirms that we've wired up the autoconfiguration correctly and that this configuration class will be loaded and evaluated regardless of the fact that it's not discovered by the component scanning machinery.

```
package com.example.beans_to_boot.autoconfigure;

import org.springframework.boot.ApplicationRunner;
import org.springframework.boot.autoconfigure.AutoConfiguration;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.context.annotation.Bean;

@AutoConfiguration ①
@ConditionalOnRandomness ②
class SillyAutoConfigurationExample {

    @Bean
    ③
    @ConditionalOnProperty(value = "my.config", matchIfMissing = true, havingValue =
"bar")
    ApplicationRunner myConditionalBean() {
        return _ -> IO.println("randomly initialized bean");
    }

}
```

1. `@AutoConfiguration` is yet another meta-annotation. You could use `@Configuration` here if you wanted.
2. `@ConditionalOnRandomness` is a conditional annotation that we'll write ourselves in just a moment.
3. `@ConditionalOnProperty` is a Spring Boot conditional annotation that will evaluate to true if the property called `my.config` exists in the `Environment`. We further stipulate that the value will be matched if it's not specified. If it is specified, and only if the value is `bar`, then this Java configuration bean will be registered. If this test evaluates to true, you'll see the output of our `ApplicationRunner` when the application starts up. If not, nothing will happen.

Let's look at the implementation of our (admittedly very contrived) condition.

```
package com.example.beans_to_boot.autoconfigure;
```

```
import org.springframework.context.annotation.Condition;
import org.springframework.context.annotation.ConditionContext;
import org.springframework.context.annotation.Conditional;
import org.springframework.core.type.AnnotatedTypeMetadata;

import java.lang.annotation.*;

①
@Target({ ElementType.TYPE, ElementType.METHOD })
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Conditional(RandomnessCondition.class)
@interface ConditionalOnRandomness {

}

②
class RandomnessCondition implements Condition {

    @Override
    public boolean matches(ConditionContext context, AnnotatedTypeMetadata metadata) {
        return Math.random() > .5;
    }

}
```

1. this annotation is a meta-annotation which we've annotated with Spring Framework's `@Conditional` annotation. We pointed it to an implementation of `Condition` that we've written called…
2. `RandomnessCondition`, which evaluates to true whenever the result of `Math.random()` is greater than `0.5`.

Now, create a text file in `src/main/resources/META-INF/spring/` called `org.springframework.boot.autoconfigure.AutoConfiguration.imports` in your project.

```
com.example.beans_to_boot.autoconfigure.SillyAutoConfigurationExample
```

Here, we've got just the one class. This is a bit of a toy, but if you restart the program a few times, you'll see your `ApplicationRunner` run (or not run) in seemingly random alternation.

We could have put this autoconfiguration class and the text file in a separate `.jar`, added that to the CLASSPATH of any Spring Boot application, and Spring Boot would give your code a chance to run *without* requiring input from the user at all. This is how you provide seamless, out-of-the-box, sensible defaults and configurations for your users.

But how do you distribute them? In a `.jar`, of course. But here again, Spring Boot's design philosophies pay dividends. Spring Boot uses something called *starter* dependencies. These are dependencies that serve only to aggregate other Maven dependencies and to manage their

versions. When you work with a Spring Boot project, as generated from the Spring Initializr, you should only ever have to change one version - at most, four. All the other Spring dependencies for the rest of the codebase come from the Spring Boot parent artifact, which ensures that versions line up. If one dependency uses one version of Log4J, for example, and another uses a different one, Spring Boot will pick one and ensure it works for all the dependencies it supports.

Recall that Spring Boot has conditional annotations, one of which is `@ConditionalOnClass`. A bean so annotated will only be registered *if* the class is on the CLASSPATH. Spring Boot *starters activate* different conditionals because they bring in different cohorts of dependencies.

An Amazing Web Server

Spring Boot autoconfiguration does a whole heckuva lot more than register a `DataSource`. You might notice that when you start the program, it doesn't just exit as soon as the `Dog` entities have been enumerated. Spring Boot started a web server. Why? Because we asked it to by adding `Spring Web` to the project *way* back at the very beginning of this chapter. So we now have a full web framework and web server in play.

Here, I even wrote a trivial Spring MVC controller that loads all the results from the repository.

```
package com.example.beans_to_boot.boot;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;

import java.util.Collection;

@Controller
@ResponseBody
class DogController {

    private final DogRepository repository;

    DogController(DogRepository repository) {
        this.repository = repository;
    }

    @GetMapping("/dogs")
    Collection<Dog> dogs() {
        return this.repository.findAll();
    }

}
```

Run the program and then hit `http://localhost:8080/dogs` in your browser to see that it's working.

By the way, when you ran the program, did you notice the pretty colored log output? And the *amazing* ASCII art banner?

If you put a custom banner in `src/main/resources/banner.txt`, Spring Boot will use it instead of the default built-in ASCII art.

Remember when we first introduced `application.properties`? I didn't speak to some of the configuration values in the property file. Only those concerned with the `DataSource`. Let's revisit some of those.

```
spring.application.name=beans-to-boot
#
①
spring.datasource.url=jdbc:postgresql://localhost/mydatabase
spring.datasource.password=secret
spring.datasource.username=myuser
#
②
spring.threads.virtual.enabled=true
#
③
spring.sql.init.mode=always
④
management.endpoint.health.show-details=always
management.endpoints.web.exposure.include=*
management.tracing.sampling.probability=1.0
```

1. the original database credentials… moving on.
2. this property enables virtual threads, a game-changing feature in Java 21 or later. This will dramatically improve the scalability of blocking I/O.
3. this property tells Spring Boot to run `src/main/resources/schema.sql` and `src/main/resources/data.sql` on startup. Super convenient to ensure the system is correctly configured.

Observable Services

Spring Boot is production-ready and worthy out of the box. When you configured the application back on the Spring Initializr, you added the `Actuator` and `OpenTelemetry` dependencies. The Actuator dependency brings in a slew of endpoints, usually mounted under `/actuator/*`. There are many endpoints, but most of them are not visible unless you *expose* them with another property added to `application.properties`.

If you want to see all the Actuator endpoints and the full health endpoint, you might use the following configuration properties.

```
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
```

Now you can visit `/actuator/metrics`, `/actuator/health`, `/actuator/beans`, `/actuator/configprops`, etc.

Visit just `/actuator` to see all the sub-endpoints.

We've also got Grafana running in the background. As you work through the code in Spring Boot, you might notice that, first of all, you're seeing a good deal more application events. Second, if you visit http://localhost:3000 and then navigate to `Drilldown → Metrics`, you can see the same metrics you saw under `/actuator/metrics`, but now graphed over time. Neat!

[grafana] | *images/boot-and-beyond/grafana.png*

GraalVM Native Images

This application runs really quickly and scalably, but could it be faster and use less RAM? Sure! One way you might help with that is to embrace GraalVM native images. GraalVM [https://graalvm.org] is an OpenJDK distribution that carries with it some amazing extras. You'll need to have it installed on your machine.

I'm interested in the `native-image` compiler tool that comes with GraalVM. It takes a Java program's `.class` files and turns them into native, operating-system- and architecture-specific machine code. The kind you'd get from using C, Go, or Rust. The result is a *markedly* smaller memory footprint and a *markedly* faster startup time, at the dubious expense of portability. The resulting binaries only run on the platform for which they were compiled. If you compile the binary in a Linux CI environment, it'll be fine in a Linux production environment, so it's not as big a deal-breaker as it might sound.

Keep in mind that GraalVM native images are *not* standard Java and in fact run counter in some ways to Java's write-once, run-anywhere mantra and promise of portability.

GraalVM was the first scenario where Spring Boot could leverage ahead-of-time code generation to transform the program into a form that would execute more efficiently at runtime. One of the AOT transformations that's just been made possible in Spring Boot 4 and Spring Framework 7 is compile-time, ahead-of-time code generation for Spring Data repositories like our Spring Data JDBC repository. For it to work, however, the AOT engine will need to know which SQL query dialect is being used *at compilation time*.

Add the following bean to help it along.

```
package com.example.beans_to_boot.boot;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.jdbc.core.dialect.JdbcPostgresDialect;

@Configuration
class DataAotConfiguration {

    @Bean
    JdbcPostgresDialect jdbcPostgresDialect() {
        return JdbcPostgresDialect.INSTANCE;
```

```
    }

}
```

You can take advantage of native-image technology thusly:

```
./mvnw -DskipTests -Pnative native:compile
```

Then, stand back. It might take a while. Maybe even minutes. On my machine, for this application, it takes about 30 seconds to do the full compile. Your mileage may vary.

Try it. Go to the `target` folder and run `target/beans-to-boot` or whatever your Maven artifact name was.

GraalVM can have *amazing* results, but it does require some compromises about which you need to be aware. The way GraalVM works is by starting from your `main` method entry point into the application, scanning every class that you use, and every class that those classes use, and every class that those classes use, and so forth, and onward until it's built up a giant graph of which classes your program will use. It retains those and throws every other class away before it compiles your program. It also throws away reflective metadata (you know, the types and instances that you get back when you try to do something like `Customer.class.getMethods()`). It also throws away the metadata for JNI references. And `Serialization`. Also, all the stuff that aren't `.class` files, but that ship in the `.jar` files for your application - things like `.properties` files, images, etc.? It throws all that away, too. Basically, all the stuff that the JVM normally would create and/or load on every startup, but that your program may never need: it throws it all away. The result is a *lightning* fast native image which uses a tiny fraction of the usual RAM.

But it's not perfect. The analysis that results in this much smaller set of classes happens during compilation. Java's a Turing-complete language. It's impossible to completely understand at compile time what will happen at runtime. So the GraalVM compiler might throw things out that you might actually end up needing during runtime. So you'll need to tell it what to keep. What *not* to throw out. Normally, in a GraalVM application, you do this with `.json` configuration files that live in `.jar` files under `/META-INF/native-image/${groupId}/${artifactId}/*.json`.

The trouble is that these configuration files are fiddly. If you refactor a type in your Java code, your IDE might forget to update the entry in the `.json` configuration file. So, you'll need to update it yourself. It gets tedious.

Spring Framework 6 and Spring Boot 3 introduced an Ahead-of-Time (AOT) component model that gets evaluated *during compilation*. Remember when we talked about the Spring `BeanFactory` and said that it handles *ingest* (the loading of bean configuration from various sources like component scanning, Java configuration, etc.), then turns everything into metamodel objects of the class `BeanDefinition`? Those `BeanDefinition` objects give any would-be examiner enough information about the objects in the application, including their classes.

Spring's AOT component model builds your application, to that point, giving you access to the `BeanDefinition` objects, and allowing you to programmatically register *hints*, which get translated into the aforementioned `.json` files.

We won't belabor the point, but there are different ways to register these AOT hints, depending on the level of power you need.

For most applications, users should probably get comfortable with `RuntimeHintsRegistrar`.

Let's suppose we wanted to read in a text file (`message`) and write out its contents to the command line when the program starts up. You could easily construct a `Resource` object and then read its contents in the JVM. But that text file won't exist in a GraalVM native image, because it'll have been thrown out. We need to tell GraalVM not to.

Here's the text file.

```
hello, AOT!
```

Now we need to register some code to contribute the AOT hints.

```
package com.example.beans_to_boot.boot;

import org.jspecify.annotations.Nullable;
import org.springframework.aot.hint.RuntimeHints;
import org.springframework.aot.hint.RuntimeHintsRegistrar;
import org.springframework.boot.ApplicationRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.ImportRuntimeHints;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

import java.nio.charset.Charset;

@Configuration
①
@ImportRuntimeHints(AotConfiguration.Hints.class)
class AotConfiguration {

    ②
    private static final Resource MESSAGE = new ClassPathResource("/message");

    static class Hints implements RuntimeHintsRegistrar {

        @Override
        public void registerHints(RuntimeHints hints, @Nullable ClassLoader
classLoader) {
            ③
            hints.resources().registerResource(MESSAGE);
        }

    }
```

```
    ④
    @Bean
    ApplicationRunner messageLoader() {
        return _ -> IO.println(MESSAGE.getContentAsString(Charset.defaultCharset()));
    }

}
```

1. The `AotConfiguration` class is a stock-standard Java configuration class. The `@ImportRuntimeHints` annotation loads the AOT `RuntimeHintsRegistrar` class. This class will be invoked during compilation.
2. Spring's `Resource` abstraction is a nice way to represent anything that could be read (that is, it produces `byte`) or written to. There are many implementations. Here we're using the one to represent resources on the CLASSPATH.
3. the `RuntimeHints` object offers convenient DSL methods to register resources for loading, types for reflection, or serialization, etc.
4. By the time this `ApplicationRunner` executes at runtime, the `message` file *will* be in the memory of the native image and everything will continue to work, all because of the work we did at compile time to make it so.

Conceptually, that's all there is to it! There are other callback classes besides `RuntimeHintsRegistrar`, but that's a topic for discussion some other day. What's important to know is that you'll almost never need to worry about registering these AOT hints. Spring Boot will do it. Or there'll be configuration in the libraries that you consume. But not *always*. In which case, you'll have to do the work yourself. But at least you now know how.

Project Leyden

GraalVM native images are my favorite approach for optimization, but they do come with their own caveats. They don't always work without sometimes considerable finessing, and I won't get into all that here, but there is a whole Spring Ahead-of-Time (AOT) component model that allows you to add supplementary configuration to your Spring Boot applications so that they behave well in a GraalVM native-image context.

If you want a standard way to improve the startup time (and not much else) of your application, you can also use Project Leyden, part of the core Java SDK. It works reliably on *every* program, with no compromises, but the results aren't quite as impressive, either.

The thrust of it is that you run the program in a special mode: the JVM dumps out information about which things are used into a file, and that file can then be fed into the `java` CLI on a subsequent run so it'll start up a lot faster. On my machine, this application is almost 50% faster on startup as a result of using Project Leyden. Here's an example of doing this training run and then getting great results.

```
#!/usr/bin/env bash
①
rm -rf extracted
```

```
rm -rf aot

②
mvn -DskipTests -Pnative  clean package
③
cp target/*jar application.jar
④
java -Djarmode=tools -jar application.jar extract --destination aot

cd aot

⑤
java -XX:AOTCacheOutput=app.aot   -Dspring.aot.enabled=true
-Dspring.context.exit=onRefresh -jar application.jar

⑥
rm -rf ../application.jar

⑦
java -XX:AOTCache=app.aot   -Dspring.aot.enabled=true -jar application.jar
```

1. clean up and staging
2. use Spring's AOT mode and do a package
3. move the resulting packaged `.jar` to a well-known name, `application.jar`
4. run the program using Spring Boot's `.jar` tools to extract the normally nested Spring Boot `.jar` into a folder called `autoconfigure`.
5. run the Java program up until the application context's been refreshed, and dump the analysis out to `app.aot`.
6. cleanup
7. finally, run the program but use the `app.aot` index to dramatically speed up the application.

The Spring Boot Buildpack integration

For most scheduling solutions, you'll need a container containing your application. This doesn't mean that you'll need a `Dockerfile`, however. Quite the contrary. You're better off automatically building the container rather than manually defining it. In the world of mystery-meat cloud containers, *less is more*.

Spring Boot's Maven and Gradle plugins can do the work for you by deferring to the Buildpacks project [https://buildpacks.io]. Buildpacks provide recipes for taking application artifacts and packaging them up as containers. These recipes represent the collective wisdom of the Spring and Java communities that maintain them. There are also buildpacks for scores of other languages and ecosystems, including COBOL!

Here's how you can use it for your own application. If you want to build a container for a standard JVM-based application, do the following.

```
./mvnw -DskipTests spring-boot:build-image
```

If you want to build a container for a GraalVM native image, do the following:

```
./mvnw -DskipTests -Pnative spring-boot:build-image
```

Let this run for a bit, and it'll give you the name of the container that's just been created. You can run it or `docker push` it to whatever container registry you want. From there, you're off to production!

Next Steps

In this section we explored the awesome force-multiplying potential of Spring Boot and saw how it frees you to focus on that which matters: building the best, most valuable application possible for your customers.

At this point, you're grounded enough in the concepts that you at least know where to start looking when working with new features in a Spring Boot world. This will serve you well as we move up the abstraction stack in subsequent chapters. :code: ..

Spring Security Fundamentals

In this section we'll look at the fundamentals of Spring Security. After all, we won't get far without those core concepts in place.

Spring Security, the Project

Spring Security began in late 2003 when Ben Alex created what was originally called Acegi Security (or the Acegi Security System for Spring) after discussions on the early Spring developer mailing lists about the need for a comprehensive security framework. At the time, the Spring Framework lacked a robust security architecture, leaving developers to build custom, application-specific authentication and authorization solutions. Acegi was designed from the ground up to leverage Spring's core principles of dependency injection and aspect-oriented programming, providing a configurable, pluggable security architecture built around a flexible servlet `FilterChain` that allowed developers to bypass rigid container-managed security entirely. It quickly gained traction by handling both robust authentication and complex authorization directly within the Spring application context.

Ben released Acegi as an open source project under the Apache License in March 2004. The project rapidly attracted a small but enthusiastic community and gained traction within the Java ecosystem. By 2006, Acegi had matured into a production-ready framework, and in 2007 it was officially adopted into the Spring portfolio as an official Spring subproject.

This era, for me, was where I started to really get excited about Spring. It's where I decided I wanted to be a part of the story. I'd used Spring Framework and all of its wonderful little `Template` classes and its web framework and dependency injection framework and all that in my various applications, but 2006 is where I started to appreciate the big-picture potential: in knowing the Spring component model, I had purchase in all the other domains that Spring would take me. Spring Security meant that I was already halfway towards having secure applications and services. Spring Batch, also released around this time, let me learn the domain of batch processing in terms of the familiar idioms in Spring. I was already halfway there! Spring Integration, also released around this time, let me learn the domain of enterprise application integration, messaging, and event-driven architecture in terms of the familiar idioms of Spring. I was already halfway there. To be clear, there were alternatives for these particular use cases, but none of them integrated so deeply with Spring and its guiding philosophies. I'd have to learn a whole different paradigm, ecosystem, and maybe even language and runtime to get any footing in those alternatives. Spring opened doors and let me cut to the heart of the matter.

Later that year, Acegi was rebranded as *Spring Security*, with Ben Alex handing the leadership reins over to Luke Taylor to guide the framework into its next major chapter: the release of Spring Security 2.0 in April 2008.

The transition into the Spring portfolio brought even tighter integration with the Spring ecosystem and continued refinement of the framework's architecture. Under this new era of stewardship, Spring Security 2.0 introduced namespace-based XML configuration, *dramatically* reducing the boilerplate that had made earlier Acegi configurations notoriously verbose. I can't stress this enough. Spring Framework's main configuration mechanism *used* to be XML, and in 1.0, the only way to wire up objects was to define instances of every bean required for a given application, no

matter how many were required. There was no mechanism for composition. No way to say, *I want all **these** beans in **that** namespace, now*. We'd have to type them out, one by one, and configure them all, one by one. As you can imagine, in the domain of security, there are *many* moving parts, and every wayward developer had to understand all of them to get even the simplest application up and running. You won't have to deal with that in this book, of course. We've had nearly two decades of Java configuration, and more than two decades since XML namespaces and schemas were introduced. I just wanted to register that it used to be unwieldy. I don't know if writing a book is cheaper than therapy, but it's definitely more validating.

The framework evolved well beyond basic authentication and authorization, cementing a flexible architecture centered around servlet filters, authentication providers, method security, and deep integration with the Spring Framework.

As stewardship expanded to a group of maintainers - including Luke Taylor, Rob Winch, and a growing community of contributors - Spring Security continued to modernize alongside both the Java ecosystem and the changing web. At this point, no single engineer has done more to improve Spring Security than its current lead Rob Winch, about whom you'll no doubt hear more from me in the course of this book.

The 3.x generation introduced features such as *Spring Expression Language* (SpEL)-based access control, improved LDAP integration, OpenID support, and built-in protection against common web vulnerabilities like Cross-Site Request Forgery (CSRF). Spring Security 3.2 and 4.x shifted the preferred programming model away from verbose XML toward Java-based, type-safe configuration and fluent APIs.

The framework's evolution accelerated as application architectures shifted toward cloud-native systems and distributed services. Spring Security 5 introduced first-class support for OAuth 2.0, OpenID Connect, reactive applications built with Spring WebFlux, and a modernized password encoding strategy. Spring Security 6, aligned with Spring Framework 6 and Java 17+, further refined its Lambda-based DSL, strengthened secure-by-default behavior, retired legacy APIs, introduced native support for modern WebAuthn passkeys, and fully embraced modern Java and Spring practices. Spring Security 7 introduced multi-factor authentication, pulled in the Spring Authorization Server and Spring Security Kerberos projects, unifying them with the rest of the Spring Security efforts.

Today, Spring Security is a highly modular, extensible framework that secures everything from traditional servlet-based applications to reactive services and OAuth 2.0 Authorization Servers. It provides first-class support for JWTs, decentralized identity, modern authorization models, and deep integration with Spring Boot and the broader Spring ecosystem. Despite more than two decades of evolution, it remains faithful to Ben Alex's original vision: providing a powerful, reusable, and extensible security framework that integrates naturally with Spring applications.

With that high-level overview out of the way, let's talk specifics.

Spring Security for all Contexts

Spring Security has a good story for securing all manner of different workloads. Most of this book assumes that you're building HTTP-based clients and services, but part of the reason I wanted to write this book is precisely because it can do *so much more*! You can use it to secure HTTP clients

and services, MCP clients and services, command line interface (CLI) clients, back-office messaging code, and so much more.

Authentication

Authentication answers the question: *who is this request from*? What's their name? Do we know about them? And with that information, you can answer other more interesting questions like *do they work for our organization?* or *are they a customer with a paid subscription in good standing*? Identity is everything. There's no continuity of service if there are no users for whom to ensure continuity. There are no users if there is no identity.

Authentication works by *challenging* the user for proof of some sort of *factor*, the demonstration of which confirms the identity of the user. A password is a widespread example of this. Anybody can claim to be Rob Winch, lead of Spring Security, but if they don't have his supersecret, well-guarded password, they're not logging into his email or gaining access to his systems. A password demonstrates that the user has knowledge that presumably only they should have.

There are many other factors. One-time tokens, passkeys, and certificates are all examples of authentication factors.

A well-secured system will use multiple factors for authentication, sometimes called multi-factor auth or *MFA*. When should you choose which factor? Authentication factors usually fall into three buckets:

- **knowledge** - something you know. Examples are passwords, PINs, passphrases, security questions, etc. Some weaknesses of this factor are that it can be guessed, phished, reused, leaked, or written down.
- **possession** - something you have. Examples of this are a phone, a hardware security key, a smart card, an authenticator app, a passkey stored on your device, etc. Some weaknesses of this factor are that the device or token can be lost, stolen, cloned in a weak system, or intercepted if it relies on text messages (SMS).
- **inherence** - something you are. Examples of this are fingerprints, face scans, iris scans, voiceprints, etc. Some weaknesses of this factor are that biometrics can be spoofed, mishandled, or impossible to *rotate* if compromised.

Sometimes the demonstrated ability of the user to authenticate in more than one way is, itself, a factor. A user might have access to most of a given system if they present *just* a single factor, like a password, but might have access to more sensitive facets of the system if they present multiple factors.

Authentication in Spring Security

In Spring Security, the core concept of authentication is represented by an object named, unsurprisingly, `Authentication`. Spring Security features an `AuthenticationManager` which in turn delegates to a chain of `AuthenticationProvider` types. There are *dozens* of these included in Spring Security, many of them having specific knowledge on how to authenticate a given user in terms of a given specification or protocol or scenario.

Here's the definition of `AuthenticationManager` in Spring Security.

```
package org.springframework.security.authentication;

import org.springframework.security.core.Authentication;
import org.springframework.security.core.AuthenticationException;

@FunctionalInterface
public interface AuthenticationManager {

    Authentication authenticate(Authentication authentication) throws
AuthenticationException;

}
```

The most common implementation of `AuthenticationManager` is `ProviderManager`, which as I say manages a chain of `AuthenticationProvider` instances.

Here's the definition of `AuthenticationProvider` in Spring Security.

```
package org.springframework.security.authentication;

import org.jspecify.annotations.Nullable;

import org.springframework.security.core.Authentication;
import org.springframework.security.core.AuthenticationException;

public interface AuthenticationProvider {

    @Nullable Authentication authenticate(Authentication authentication) throws
AuthenticationException;

    boolean supports(Class<?> authentication);
}
```

One of the most commonly used `AuthenticationProvider` types is the `DaoAuthenticationProvider`, which ties authentication to users stored in some implementation of `UserDetailsService`. `UserDetailsService` in turn assumes you can load the entire user, including its (encoded and hopefully not plaintext) password from some backing store, like a SQL database, or in-memory.

It sounds like a lot, but keep in mind that you don't have to interact with most of this stuff almost ever. Here's a high-level diagram to make clear the relationship between these types.

[authentication] | *images/security/authentication.png*

1. here's a high-level overview of the flow of authentication requests

And now let's actually pull these pieces together in a simple test. We'll construct these objects ourselves, using Spring Security directly.

```
package com.example.security;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.authentication.ProviderManager;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;

import java.util.List;

@SpringBootTest
class SecurityApplicationTests {

    @Test
    void authentication() throws Exception {

        ①
        var pw = PasswordEncoderFactories.createDelegatingPasswordEncoder();

        ②
        var josh =
User.withUsername("josh").password(pw.encode("pw")).roles("USER").build();
        var rob = User.withUsername("rob").password(pw.encode("pw")).roles("USER",
"ADMIN").build();
        var users = List.of(josh, rob);
        var userDetailsService = new InMemoryUserDetailsManager(users);

        ③
        var daoAuthenticationProvider = new
DaoAuthenticationProvider(userDetailsService);
        daoAuthenticationProvider.afterPropertiesSet();

        ④
        var authenticationProviders = List.of((AuthenticationProvider)
daoAuthenticationProvider);
        var authenticationManager = new ProviderManager(authenticationProviders);
        authenticationManager.afterPropertiesSet();

        ⑤
        var authentication = new UsernamePasswordAuthenticationToken("josh", "pw");
        Assertions.assertFalse(authentication.isAuthenticated(), "the user is not
authenticated");
        var authenticated = authenticationManager.authenticate(authentication);
        Assertions.assertTrue(authenticated.isAuthenticated(), "the user is
```

```
authenticated");
    }

}
```

1. you don't store passwords in plaintext, do you? Of course not! You need to encode it. And Spring Security has a nifty composite implementation of the `PasswordEncoder` hierarchy that I try to use for basically everything.
2. we're going to store users *in-memory*. Obviously, you could source these users from a SQL database. Any implementation of `UserDetailsService` will work. We're just going to hardcode our users for this example. (Which does make the act of encoding the passwords seem a bit silly, doesn't it?)
3. We'll wire up a lonesome `AuthenticationProvider` of type `DaoAuthenticationProvider` which expects a reference to the aforementioned `UserDetailsService`.
4. We'll wire up an implementation of `AuthenticationManager` called `ProviderManager` to handle any and all `AuthenticationProvider` instances. `ProviderManager` will give incoming `Authentication` requests to all configured instances of `AuthenticationProvider`. If any of them can handle it, then processing stops. If not, they either throw an exception or return an `Authentication` with `authenticated == false`.
5. finally, the rubber meets the road. Imagine we're an HTTP client using an HTTP form or an HTTP basic request. Spring Security's filter would intercept those requests, wrap them into a `UsernamePasswordAuthenticationToken` instance, and then ask the `AuthenticationManager` to try to authenticate it. Here, we see that it does so successfully.

Filters

Now that you know, conceptually, about how to *do authentication*, you'll hopefully appreciate that the hardest part about security is getting your code in the right place to do authentication in the first place. In a web application, Spring Security installs a filter that intercepts requests bound for your application, and it is there that code of a kind with the code in that test actually runs.

Filters are a natural way to implement this sort of gatekeeper logic on your application. Spring Security uses filters all over the place to ensure that it is able to put up certain pages for certain use cases (like login pages). You can leverage this mechanism too. Spring has two modalities: reactive and non-reactive. In this book I'm assuming we're using the non-reactive stack. When you bring in Spring Boot's `spring-boot-starter-webmvc` starter, it configures Apache Tomcat by default. You could optionally also use Jetty. Spring Boot provides a natural way to contribute filters, side-stepping the tedious configuration required when interacting with the lower-level Servlet engine.

Here's an example.

```
package com.example.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpFilter;
```

```
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.boot.web.servlet.FilterRegistrationBean;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.Ordered;
import org.springframework.core.annotation.Order;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;

import java.io.IOException;

@Configuration
class CustomFilterConfiguration {

    ①
    @Bean
    @Order(Ordered.LOWEST_PRECEDENCE)
    FilterRegistrationBean<MyFilter> myFilterFilterRegistrationBean() {
        var myFilter = new MyFilter();
        var frb = new FilterRegistrationBean<>(myFilter);
        frb.addUrlPatterns("/test", "/test/*");
        frb.setAsyncSupported(true);
        return frb;
    }

    ②
    static class MyFilter extends HttpFilter {

        @Override
        protected void doFilter(HttpServletRequest request, HttpServletResponse
response, FilterChain chain)
                throws IOException, ServletException {

            IO.println("before " + request.getRequestURL());
            ③
            if (request.getUserPrincipal() == null) {
                ④
                chain.doFilter(request, response);
            } //
            else {
                response.setContentType(MediaType.TEXT_HTML_VALUE);
                response.setStatus(HttpStatus.OK.value());
                var userPrincipal = request.getUserPrincipal();
                response.getWriter().write("""
                        <html>
                         <body>
                            <h1>
                             Hello, %s!
                            </h1>
                         </body>
```

```
                        </html>
                        """.formatted(userPrincipal.getName()));
            }
            IO.println("after " + request.getRequestURL());
        }

    }

}
```

1. You can register beans of type `FilterRegistrationBean` and they'll let you wire up all the same things you might've configured when using `web.xml` or programmatic `Servlet` registration. We can specify in which order the filter is to be processed. I want this to be processed dead-last, *after* all the other filters have run. Here, we're specifying the URL patterns to which to map our `Filter` and whether the `Filter` supports the *async* processing introduced in Servlet 3.0, in 2009.
2. I use the word *filter* to refer to a conceptual thing, but in the Servlet API there are at least two interesting classes to be aware of: `Filter` and `HttpFilter`. We're interested in `HttpFilter` here because we want to intercept HTTP requests, not just generic unspecified requests bound for the server.
3. Spring Security installs a filter *before* our filter, and one of the things it does is *wrap* the raw native `HttpServletRequest` that comes from the native Servlet container in a new class that provides answers to `getUserPrincipal()` by sourcing the information from the Spring Security authenticated principal.
4. We *should* be last, and the request *should* be secured, but if for some reason there's no authenticated principal in the request, then let's just continue the filter chain, letting somebody else have a chance at resolving this. There's nothing we can do.

Where does that user come from? Who put it there? Behind the scenes, when Spring Security authenticates a request, it creates an object of type `Authentication` and stores it in a place that the current ongoing request can get to it. Often, but not always, this is backed by a `ThreadLocal<?>`. You can access the current authenticated user with `SecurityContextHolder`:

```
var authentication = SecurityContextHolder
                        .getContextHolderStrategy()
                        .getContext()
                        .getAuthentication();
var authenticationName = authentication.getName();
```

The name returned here and the name returned from `getUserPrincipal` in the example above should be the same, pointing to the same reference.

Authentication in a Web Application

We'll see how to do so in various other contexts at other places in this book. But for now, let's start with a web application. Let's take these simple concepts and look at them in practice, in a more fleshed-out example implemented using Spring Boot and Spring Security.

Visit the Spring Initializr, add `PostgreSQL Driver`, `JDBC API`, `GraalVM Native Support`, `Spring Web`, `Spring Security`, and `WebAuthn for Spring Security` to the build. I always choose Apache Maven and, of course, the latest version of Java or the latest long-term support (LTS) version of Java. Click `Generate` and open the resulting zip file in your IDE.

We're going to connect this application to a SQL database. As always, we'll use Docker Compose to make short work of this. I've created a `compose.yml` file here.

```
services:
  postgres:
    image: 'postgres:latest'
    environment:
      - 'POSTGRES_DB=mydatabase'
      - 'POSTGRES_PASSWORD=secret'
      - 'POSTGRES_USER=myuser'
    ports:
      - '5432:5432'
```

Run that in the usual way:

```
docker compose up -d
```

Here's our initial `application.properties` configuration.

```
spring.application.name=security
①
spring.datasource.username=myuser
spring.datasource.password=secret
spring.datasource.url=jdbc:postgresql://localhost/mydatabase
②
spring.sql.init.mode=always
```

1. define SQL database connection information
2. tell Spring Boot to evaluate `src/main/resources/{schema,data}.sql`

Let's set up our Spring Security application. Let's install a single controller that we will protect and that will require an authenticated user to access.

```
package com.example.security;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;

import java.security.Principal;
import java.util.Map;
```

```
@Controller
@ResponseBody
class MeController {

    @GetMapping("/")
    Map<String, String> me(Principal principal) {
        return Map.of("name", principal.getName());
    }

}
```

This endpoint is protected. You'll need to authenticate to visit this endpoint: `http://localhost:8080/`.

By default, a Spring Boot application with `org.springframework.boot : spring-boot-starter-security` is completely locked down, and all endpoints will require authentication.

Spring Boot registers a *lot* of common-sense and useful defaults, locking down the application, applying useful authentication methods like HTTP BASIC and form-based login, applying cross-site request forgery (CSRF) protection, and more.

To access any endpoint, a user would need to authenticate. But how? We could specify a single username and password, for demonstration, using properties, but it's almost too contrived. Even for me. So let's instead register a bean of type `UserDetailsService`.

In-Memory Users

We'll start by registering an `InMemoryUserDetailsManager` to see everything work.

```
package com.example.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;

@Configuration
@Profile("memory")
class InMemoryConfiguration {

    @Bean
    InMemoryUserDetailsManager memoryUserDetailsManager(PasswordEncoder pw) {
        var user1 = User.withUsername("user@anotherone.site") ①
            .roles("ADMIN") //
            .password(pw.encode("p@ssw0rd")) //
            .build();
```

```
        var user2 = User.withUsername("josh@joshlong.com") //
            .roles("USER", "ADMIN") ②
            .password(pw.encode("pw")) //
            .build();
        return new InMemoryUserDetailsManager(user1, user2);
    }

}
```

1. we can specify whatever we want for the `username`. In this case I'm using something that looks like an email, but there's no schema or requirements here.
2. users have *authorities*, which help us determine what privileges a given user has. We'll look at these in more depth when we look at authorization.

Start the application and visit the endpoint, and you'll be prompted to authenticate. Use `josh@joshlong.com` and `pw` as the username and password, respectively.

You should see the username returned to you in a JSON response. It worked!

DAO-Backed Authentication

Let's change the `UserDetailsService` implementation to read from our SQL database instead of from in-memory. We'll need some schema to define the tables in which our users and the enumeration of their *authorities* will reside.

I've placed the following file in `src/main/resources/schema.sql`.

```
create table if not exists users
(
    username varchar(200) not null primary key,
    password varchar(500) not null,
    enabled  boolean      not null
);

create table if not exists authorities
(
    username  varchar(200) not null,
    authority varchar(50)  not null,
    constraint fk_authorities_users foreign key (username) references users
(username),
    constraint username_authority UNIQUE (username, authority)
);
```

I've placed the following file in `src/main/resources/data.sql`.

```
delete from authorities;
delete from users;
```

```
insert into users (username, password, enabled)
values ('josh@joshlong.com',
'{bcrypt}$2a$10$i4a5jw4y9LnKS//8DNDVYe2VfC6Jr2m0Ds1j5un5JNhnkJxg6jAT.', true);

insert into users (username, password, enabled)
values ('user@anotherdomain.site',

'{sha256}0a598ce65440a7c4ad18b48fdc2cf28c4e58aec803780f59548535696a23b16624e47ff4f59e6
f39', true);

--
insert into authorities (username, authority)
values ('josh@joshlong.com', 'ROLE_USER');

insert into authorities (username, authority)
values ('josh@joshlong.com', 'ROLE_ADMIN');

insert into authorities (username, authority)
values ('user@anotherdomain.site', 'ROLE_USER');
```

I've taken this schema directly from the Spring Security documentation and the source code for Spring Security. We're using PostgreSQL, but there are equivalent schemas that you could use for various other databases, too.

Delete the conflicting `InMemoryUserDetailsManager` bean from earlier. Let's replace it with one that delegates to these newly minted SQL tables and their data.

```
package com.example.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.JdbcUserDetailsManager;

import javax.sql.DataSource;

@Configuration
class SecurityConfiguration {

    @Bean
    JdbcUserDetailsManager jdbcUserDetailsManager(DataSource dataSource) {
        ①
        var userDetailsManager = new JdbcUserDetailsManager(dataSource);
        ②
        userDetailsManager.setEnableUpdatePassword(true);
        return userDetailsManager;
    }

}
```

1. this implementation of `UserDetailsService` reads user data from the aforementioned SQL tables
2. this property is a relatively new feature.

It allows Spring Security to update the password in the SQL database if the encoding used isn't the default configured encoding in the system.

Restart the program. You should see data in your PostgreSQL `users` and `authorities` tables.

Open up the browser and hit the same endpoint and authenticate with username `josh@joshlong.com` and password `pw`. You should authenticate just fine. But *how*? We'll need to dive into password encoding before we have a good answer.

Password Encoding

Looking at the output, you'll notice that the passwords written to the database were basically garbled (*encoded*) text preceded by a prefix in curly brackets: {, the algorithm name, and }.

This was intentional: I wanted these passwords to work with the `PasswordEncoder` configured here:

```
package com.example.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
class PasswordConfiguration {

    @Bean
    PasswordEncoder passwordEncoder() {
        return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    }

}
```

A `PasswordEncoder` is an object that takes plaintext Strings and *encodes* them with an algorithm. Spring Security ships with dozens of implementations for algorithms like SHA (*do not* use this one!), SHA 256 (maybe don't use this one either), PBKDF2, SCrypt, BCrypt, Argon2, etc. Many of the implementations are duplicated, in fact, because there is dual support in Spring Security for various algorithms from the third-party BouncyCastle library *and* from the third-party Password4J library. Indeed, most of the good implementations come from either the BouncyCastle or Password4J libraries and require their presence on the CLASSPATH.

You could use these algorithms directly by constructing the relevant instances of the `PasswordEncoder`, like so:

```
var bcrypt = new BCryptPasswordEncoder();
```

I always prefer instead to use the composite that comes from `PasswordEncoderFactories`, which is itself a composite `PasswordEncoder`. It maintains a dictionary of different algorithm names mapped to an instance of their implementation. Security is a fast-moving space. Algorithms that are deemed secure and uncrackable today might be deemed cracked and insecure tomorrow. Spring Security reserves the right to bump the default algorithm as it evolves, should the need arise. Right now, the best algorithm provided out-of-the-box with no need for any other dependency is `BCryptPasswordEncoder`.

When you use the `DelegatingPasswordEncoder` from `PasswordEncoderFactories` today, at the time of this writing in mid-2026, the default algorithm used is `bcrypt` (which maps to an instance of `BCryptPasswordEncoder`, as shown above). That might change tomorrow as the algorithms are discovered to be less secure. If you have BouncyCastle or Password4J on the CLASSPATH, I'd prefer Argon2. Use Argon2 when available because it is memory-hard and tunable for memory, CPU time, and parallelism, making large-scale GPU/ASIC cracking harder. OWASP currently recommends Argon2 as the first choice.

The `DelegatingPasswordEncoder` prefixes all passwords with the name of the algorithm used to encode it.

Let's walk through the (rough) authentication flow, with the `PasswordEncoder` in mind.

- a user enters a username and a password in a form on your website and then submits the request
- Spring Security intercepts the request and constructs a `UsernamePasswordAuthenticationToken` with the username and password as parameters.
- the configured `AuthenticationManager` is asked to authenticate the request.
- the `AuthenticationManager`, in this case, is probably an instance of `ProviderManager`, so it in turn delegates to a chain of `AuthenticationProvider` instances, which get a chance to authenticate the `Authentication` request.
- finally, the `DaoAuthenticationProvider` instance loads the user from the `UserDetailsService` using the presented username.
- It needs to check that the passwords match, so it obtains an instance of the `DelegatingPasswordEncoder` and checks that the presented password (in plaintext) matches the one stored in the database (encoded, and prefixed with an algorithm name) by calling `PasswordEncoder#matches(CharSequence plain, String encoded)`. Internally, this uses the prefix to identify the ultimate implementation of `PasswordEncoder` to use, then encodes the presented password with that implementation.

You can of course configure your own `DelegatingPasswordEncoder` if you like to use Argon2 (or anything else). Add Password4J to the CLASSPATH: `com.password4j` : `password4j` : `1.8.4`. I'm using that version in the middle of 2026. (You should use whichever version is the latest-and-greatest as you're reading this.)

```
package com.example.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.DelegatingPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.crypto.password.StandardPasswordEncoder;
import org.springframework.security.crypto.password4j.Argon2Password4jPasswordEncoder;

import java.util.Map;

①
@Configuration
@Profile("password4j")
class CustomPasswordEncoderConfiguration {

    @Bean
    DelegatingPasswordEncoder delegatingPasswordEncoder() {
        var argon = new Argon2Password4jPasswordEncoder();
        var defaultPasswordEncoder = "argon@Password4j";
        var mapOfEncoders = Map.<String, PasswordEncoder>of(defaultPasswordEncoder,
argon, //
                "bcrypt", new BCryptPasswordEncoder(), //
                "sha256", new StandardPasswordEncoder()②
        );
        return new DelegatingPasswordEncoder(defaultPasswordEncoder, mapOfEncoders);
    }

}
```

1. this implementation configures the Password4J-powered Argon2 implementation as the *default* encoder, with optional support for BCrypt.
2. we're registering an encoder for `sha256`, but keep in mind that this implementation is deprecated! In a happier world, we wouldn't use it.

Make sure you replace the other `PasswordEncoder` implementation from the `PasswordConfiguration` class with this one if you'd like to use it.

We've configured custom password encoding so that we can handle the data in the SQL database that uses `sha256` encoding. It's a deprecated algorithm, and we'd certainly prefer it if people weren't using it at this point. The trouble is, there's always *legacy*. Maybe people used that algorithm before, and now they need to migrate the passwords to the new algorithm. Encoded passwords are, by definition, *challenging* to unencode, and so we can't convert them to a new password unless we somehow have the plaintext. The only time the system has the plaintext version of the password is when the user logs in and presents a password that, when encoded with the same algorithm, matches the one in the database. Earlier, when we configured the `JdbcUserDetailsManager`, we set `setEnableUpdatePassword` to `true`. This allows Spring Security to re-encode a password in the

database that's encoded using something besides the current *default* password encoder when the user authenticates successfully.

Consult the `data.sql`, or your database, and you'll see that the user `user@anotherdomain.site` has a password encoded with `sha256`. So, restart the application and visit http://localhost:8080/ and then authenticate as `user@anotherdomain.site` and `pw`. Check the database again, and you'll see it's been migrated to use the *default* password encoder. If you're using the `PasswordEncoder` from `PasswordEncoderFactories`, you'll see it's been encoded using `bcrypt`. If you're using the custom `PasswordEncoder` we set up above, you'll see it's been encoded using `argon@Password4j`. Either way, *much* better than `sha256`! Take a victory lap, you've earned it. You can say that you're storing passwords as safely as possible.

But Passwords Are Bad!

Yah, I said it! Passwords are a terrible *factor* for authentication. Which is a shame because, for the vast majority of systems in production today, they're all we've got. But they're also harder to maintain, most people don't have the tools to deploy them correctly, and - to make matters worse - so much of the received wisdom around them has worked against us.

Humans aren't built to remember dozens of high-quality, unique, and complex passwords. It would be great if everybody used a good password manager like Bitwarden, 1Password, LastPass, etc. But they don't. Or, worse, they might've accidentally started using password managers in one context (e.g., Google Chrome) but gotten locked out because they switched away from whatever that context was (e.g., Windows to macOS, Android to iPhone, Google Chrome to Safari, etc.). The result? People don't develop good password hygiene. They reuse passwords. They scrawl them on stickies affixed above their monitor. Or, worse, in a text file on the public cloud.

Insufficient engineering practices are often to blame. In the example above, we showed how to store passwords in an encoded fashion *and* how to migrate them to better encodings whenever possible. Sometimes folks don't do the right thing. Meta (formerly known as Facebook) was caught storing hundreds of millions of user passwords in plain text [https://krebsonsecurity.com/2019/03/facebook-stored-hundreds-of-millions-of-user-passwords-in-plain-text-for-years/?utm_source=chatgpt.com] for years in 2019. In 2019, it also came out that Google had stored some passwords in plaintext [https://www.wired.com/story/google-stored-gsuite-passwords-plaintext/?utm_source=chatgpt.com] since 2005. The 2012 Yahoo! Voices attack [https://en.wikipedia.org/wiki/2012_Yahoo_Voices_hack?utm_source=chatgpt.com] revealed that Yahoo! had stored around 450,000 passwords in plaintext. If it can happen to these organizations, it can happen to anyone. (Why can't everyone just use Spring Security?)

Now you might say that those issues are a sign of poor password management, and I'd say you can't have poor password management if you don't have passwords. We can both be right.

Bad "Received Wisdom"

Sometimes, it boils down to bad "best practices." Should you rotate your passwords every month? No. Password rotation used to be recommended because the assumption was that passwords could be stolen without anyone noticing. Changing them periodically would reduce the window during which an attacker could use them. This made sense before the widespread adoption of multifactor

authentication, breach detection, password managers, and monitoring for compromised credentials. But periodic rotation makes security *worse*. Research shows that people tend to increment the number (`Passw0rd1` → `Passw0rd2`) or make tiny, predictable modifications, or write passwords down, or choose weaker passwords because they're tired of changing them. All of which makes passwords *easier* to guess or derive. It's much better to securely store a long, secure password in a password manager rather than changing it without proper handling every month (or whenever).

Authentication Around the World

Indeed, for government work in many government contexts, passwords are discouraged or disallowed.

In the **United States**, the Office of Management and Budget issued the federal Zero Trust strategy requiring agencies to adopt phishing-resistant multifactor authentication (MFA) for users whenever possible.

The **European Union** is one of the strongest proponents of MFA: the NIS2 directive explicitly includes the use of multifactor authentication or continuous authentication solutions, where appropriate, as one of the required cybersecurity risk-management measures for covered entities (energy, healthcare, banking, transportation, cloud providers, public administration, digital infrastructure, etc.). The European Union's Digital Operational Resilience Act (DORA) for financial institutions requires strong identity and access management. While it does not literally say "passwords are banned," password-only authentication generally won't satisfy modern expectations for privileged or high-risk access. And the Revised Payment Services Directive introduced *Strong Customer Authentication* (SCA), which requires authentication using at least two independent factors from categories such as knowledge, possession, and inherence. A password alone is not enough for many electronic payment transactions.

The **United Kingdom**'s *National Cyber Security Centre* strongly recommends MFA for important accounts and increasingly recommends phishing-resistant authentication for government systems.

The Monetary Authority of **Singapore** requires strong authentication for many banking and financial services. Banks generally must use multi-factor authentication for customer access and sensitive transactions.

Japan does not broadly prohibit passwords. However, government cybersecurity guidance strongly encourages MFA, and major financial institutions require additional authentication. Passkeys are increasingly being adopted across both government and private sectors.

South Korea has historically had some of the world's strongest online authentication requirements. Although earlier systems relied heavily on certificates, modern regulations increasingly permit FIDO authentication, biometrics, and MFA instead of password-only systems.

We won't be completely escaping passwords for a while, but every journey of a thousand miles starts with but a single step. It starts with us. Let's explore some alternatives to passwords. But before we can do that, we need to think about how to customize a Spring Security application. So far we've just registered beans of the right type in the right place, and things have sort of worked out.

Remember when I told you that there's a giant filter sitting in between your requests and the back-end application logic? Well, we can configure how that works by defining the `SecurityFilterChain` ourselves, like this.

```
package com.example.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@Profile("sfc")
class SecurityFilterChainConfiguration {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity security) throws Exception {
        return security //
            .formLogin(Customizer.withDefaults())
            .httpBasic(Customizer.withDefaults())
            .csrf(Customizer.withDefaults()) //
            .cors(AbstractHttpConfigurer::disable) //
            .authorizeHttpRequests(a -> a //
                .requestMatchers("/admin")
                .hasRole("ADMIN")//
                .requestMatchers("/**")
                .authenticated() //
            )//
            .build();
    }

}
```

We won't get into what's happening here just yet. Suffice it to say that in defining this bean, we are overriding the defaults that Spring Boot provides for Spring Security and that you get for free when you add `spring-boot-starter-security` to the CLASSPATH. By default, Spring Boot declares a bean of type `SecurityFilterChain` that:

- enables HTTP Basic authentication
- enables form login
- requires that all access to any endpoint be authenticated

Historically, if you wanted to change this, you'd have to define a `SecurityFilterChain` bean of your own and you'd instantly lose the default protection Spring Boot gave you. So, your first order of

business was to immediately redefine the same bean, which looks more or less like this:

```
package com.example.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Profile("defaultsfc")
@Configuration
class BootsDefaultSecurityFilterChainConfiguration {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity security) throws Exception {
        return security //
            .formLogin(Customizer.withDefaults())
            .httpBasic(Customizer.withDefaults())
            .authorizeHttpRequests(a -> a.anyRequest().authenticated())
            .build();
    }

}
```

Only then could you start customizing the security for the application. But for the brief moment between overriding the old bean and fleshing out the new one, your application was *less* secure than if you'd done nothing at all!

Not a good result. In Spring Security 7, we introduced a way to iteratively *add* to the default security, instead of replacing it wholesale; we only need to define a bean of type `Customizer<HttpSecurity>`. For most of this book, I will do exactly that *almost* everywhere. I show you the `SecurityFilterChain` approach because on the occasions when you want to make authorization more granular, or when reading older code and older text, you'll need to appreciate what's happening.

One-Time Tokens, or "Magic Links"

You ever use Slack?

If so, then I am so very sorry. I am not a huge fan of the huge fans required to cool the computer running Slack. Or indeed almost any Chromium or Electron-based software.

When you authenticate, it prompts you to enter your email. You oblige it, and it sends you an email. You click on the link in your email, and now you're authenticated. You have access to your email, so therefore you are who you say you are. Here, it's an email. But it could be any out-of-band

messaging medium. Email, WhatsApp/SMS (though these two, and anything tied to a spoofable phone number, are increasingly discouraged these days), Discord, etc.

Do these services require a password? Almost certainly. And probably another factor. And you can trust that Google (behind Gmail), Apple (and iCloud), Microsoft (and Active Directory and Outlook), etc. have all done a ton to ensure the integrity of passwords for their services. They have whole teams working on the security of passwords for their products. So in a way this just kicks the can down the road, but it does remove a significant burden from your organization, and it simplifies the user experience while also making their lives (hopefully) more secure.

I don't want to set up Twilio, JavaMail, or SendGrid here, so let's simplify things and just imagine that the out-of-band messaging medium by which people are communicating is our application's logs. Is this *real*? No, of course not. But it'll help to see things work end-to-end.

Let's try it out. First, we'll need to enable one-time tokens. Do so within a `Customizer<HttpSecurity>`.

```
package com.example.security.ott;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;

@Configuration
class OttConfiguration {

    static final String SENT_URL = "/login/ott/sent";

    @Bean
    Customizer<HttpSecurity> ottCustomizer(ConsoleOneTimeTokenGenerationSuccessHandler
handler) {
        return security -> security ①
            .oneTimeTokenLogin(config -> config //
                .tokenGenerationSuccessHandler(handler))//
            .authorizeHttpRequests(a -> a
                ②
                .requestMatchers(SENT_URL)
                .permitAll()); //
    }

}
```

1. first, tell Spring Security to allow one-time tokens as an authentication mechanism, pointing it to an implementation of `OneTimeTokenGenerationSuccessHandler`, which we'll look at momentarily.
2. then we need to make publicly accessible an HTML page (visible under the path `/login/ott/sent`) that we intend for the users to see when authenticating.

In the configuration class, I've extracted out the HTML page's path into a `static final` constant so

that I can reference it in a few other places for consistency. To be 100% honest with you, I'd normally write basically all of the code to do with wiring this up in the same outer `OttConfiguration` class, using a lambda and private static inner classes to do the work. There just isn't that much complexity here. But this is a book, and I want the examples to be clear and to introduce the concepts one at a time, so I've extracted them into separate classes. And since I had more than one class about the same *thing*, I've put them all in the same nested package, `ott`.

What is this `OneTimeTokenGenerationSuccessHandler`, I hear you say? It's a callback interface that gets involved once the user has entered their username on the login page. You can use this as a chance to set up any state and then look the user up and email them, call or text their phone number or WhatsApp, etc.

But again, I want to keep things simple, so I'll simply print on the console the URL to which the authenticating user should go.

```
package com.example.security.ott;

import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.ott.OneTimeToken;
import
org.springframework.security.web.authentication.ott.OneTimeTokenGenerationSuccessHandl
er;
import
org.springframework.security.web.authentication.ott.RedirectOneTimeTokenGenerationSucc
essHandler;
import org.springframework.stereotype.Component;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.io.IOException;

import static com.example.security.ott.OttConfiguration.SENT_URL;

@Component
class ConsoleOneTimeTokenGenerationSuccessHandler implements
OneTimeTokenGenerationSuccessHandler {

    ①
    private final OneTimeTokenGenerationSuccessHandler successHandler = new
RedirectOneTimeTokenGenerationSuccessHandler(
            SENT_URL);

    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response,
OneTimeToken oneTimeToken)
            throws IOException, ServletException {

        ②
        var url = ServletUriComponentsBuilder.fromContextPath(request) //
```

```
            .path("/login/ott") //
            .queryParam("token", oneTimeToken.getTokenValue()) //
            .build();

        ③
        IO.println("sending email " + oneTimeToken.getUsername() + " who should go to
" + url);

        ④
        this.successHandler.handle(request, response, oneTimeToken);
    }

}
```

1. this is a built-in `OneTimeTokenGenerationSuccessHandler` that simply redirects to a particular view. We will want to do that, eventually.
2. but first, we need to know where to send the user. Spring MVC provides this handy convenience class called `ServletUriComponentsBuilder` that makes building up a relative URL pretty darned easy.
3. finally, we print out where we expect the user to go to access the system. Here, we're conveying this link on the system console, but in a real application we'd send an email or SMS or something.
4. finally, with all the setup done, we need to respond to the HTTP request. We do so by redirecting to a URL on the server that'll render an HTML page.

Let's quickly touch on that `ServletUriComponentsBuilder` use. Spring uses the current `HttpServletRequest` to build up a fully qualified URL. But what happens if your service is behind a gateway or a router? You don't want your users trying to click on a URL bound for `http://192.168.2.5:20482`, you want them going to `http://example.com/`. If your gateway forwards proxy headers like `Forwarded` or the `X-Forwarded-*` headers, then Spring can be made to honor those with the `ForwardedHeaderFilter`. You can customize how this is set up with `server.forward-headers-strategy` in a Spring Boot application. I usually use `server.forward-headers-strategy=framework`. If you use this, then be sure to *strip* those headers at the trust boundary (your gateway), and then add them again before sending it to your downstream service. Spring Boot has no way of knowing whether these headers are trustworthy!

When we set up the application, we configured Thymeleaf so that we could produce HTML views like the one we're presenting to the user after sending the authentication link. Let's do that here.

```
package com.example.security.ott;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

import static com.example.security.ott.OttConfiguration.SENT_URL;

@Controller
```

```
class OttController {

    @GetMapping(SENT_URL)
    String sent() {
        ①
        return "ott-sent";
    }

}
```

1. this is a regular `@Controller` with no `@ResponseBody` to be found. Spring interprets the return value as a template to be resolved via the autoconfigured `ViewResolver`. In this case, it'll resolve to `src/main/resources/templates/ott-sent.html`.

Here's the Thymeleaf page:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Sent</title>
</head>
<body>
<h1>
    You've got console mail!
</h1>
</body>
</html>
```

Nothing fancy.

With all this in place, start the application and hit http://localhost:8080. Enter the username (`josh@joshlong.com`). It'll tell you that you've got console mail. Consult the console, and you'll see a link. Copy and paste it into the browser, and you should see your username.

Easy!

My friend James Ward [https://linkedin.com/in/jamesward] did something interesting with this mechanism for another book we're jointly writing (and perhaps he'll do it for this book?), where users who bought the book on a website would have had to enter their emails. Those emails are turned into users in a database. Somebody comes along, signs in, and that grants an OAuth token which is then used to provision and provide access to an MCP service that in turn has access to the chapters of the book. Users can point their Claude Codes and Gooses and JetBrains Airs to the MCP service and consume the content and ask questions about it as MCP resources. Super innovative!

One-time tokens are great for passwordless authentication. It's important to remember their limitations, too. They are *not* signup links to confirm emails. Nor are they password-reset links. And of course, if somebody loses access to the account where the out-of-band communication is sent (email, SMS, whatever), then they're now effectively locked out of *two* systems!

One-time tokens are an easy way to build upon the considerable technical *and* legal infrastructure that companies like Google, Microsoft, Apple, etc., have built up to manage passwords and fight breaches and the like. They allow you to secure a system while avoiding passwords. That said, there is a password in the mix. It's just not *your* password to manage.

The Powerful Passkeys

Passwords are the weakest link. By the early 2000s, companies like PayPal began exploring alternative authentication methods, recognizing that the password model was fundamentally broken. The industry-wide acknowledgment that passwords needed to be eliminated entirely came in 2013 with the founding of the FIDO Alliance (Fast Identity Online). Major technology companies like Microsoft, Google, Apple, Meta, and others created the FIDO Alliance with a mission to eliminate password-based authentication through open standards. You can see the full roster of contributing members [https://fidoalliance.org/members/] here.

The FIDO Alliance's work produced two major specifications. FIDO U2F (Universal Second Factor) enabled secure two-factor authentication using hardware security keys, while FIDO2—developed in close collaboration with the World Wide Web Consortium (W3C)—introduced a more comprehensive approach. FIDO2 consists of two complementary standards: the Client to Authenticator Protocol (CTAP), which defines how external authenticators communicate with client devices, and Web Authentication (WebAuthn), which provides the standardized browser API that allows websites to register and authenticate users using cryptographic credentials. WebAuthn was officially released as a W3C Recommendation in 2019 and is now widely supported across modern browsers. This is the bit that makes this interesting for me: what we're about to look at isn't some hypothetical, would-be-nice thing that might ship in the not-too-distant future; it's already here!

Rather than creating a shared secret like a password, a user's device generates a unique public/private key pair for each website or application. The private key never leaves the user's device. It's kept secure in hardware such as a phone's Secure Enclave, while only the corresponding public key is registered with the service. When signing in, the device proves possession of the private key by cryptographically signing a challenge from the server, typically after the user authenticates locally using biometrics, a PIN, or another device-unlock mechanism. Because there is no reusable password to steal, passkeys dramatically reduce the risk of phishing (a public key stored by the server is useless to attackers), credential stuffing, and password database breaches (because servers do not store reusable secrets).

However, early FIDO2 deployments had significant limitations. They often required dedicated hardware security keys such as YubiKeys or were tied to a single device, meaning users could lose access if that device was lost. The major breakthrough came in 2022–2023 when Apple, Google, and Microsoft committed to expanding the FIDO standard to support what became known as "passkeys"—FIDO credentials that could be securely synchronized across a user's trusted devices through cloud-based credential managers such as LastPass, BitWarden, 1Passwod, iCloud Keychain, Google Password Manager, and Windows Hello. Now a passkey could be created once and used seamlessly across phones, tablets, and computers while remaining protected by each device's secure hardware and local authentication. This eliminated much of the backup-and-recovery problem while maintaining strong security.

The term "passkey" itself represents an evolution in how these technologies are presented to end

users. By synchronizing FIDO credentials across devices and making the underlying cryptography largely invisible, the industry adopted “passkey” to describe these user-friendly, discoverable credentials. When you use a passkey today, you’re benefiting from over a decade of standards work and billions of dollars in platform investment. FIDO Alliance standards provided the vision and interoperability framework; the W3C standardized the browser-facing WebAuthn API; browser vendors and operating system providers implemented the technology; and online services integrated support into their applications.

So, all together now: Passkeys rely on public-key cryptography. Each passkey consists of a unique public/private key pair, with the private key securely stored on one of your trusted devices. When you authenticate, the private key is unlocked using something that’s easier for you to remember or present—such as your face, your fingerprint, or your device PIN—and then used to prove your identity cryptographically.

Implementing Passkeys in Spring Security

Now for the easy part. Whereas the explanation took a fair bit of time, the implementation is trivial.

Introducing OAuth

I have been using Yelp [https://www.yelp.com] for a *very* long time.

Since 2007, I'd guess. I was living in Phoenix, Arizona at the time, and I'd started making regular pilgrimages to my hometown of Los Angeles, California. While I was there, I wanted to make the most of the limited time I had in town, so I started looking for tools to help.

Enter: Yelp.

This was the beginning of my journey as a prolific traveler. I love travel. I maintained my Yelp account and even uploaded a profile photo - one that's still there to this day, I think - of me from a trip to Manila, Philippines.

I loved my account. I love Yelp! It connected me with the world. But there's a limit to my love, and a limit to how much I want it connecting with *my* world.

Did you know there was a (mercifully) brief period when Yelp asked users for their Gmail usernames and passwords so it could scan their contacts and help them connect with friends already on Yelp? Yes, that's right. Yelp wanted you to authenticate with your Gmail account on their website so they could log in to your email and *poke around*, basically.

Insanity! Or at least, by today's standards, it is. But back then, there really wasn't an alternative.

The All-or-Nothing Problem

The internet was becoming a more connected place. Companies were beginning to build on the promise of interconnected services, and these kinds of hurdles were common. Yelp wasn't the only site that asked you to enter credentials on its property so it could log in on your behalf somewhere else.

And we all just sort of went with it because, after all, what was the alternative?

This was the all-or-nothing problem. And it led to even more problems.

I trusted Yelp, so I'm sure it would have handled my data with care. But the same couldn't be said of every service that asked. This created a fraught situation where, with every integration and authentication, you were potentially opening yourself up to catastrophic data loss, a loss of privacy, or worse. Yelp would've been fine, but what about some less-than-scrupulous website? What if they went rogue with your credentials? You might not even have had a way to undo the damage if you later changed your mind, because they had your password; they could lock you out of your own account. Also: how would most of these websites have stored your password? Presumably, they'd have to use a two-way hash, encoded using some sort of algorithm and then decoded with the key. But that means that your Google password was one stolen key away from being stored in plaintext! As bad as the AI-driven CVE situation is today, can you imagine if every site you integrated with had access to your *Google password*? You'd have to trust that every one of them doesn't get *pwned*! Sleeping would be *much* more difficult! OK, I'll stop. I know it's all too deeply upsetting to consider. Mercifully, things improved.

Open Authorization for an Open Web

Developers across the SaaS community - particularly at Twitter and Ma.gnolia - realized there had to be a better way for systems and services to federate identity without sharing passwords.

OpenID, the reigning identity protocol of the day, handled *who you are* well enough, but it didn't cleanly cover the problem Twitter and Ma.gnolia kept hitting: third-party apps that wanted to *act on a user's behalf* without ever holding the user's password. By 2007, Chris Messina (yes, the hashtag guy), David Recordon of Six Apart, Twitter's Blaine Cook, and Ma.gnolia's Larry Halff were sketching out something new. Yahoo!, AOL, Flickr, and Google all pitched in engineering effort.

In December 2007, the final draft of OAuth 1.0 was released to the world.

OAuth

OAuth (Open Authorization) is a valet key for the internet. A valet key starts the car and opens the driver's door, but it won't open the glove box or the trunk. OAuth works the same way: it lets a third-party application act on your behalf with a narrowly scoped key, without ever handing over your password.

You have probably used OAuth before. If you've ever clicked "Log in with Facebook" or "Log in with Google" or "Log in with Apple" on a website, you were using OAuth. Instead of creating a new username and password for that site, you're granting it permission to read a small slice of information from your account at the identity provider - usually just enough for the site to put together a profile and know what to call you.

People incorporate OAuth into their applications in all manner of different ways, but the basics are the same. You're using an *OAuth Client* - the application running in your browser, on your phone, or on your desktop - and it tries to access a resource that requires a token.

The OAuth Client redirects you to an *OAuth Authorization Server* (like Okta, Keycloak, Auth0, Active Directory, Clerk, etc.) where you authenticate. It is this OAuth Authorization Server that knows who you are, or can talk to something that does. Once you've authenticated, the OAuth Authorization Server issues a token and sends it back to the OAuth Client. There are several different flows for that handoff, each with its own trade-offs; we'll meet them later.

The OAuth client can now make requests on behalf of the user to APIs (these are called *Resource Servers*) using that OAuth token.

These *Resource Servers* will receive requests and need to validate them. If they receive a request that doesn't have a token, then the request is rejected. If the request has a token, and the token is a stateless JSON Web Token (JWT), then the token is signed and can be validated cryptographically without talking to the Authorization Server for each request. This works because the resource server has obtained the Authorization Server's public cryptographic keys on startup. The resource server validates the token *in-memory* using these public keys. It verifies the signature, the expiration date, and the issuer completely offline.

Otherwise, if the token is stateful (an opaque reference token), then the resource server *must* contact the Authorization Server on every single request.

The tokens contain information about what rights (*claims*) the user has. These tokens support *authorization*.

OpenID Connect (OIDC)

OAuth was a huge leap forward, but it didn't handle every interesting use case. One of the first things people did once they had a valid access token was use it to ask the provider's API *who is this user?* - their email, their name, a profile picture - and then sign them into their own application based on the answer. The implication was that providers like Google or Facebook had already done the work of confirming the email belonged to the person (and sometimes even checked government-issued ID along the way), so an application could trust that a returned email really *was* that person's email. If Google and Facebook both returned the same email, it really was the same person across two different ecosystems. This pattern became the basis of the early *Sign In with <X>* capabilities all around the internet, and let services focus on building interesting things instead of solving identity over and over again.

People were using OAuth to do *authentication*, and it caught on fast. The trouble was that every provider exposed those identity details in its own bespoke way, and it became pretty self-evident after OAuth 2.0 that there was a gap.

Enter OpenID Connect (OIDC).

OIDC launched in 2014 as a thin, standardized identity layer on top of OAuth 2.0. Its creation was a massive collaborative effort involving engineers from Google, Microsoft, Salesforce, Ping Identity, and Yahoo, with Nat Sakimura, John Bradley, Michael B. Jones, Chuck Mortimore, and Breno de Medeiros among the key contributors.

What OAuth Gives You

OAuth introduced three things you didn't have when applications were just trading passwords around.

First, **tokens instead of credentials**. Even if an attacker steals a token, they only have access for a limited time and only to the data the token was issued for.

Second, **scopes**. The user (or the developer on their behalf) declares up front what an application is allowed to do - read your calendar, post on your behalf, see your profile - and the token carries that entitlement. The valet key gets the car moving without unlocking the trunk.

Third, **revocation**. The user can cut off an application's access at any time without changing their password and without affecting any of their other connected apps.

There are many scenarios where this matters:

- Third-party applications and services: applications that want to access services like Google Drive, Twitter feeds, or Facebook posts.
- Mobile applications: smartphone applications that access web services but don't want to store passwords.

- Content aggregation: an app that pulls data from multiple sources (e.g., various email accounts).
- Federation: if you need to maintain secure integrations with a number of OAuth platforms, it's possible to use an OAuth IDP like Spring Authorization Server to act as a facade.

Evolution of OAuth

OAuth has evolved over the years to address shortcomings and to better meet the requirements of developers and organizations. There are two main versions of OAuth:

OAuth 1.0: The original specification, finalized in December 2007 and later refined as RFC 5849 in 2010. It was complex, required clients to sign every request cryptographically, and was painful to implement correctly. OAuth 1.0a tightened a few security gaps, but we'll leave both behind.

OAuth 2.0: Standardized as RFC 6749 in 2012 as a successor, it simplifies the protocol and separates the roles of obtaining credentials from using them. It requires TLS for the authorization and token endpoints - the whole protocol assumes you're operating over HTTPS - which is a large part of what lets it drop OAuth 1.0's request-signing machinery.

OAuth vs. SAML

You'll often see OAuth contrasted with SAML (Security Assertion Markup Language), and you'll often see the contrast drawn as "SAML is for authentication, OAuth is for authorization." That's a useful starting point but it overstates the case - as you just saw, OAuth was being used for authentication well before OIDC formalized it.

The more practical differences:

	OAuth 2.0 / OIDC	**SAML 2.0**
Era and ecosystem	2012 onward; mobile, SPAs, APIs	2005 onward; enterprise SSO
Wire format	JSON, with tokens that may be opaque or JWTs	XML assertions, signed and sometimes encrypted
Transport	HTTP redirects and JSON over HTTPS	HTTP redirects, form POSTs, and SOAP bindings
Typical use	App-to-API delegation; "Sign in with…"	Enterprise SSO into web apps from a central IdP

Both are still actively deployed. SAML is deeply entrenched in enterprise SSO; OAuth 2.0 and OIDC dominate almost everything newer.

Towards a Passwordless Future

One of OAuth's quiet superpowers is that it centralizes authentication. Authentication is a hard problem, and there are a million things to get right for it to even begin to be considered secure. Okta, Google, Meta, GitHub, and the Spring team have all committed serious engineering effort to getting it right so that you don't have to. Done well, a good OAuth integration can dramatically

reduce the number of passwords you need to maintain to live on the internet.

In this book, we're going to focus on the Spring Authorization Server. Once it's at the heart of your SSO solution, identity for your organization collapses down to a single credential.

But what about *no* credentials? Or better, more secure alternative credentials?

That's where things are heading. Large providers like Apple, Google, and Microsoft are pushing *passwordless* logins tied to device biometrics, built on WebAuthn - and, increasingly, on passkeys, which are discoverable WebAuthn credentials synced across a vendor's ecosystem so you can sign in from any of your devices.

Spring Authorization Server can be extended to participate in this future too, and by the end of the book you'll do exactly that. :code: ..

Introducing the Spring Authorization Server

In this book we're going to look at how to use Spring's OAuth stack to *secure all the things*. At the center of this discussion, necessarily, must be an OAuth authorization server. To be fair, you could use *any* valid, modern OAuth authorization server to do most of the things we'll discuss in this book. OAuth is an open protocol, after all.

Why Spring Authorization Server

Technically, the Spring Authorization Server is an OAuth and OIDC compliant solution just like any other. But as is so often the case, the decision rests on intangibles that go well beyond just the technical merit of the project.

Really, there are *two* questions before us: why should existing applications move to the Spring Authorization Server, and why should new projects start with it?

It's 2026 as I write this, and if you've got software in production, you probably have an OAuth authorization server deployed somewhere. We are spoilt for choice these days!

The space is crowded. On the commercial side you've got Okta (and the Auth0 product they acquired), Microsoft Entra ID, Amazon Cognito, Google's Identity Platform, and Ping Identity (which now includes ForgeRock), alongside a newer cohort of developer-first SaaS shops like Clerk, WorkOS, Stytch, Frontegg, and Descope. On the self-hosted side, Keycloak is the eight-hundred-pound gorilla, with Ory, Authentik, Zitadel, Dex, and Apereo CAS each holding meaningful turf. There are dozens more.

Most of them would work just fine in place of the Spring Authorization Server. I love the Spring Authorization Server precisely because it works just fine in place of most of *them*, and it works in a way that's natural for anyone already living in the Spring ecosystem.

Let's look at the technical and legal facets that drive that choice.

Technical

The pitch is simple: it's a Spring Boot starter.

If you've ever deployed a Spring Boot application, you already know how to deploy the Spring Authorization Server. The same configuration model, the same Actuator endpoints, the same Micrometer metrics, the same container image story, the same CI/CD pipeline. Your team's existing muscle memory transfers over.

Owning the IDP unlocks a few things that are awkward, expensive, or simply not possible with a SaaS:

- **Your data stays where you put it.** Users persist into the same database your application already uses - the same Postgres instance, the same backup story, the same migration tooling, no data leaving your network. That matters in regulated industries, and it matters anywhere a vendor's data-residency story doesn't line up with yours.

- **Custom claims from your business systems.** You can mint tokens that carry the exact claims your services need - tenant ID, role hierarchy, feature flags - read straight from your domain model, without paying a SaaS for a "rules engine" and writing JavaScript inside a vendor's UI.
- **Federation on your terms.** Hook up to your existing LDAP, your Active Directory, your social providers, your customer's SAML IDP - and present a single, consistent OAuth surface to your application code.
- **The same scaling story as the rest of your fleet.** Horizontal scaling, blue-green deploys, traffic shaping, observability - all the things you already do with Spring Boot apply to your auth layer.

When something doesn't fit, you reach for the same autoconfiguration override pattern you use everywhere else in Spring Boot - declare a bean of the type you want to replace, and Spring Boot backs off its default. There's no support ticket, no quarterly contract renegotiation, no rate limit on your own data.

Legal

Quick disclaimer, then the substance: *I am not a lawyer*, I am not your security consultant, and nothing in this book is legal or compliance advice. I work on the Spring team and I'm here to teach you the technology. I'll point at good practices where I can, but I can't cover every jurisdiction, every industry regulation, and every threat model, so I'm not going to try.

Here's why that matters. User management is a minefield, and when something goes wrong - and eventually, in some form, it will - regulators will ask whether you did everything reasonable to prevent it. Could the Spring Authorization Server be configured and operated in a way that complies with HIPAA, PCI-DSS, GDPR, SOC 2, or whatever else applies to you? Almost certainly yes. Will the out-of-the-box defaults get you there? No. Nobody's defaults get you there - not mine, not anyone else's. That isn't the job of an OAuth server.

To actually pass an audit, you need operational things like:

- backups
- multi-factor authentication
- HSMs (or a KMS) for signing keys
- comprehensive monitoring and alerting
- a patching cadence, and the discipline to keep to it
- a documented incident response program
- (and a lot more besides)

The Spring Authorization Server can participate in all of that, and I'll point at the hooks where it does. The operational discipline around it is on you.

And before you assume a managed SaaS makes this go away: it doesn't. Providers like Okta supply genuinely useful compliance artifacts - SOC 2 reports, ISO 27001 certifications, audit logs, security whitepapers. But regulators still hold *your* organization accountable for how you configure the service, who has access to it, your business processes, your regulatory obligations, and your overall compliance program. The vendor's compliance posture is a useful input. It is not a substitute for

your own.

The Spring Security OAuth Ship of Theseus

The Spring Authorization Server is one of my favorite new projects. It's a full-blown OAuth identity provider (IDP), distributed as Spring Boot autoconfiguration.

But getting to this oh-so-approachable place took more than a decade! It started way back in 2011, when the Spring team volunteered to take on, and evolve, an awesome third-party open-source community project. We wanted to support the project and use it for some of the use cases we had in the popular open-source cloud platform, Cloud Foundry, where it eventually became the UAA [https://github.com/cloudfoundry/uaa] component.

This project had three interesting pieces: *OAuth Resource Server* support, *OAuth Client* support, and *OAuth Authorization Server*. We took it on and even had legends like the great Dr. Dave Syer (cofounder of Spring Boot and Spring Cloud, for a start) work on it. The project worked, but by 2015 it was starting to get long in the tooth. It was built before Spring Boot. It didn't support OIDC and other improvements in the OAuth space. It didn't work in a reactive world. Generally, it needed an overhaul.

So we embarked upon a project to rebuild and reintroduce OAuth support in Spring Security. This time, we said, the OAuth support would live as part of Spring Security itself, not a separate project. The first piece to debut, in the Spring Security 5.0 and Spring Boot 2.0 generation, was the OAuth Client and OIDC support. Then we introduced resource server support. And we said that we didn't feel the need to build an OAuth Authorization Server, as the market had many good choices already.

In a GitHub issue on `spring-security` wherein we declined to build this, the community told us we were wrong. Loudly [https://github.com/spring-projects/spring-security/issues/6320#issuecomment-614218694]. This issue was one of the most vibrant in the Spring ecosystem's history! Clearly, there was demand. And so in 2020, work began in earnest, eventually landing as a fully supported, easily autoconfigured project in Spring Boot that you can use today.

The Spring Authorization Server originally lived as a separate project, with its own lifecycles and release cadences and so on, but once it stabilized, it was moved under Spring Security with the Spring Security 7 release in late 2025.

The Spring Security OAuth project is dead. Long live Spring Security's OAuth support!

One Less `users` Microservice For All

Having a real IDP at the center of your architecture is *liberating*.

The obvious win is consolidation. Every team that would otherwise have built its own `users` table, its own password reset flow, its own session story, and its own permission system can stop. There's one identity surface for the whole organization: one place to onboard a new employee, one place to revoke them when they leave, one place to add SSO into a customer's IDP. You'll have one less bespoke token-management system, too.

The less obvious win is isolation. Password hashing is expensive *on purpose* - bcrypt, scrypt, and

argon2 are designed to burn CPU so attackers can't brute-force them. In a typical monolith, that CPU is competing with every business-logic request you're serving. Move authentication into a dedicated authorization server and the expensive work happens on a node you can scale, monitor, and harden independently of everything else.

And because it's Spring, it composes with the rest of Spring Security, Spring Boot autoconfiguration, and the broader ecosystem - the hooks and customization points are there at every step.

If you knew the agony I've been subjected to learning and wielding OAuth over the years, you'd understand what it means when I say I love this project. It's the first OAuth experience I've had that doesn't feel like work.

And, dear reader, I want you to love OAuth too. The trick is to *stop having to think about OAuth* - to set it up well once, and get back to building the thing you actually meant to build. That's what the rest of this book is about.

So let's take that journey to production together, with the Spring Authorization Server at our backs.

Implementing the Spring Authorization Server

Go to the Spring Initializr (start.spring.io) [https://start.spring.io], specify a group ID and an artifact ID (I chose `bootiful` : `authorization-server`) and add `OAuth2 Authorization Server` as a dependency. I'd add `GraalVM Native Support` for good measure, but you do you. Open the downloaded project in your IDE. I'm using IntelliJ IDEA, but again, you do you. I ran the following command from the root of the newly unzipped archive: `idea build.gradle`.

You've got a new Spring Boot Authorization Server. We need to specify two things: **users**, and **clients**.

Users

Users are pretty straight forward, right? A user is the sum of the username, password, and associated information attached to the system's notion of *identity*. The beating heart of our system. The *tenant* in "multitenant."

There are a lot of ways to get this done. The easiest might be to just have one user, the *default* user, which you can describe using Spring Boot's associated properties, like this:

```
spring.security.user.name=jlong
spring.security.user.password=password
spring.security.user.roles=admin,users.write,users.read,openid
```

This gets us off the ground, but as soon as you want two or more users, you'll need to specify them a different way. The easiest is probably to define a bean of type `InMemoryUserDetailsManager`, like this.

```
package bootiful.authorizationserver;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;

@Configuration
class UserDetailsConfiguration {

    @Bean
    UserDetailsService inMemoryUserDetailsManager() {
        var userBuilder = User.withDefaultPasswordEncoder();
        return new InMemoryUserDetailsManager(

userBuilder.roles("USER").username("josh@joshlong.com").password("pw").build(),
                userBuilder.roles("USER",
```

```
"ADMIN").username("rob@spring.security").password("p@ssw0rd").build()
        );
    }

}
```

Thus configured we've got two users:

- `josh@joshlong.com` with password `pw` and roles `USER`
- `rwinch@spring.security` with password `p@ssw0rd` and roles `USER` and `ADMIN`

This implementation is fine for development as it's all in-memory. In a production system, you'll probably want something more durable. We'll look at those possibilities in a bit. For now, let's talk about OAuth clients!

OAuth Clients

An OAuth client defines how a program or process interacts with an OAuth Authorization Server (like Spring Authorization Server). Clients correspond more or less to the programs that would like to be allowed to act on behalf of users but need their permission - their *access tokens* - to do so.

I have tried to conceive of a clear illustration of clients in a vacuum, but it's not easy. so let's examine a real life example: you stumble upon some website, say Yelp [https://www.yelp.com/], a website that lets you contribute and read reviews about locations - restaurants, businesses, tourist spots, etc. You want to login to see your history. You *could* create a new account there, going through the whole signup flow and entering redundant information, but this information could soon become stale. Worse: you've probably already entered this somewhere else on the internet and done a better job of maintaining it there, too. Maybe you change house or email address, or whatever, and you've forgotten to go back to the site and change your information. Yelp know this, so they offer another path forward: `Continue with Google` and `Continue with Apple`.

[yelp signup] | *./images/yelp-signup.png*

Click the button and another window on Google or Apple's sites pop up.

[google signin] | *images/google-signin.png*

You know what to do here: you're in familiar territory. It's google.com! You know Google. And if you've ever set up an account and stayed logged in while navigating the web, Google *almost certainly* knows you too! You've got an account here, you maintain that account, and you like that account. You use it for your daily email, after all. You've even got that reassuring little padlock icon in the browser's location bar giving you the warm-n-fuzzies about this site's authenticity: it is who it claims to be. So you enter your information, login, and you do whatever mutli-factor auth things Google wants you to do. You might get a prompt on another device, you might use a passkey, etc. Let's suppose we get the prompt on another device.

[google mfa] | *images/google-mfa.png*

This shows up as a prompt on a completely different device, an iPad.

[google is it you] | *images/google-is-it-you.png*

You've approved of the login, so that Google knows it really is you logging in, and now it's got to make sure you realize you're handing over some of the data associated with your identity to this new website, Yelp.com, so it throws up a consent form.

[google consent screen] | *images/google-consent-screen.png*

You click `Confirm` and then are finally logged in, with your Google identity, on Yelp.com

[yelp logged in with google part 2] | *images/yelp-logged-in-with-google-part-2.png*

At the end of this exchange, Google.com transmitted a *token* to the application running at Yelp.com. Armed with this, the application running at Yelp.com can now transmit requests to the Google.com APIs, asking it questions about you, like your email. It might also be able to read your Google calendar events, location data, etc. What precisely the application at Yelp.com has access to is a function of the *scopes* requested by the client. The application at Yelp.com stores the token and uses it to interact with Google on your behalf. Occasionally, Google.com will expire the token. Tokens, like milk, go stale! No worries: the application at Yelp.com was also given a *refresh* token it can use to refresh the access token and get a new one.

You're glad you signed up at Yelp.com, but look at the time! It's noon, the sun's out, and the kid wants to go play mini golf at the place you just found on Yelp.com. Gotta go!

Time passes, and you return to Yelp.com a week later. By this point, Yelp.com's expired your HTTP session, and you're logged out. No problem. Click the `Continue with Google` button again, and this time you'll just be dumped into Yelp.com, fully authenticated. Both Google and Yelp remember who you are, and so there's no ceremony this time. You got fast-path'd into an authenticated HTTP session on Yelp.com. Thus: OAuth is invaluable both for establishing a new account and for subsequently logging into it. You may have changed your home address on Google.com in the meantime, and now Yelp.com can see the new address information and offer you updated recommendations, too. So Yelp.com is kept up to date, and all you had to do was keep Google.com up to date.

From the perspective of Google, Yelp.com is an OAuth client. All the particulars of how you went through that authentication flow - whether you needed to be redirected to Google.com, whether you should be shown a consent form, and what data Yelp.com was allowed to read from the Google.com API once it had a token stemming from this authentication flow, was governed by how the developers at Yelp.com registered their client with Google.

Clients must stipulate a client ID, and a client secret. The client ID and client secret are transmitted in the request initiating the authentication flow, signaling to Google that Yelp.com is making this request. Clients also stipulate what *scopes* they want. A scope is OAuth's version of rights, permissions, authorities, or claims. They're (basically) arbitrary strings that mean something to Google.com's API about what you may and may not access.

There is one scope, `openid`, which is part of the OIDC specification and allows an OAuth client to authenticate users in a standard way. Specifically, it has two effects: * an ID token that contains a JWT that client applications can consume to understand who the user is. * it gives the client access to the `UserInfo` endpoint. This endpoint provides additional information about a given user.

This is a sort of special case; Yelp.com may not want to read Google Calendar data, or read your email, or draft a spreadsheet for you in Google Drive. The scopes to support those Google-y things would necessarily be unique to Google's APIs. But signing a user into a site is a common enough thing and one that can be implemented usefully across all sorts of OAuth providers, so there's a specification called OpenID Connect (OIDC), that builds on top of OAuth 2.0, prescribing standard scopes and , importantly, standard APIs by which a client may look up information associated with a user. Yelp.com might only just need enough information from Google.com to fill out a signup form for us: name, email, etc. In this way the Yelp.com client could even reuse the same code across other OIDC compliant providers, changing only the client ID and client secret and the issuer URI (the API's root URL). Neat-o!

So, if you built a backoffice process, you'd register a client for that backoffice process. If you built a new web application that you intend to support automatic sign-in with OAuth, you'd register a new client for that web application.

The simplest way to register clients in the Spring Authorization Server is to use properties in the `application.properties` or `application.yaml` file, like in this `application.yaml` example:

```
spring:
  security:
    oauth2:
      authorizationserver:
        client:
          crm-client:
            require-authorization-consent: true
            registration:
              client-id: crm
①
              client-secret:
"{bcrypt}$2a$10$m7dGi0viwVH63EjwZc6UdeUQxPuiVEEdFbZFI9nMxHAASTOIDlaVO"
②
              authorization-grant-types: client_credentials, authorization_code,
refresh_token
③
              redirect-uris: http://127.0.0.1:8082/login/oauth2/code/spring
④
              scopes: user.read,user.write,openid
⑤
              client-authentication-methods: client_secret_basic
```

① you can use the `spring` [https://docs.spring.io/spring-boot/docs/current/reference/html/cli.html] CLI to encode a password for the client secret: `spring encodepassword BLAH`, where `BLAH` is the string you want to encode. In our case, the client ID is `crm` and the client secret is `crm`. (Again, I *know* it's a terrible password. Don't `@` me!). NB: For complex strings like this, YAML parsing rules can be problematic, so I tend to wrap these things in quotation marks.

② `authorization-grant-types` refers to the use case - web application, mobile, headless backoffice application, etc. - for the authentication flow. OAuth 2.0 is nothing if not flexible [https://oauth.net/2/grant-types/].

③ we're building a web application so the expectation is that, once you've authenticated yourself with the Spring Authorization Server, it'll redirect you back to the web application with the token in tow. But where? You specify that here. We haven't looked at the application yet, so this is a bit of foreshadowing, but the redirect URI specified here is designed to line up with Spring Security's OAuth client support, which we'll use on the web application.

④ Here we specify which scopes we'd like to be given. We've seen `openid` before, and the other two are arbitrary, and just for demonstration.

At this point, we have a valid Spring Authorization Server, and you're ready to start using it! Run the application in the usual way: `./gradlew bootRun` or `./mvnw spring-boot:run` or just run the main method from your IDE. Congratulations on your first deployment of the Spring Authorization Server. We *could* stop here, satisfied that we have got *something* to allow us to handle development chores and start building services. Indeed, if you want to, you can skip ahead and things should work fine.

Eventually, however, you're going to realize you can't leave things as they are - you'll need durable state. As-is, everything is kept in-memory. It's obviously a non-starter to have to redeploy the Spring Authorization Server every time you add a new client, user, or otherwise. People will want self-service forms by which they can register new users, clients, etc. All existing OAuth tokens would become invalid once you restart the Spring Authorization Server, too! All state related to any successful OAuth authorizations would be forgotten on every restart. The situation's not good, and in the next section, we're going to look at introducing persistence, with JDBC, to get around it.

If you want to carry on using property files, then perhaps consider the Spring Cloud Config Server. It's another piece of Spring Boot-powered middleware that, once stood up, mediates access to configuration files via an HTTP API. The configuration files live in a version control system, like Git, which the Spring Cloud Config Server monitors. When the files change, the Spring Cloud Config Server serves up the new configuration data. Even better, the Spring Cloud Config Server can, via the Spring Cloud Bus abstraction, publish notifications to your microservices (like the Spring Authorization Server) on an event bus like RabbitMQ or Apache Kafka so that you can automatically reload the new configuration. This works particularly well in tandem with the Spring Cloud's `@RefreshScope`. In such a configuration, the configuration for everything still lives in a `.properties` or `.yaml` file, as it does now, but the files are centralized and can be changed without reloading the Spring Authorization Server. Storing files in a version control system gives us niceties like versioning, auditing, rollbacks, etc., for very sensitive configuration data. And, going a step further, you can even use the Spring Authorization Server to store data encrypted at rest. For more on these possibilities, check out this video I did some years ago [https://www.youtube.com/watch?v=aC_siBP8rx8&list=PLgGXSWYM2FpPw8rV0tZoMiJYSCiLhPnOc&index=31]. And *all* of these possibilities are enabled entirely because the Spring Authorization Server is delivered as just another Spring Boot autoconfiguration! :code: ..

Persistent State in the Spring Authorization Server with PostgreSQL and JDBC

Spring makes it easy to substitute implementations with polymorphism. *Dependency injection* is one of the key reasons we use Spring. Spring Boot-based configuration makes this doubly powerful. While the Spring Authorization Server does amazing things out of the box, its real power lay in all the knobs and leavers available to you because it's just another Spring Boot autoconfiguration and starter. We have already acknowledged that keys parts of the Spring Authorization Server defer to in-memory implementations. In this section, we'll swap those, and more, out for implementations using JDBC. We'll use JDBC, but these are just interfaces. If you don't want to use JDBC, then feel free to implement the interfaces for yourself, deferring to whatever underlying storage mechanism you want. Spring Data is your friend…

Before we can get started, we'll need a PostgreSQL database. Some place in which to store our state. In the root of the project, run `docker compose up`.

Now, we're going to need to retool our project to accommodate JDBC, so add the following dependencies to the Gradle build:

```
    runtimeOnly 'org.postgresql:postgresql'
    implementation 'org.springframework.boot:spring-boot-starter-jdbc'
```

Modify `application.properties` or `application.yaml` to point to the newly configured PostgreSQL database with username, schema, and password all set to `postgres`. (Sigh. I can feel Spring Security lead Rob Winch staring at me disapprovingly because of my terribad passwords: I'm trying to demonstrate something here!) Here's the relevant configuration for `application.properties`.

```
spring.datasource.url=jdbc:postgresql://localhost/postgres
spring.datasource.username=postgres
spring.datasource.password=postgres
①
spring.sql.init.mode=always
②
spring.sql.init.schema-locations=classpath:sql/schema/*sql
③
# spring.sql.init.data-locations=classpath:sql/data/*sql
```

① This tells Spring Boot to initialize the SQL database with the schema in `src/main/resources/schema.sql` and `src/main/resources/data.sql`. Be sure to disable this property in production!

② This tells Spring Boot to not use `schema.sql` specifically, but to instead use *all* `.sql` files in `src/main/resources/sql/schema/`. This way, we can keep the various DDL statements separate.

③ If enabled, this line tells Spring Boot to not use `data.sql` specifically, but to instead use *all* `.sql` files in `src/main/resources/sql/data/`. This way, we can keep the inserts separate. Make sure to create this directory if you enable this property.

A Brief Note on Storing Credentials

Every time you see a password, or any kind of credential like a key, in the source code of these programs, remember, this is a *demonstration*. There are a million ways to externalize credentials from the source code, and you should use one of them!

It's essential to keep credentials both secure and accessible for the applications that need them, without risking exposure. Here are some methods and tools to store and make private keys accessible to a Spring Boot application:

- **Environment Variables**: One of the most common ways is to use environment variables to store sensitive data. These variables can be set on the system where your application runs. This method separates configuration from the application, but it's not the most secure method on its own, especially if multiple applications share the environment.
- **Java's KeyStore (JKS/JCEKS)**: Java provides its built-in mechanism for securely storing cryptographic keys and certificates - Java KeyStore.
- **AWS Secrets Manager or AWS Parameter Store**: If you are deploying on AWS, you can use these services to store and retrieve your application secrets.
- **HashiCorp Vault**: An open-source tool for secret management. It allows you to centrally store, access, and deploy secrets across applications and infrastructure.
- **Azure Key Vault**: If you are on Azure, you can use Azure Key Vault to store, manage, and access secrets.
- **Google Cloud Secret Manager**: For applications deployed on GCP.
- **Encrypted Configuration Files**: Use tools like Jasypt with Spring Boot. Jasypt provides Spring Boot integration and allows you to encrypt property values in your configuration files.
- **Kubernetes Secrets**: If you are deploying your application in Kubernetes, it offers its own secrets management mechanism. Though it is better than plain config maps (`ConfigMap`), Kubernetes Secrets are Base64 encoded by default and not encrypted. For enhanced security, consider integrating with external secrets managers like HashiCorp Vault.
- **Dedicated Hardware Security Modules (HSMs)**: These are physical devices that safeguard and manage digital keys, perform encryption and decryption. Cloud providers like AWS offer their own cloud HSM services.
- **Use Configuration Servers**: For example, Spring Cloud Config Server. Though not a secrets manager *per se*, when combined with encryption, it can serve configurations securely to your applications.

Always encrypt sensitive data in transit and at rest. Rotate secrets regularly. Use IAM roles, policies, and least privilege principles to restrict who can access secrets. Monitor and audit access to secrets. Avoid hardcoding secrets or placing them in a version control system (VCS), even if encrypted. Backup your secrets, but ensure backups are also secured. No matter which approach you choose, it's essential to keep the principle of least privilege in mind. Only the necessary entities should have access to your secrets, and they should only be decrypted at the last possible moment (e.g., by the application when needed).

Also, we're going to need to encode passwords so that they're not lying around at rest in plain text.

Define a bean of type `PasswordEncoder` for use in your program.

```
package com.example.authorization_server;

import com.nimbusds.jose.jwk.JWKSet;
import com.nimbusds.jose.jwk.RSAKey;
import com.nimbusds.jose.jwk.source.ImmutableJWKSet;
import com.nimbusds.jose.jwk.source.JWKSource;
import com.nimbusds.jose.proc.SecurityContext;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;

import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;

@Configuration
class SecurityConfiguration {

①
    @Bean
    PasswordEncoder passwordEncoder() {
        return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    }

}
```

① this is a sort of compose `PasswordEncoder`, checking the prefix of the password string for information as to which encoder to use. We've already seen it action. The `spring encodepassword` CLI command produces a string that starts with `{bcrypt}⋯`. The default for Spring Security, today, as of this writing, is to use BCrypt. But that may change, and when the default changes, existing passwords will continue to work because the `PasswordEncoder` will know to look for the prefix and use the older BCrypt encoder when dealing with those older passwords.

We'll use the `PasswordEncoder` more later.

Persisting Users

There are other implementations of the `UserDetailsService` interface that you can use to persist users durably. Thinking from a more operational perspective, it's possible you'd want to dynamically register users dynamically, rather than having to restart the Spring Authorization Server instance after you've updated the source code. Spring Security has an extension of the `UserDetailsService` interface called `UserDetailsManager` which gives you explicit control over the lifecycle of `UserDetails`: adding, updating, deleting, etc. And, as you might imagine, there's a persistent implementation of this interface called `JdbcUserDetailsManager` that uses JDBC.

You'll need to install some SQL schema first. Spring Security ships with some usable schema on the

classpath, but unfortunately it doesn't work with PostgreSQL, and it's going to fail if there are already table definitions. So we'll modify it accordingly, as shown in `src/main/resources/sql/schema/users.sql`.

```
create table if not exists users
(
    username varchar(200) not null primary key,
    password varchar(500) not null,
    enabled  boolean      not null
);

create table if not exists authorities
(
    username  varchar(200) not null,
    authority varchar(50)  not null,
    constraint fk_authorities_users foreign key (username) references users (username
),
    constraint username_authority UNIQUE (username, authority)
);
```

We could also define the users with SQL in a file under the `sql/data/` folder, but I want you to see what it looks like to use the Java API to programmatically register a user, so here is both the `UserDetailsService` registration and a bean that uses the `UserDetailsService` to write some data to the database.

```
package com.example.authorization_server;

import org.springframework.boot.ApplicationRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.JdbcUserDetailsManager;
import org.springframework.security.provisioning.UserDetailsManager;

import javax.sql.DataSource;
import java.util.Map;

@Configuration
class UsersConfiguration {

    @Bean
    JdbcUserDetailsManager jdbcUserDetailsManager(DataSource dataSource) {
        return new JdbcUserDetailsManager(dataSource);
    }

    @Bean
    ApplicationRunner usersRunner(PasswordEncoder passwordEncoder, UserDetailsManager
userDetailsManager) {
```

```
        return args -> {
①
            var builder = User.builder().roles("USER").passwordEncoder(
passwordEncoder::encode);
②
            var users = Map.of("josh@joshlong.com", "pw", "user@anotherone.site",
"p@ssw0rd");
            users.forEach((username, password) -> {
                if (!userDetailsManager.userExists(username)) {
                    var user = builder.username(username).password(password).build();
                    userDetailsManager.createUser(user);
                }
            });
        };
    }

}
```

① this is a convenient pattern: define the prototype for all new users once and then reuse the builder

② Poor Rob Winch's eyes! why are there passwords just strewn about our Java source code? Remember what we talked about earlier: don't do this in production code!

Persisting Clients

The `RegisteredClientRepository` interface is trivial and lends itself to implementation with a persistent store. It's easy enough to do that here, too, with implementations of the `RegisteredClientRepository`. There's an implementation called `JdbcRegisteredClientRepository` that uses JDBC to manage registered clients. It would be a fairly trivial project to implement alternatives using other persistence mechanisms like MongoDB or Hashicorp Vault.

The obvious advantage of a `RegisteredClientRepository` backed by a persistent store is that you could build a self-service registration form (or workflow) - just like Google and Apple do - for your organization developers to register clients on demand without having to restart anything or manipulate source code.

There's some schema on the classpath (`classpath:org/springframework/security/oauth2/server/authorization/client/oauth2-registered-client-schema.sql`) for the implementation that we need to take care to install first. We could tell Spring Boot to run this directly, but the trouble is that it'll fail on the second run when it executes the same DDL statements and experiences a conflict trying to create something that's already there. Create a new file `src/main/resources/sql/schema/oauth2-registered-client.sql` and use this DDL instead.

```
CREATE TABLE if not exists oauth2_registered_client
(
    id                            varchar(100)                            NOT NULL,
    client_id                     varchar(100)                            NOT NULL,
```

```
    client_id_issued_at           timestamp     DEFAULT CURRENT_TIMESTAMP NOT NULL,
    client_secret                 varchar(200)  DEFAULT NULL,
    client_secret_expires_at      timestamp     DEFAULT NULL,
    client_name                   varchar(200)                            NOT NULL,
    client_authentication_methods varchar(1000)                           NOT NULL,
    authorization_grant_types     varchar(1000)                           NOT NULL,
    redirect_uris                 varchar(1000) DEFAULT NULL,
    post_logout_redirect_uris     varchar(1000) DEFAULT NULL,
    scopes                        varchar(1000)                           NOT NULL,
    client_settings               varchar(2000)                           NOT NULL,
    token_settings                varchar(2000)                           NOT NULL,
    PRIMARY KEY (id)
);
```

And here's the definition of the `RegisteredClientRepository` and a runner that uses it to install a client, more or less identical to the client we registered earlier in `application.properties`.

```
package com.example.authorization_server;

import org.springframework.boot.ApplicationRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.oauth2.core.AuthorizationGrantType;
import org.springframework.security.oauth2.core.ClientAuthenticationMethod;
import org.springframework.security.oauth2.core.oidc.OidcScopes;
import org.springframework.security.oauth2.server.authorization.client
.JdbcRegisteredClientRepository;
import org.springframework.security.oauth2.server.authorization.client
.RegisteredClient;
import org.springframework.security.oauth2.server.authorization.client
.RegisteredClientRepository;

import java.util.Set;
import java.util.UUID;

@Configuration
class ClientsConfiguration {

①
    @Bean
    RegisteredClientRepository registeredClientRepository(JdbcTemplate template) {
        return new JdbcRegisteredClientRepository(template);
    }

②
    @Bean
    ApplicationRunner clientsRunner(PasswordEncoder pwe, RegisteredClientRepository
repository) {
```

```
        return _ -> {
            var clientId = "crm";
            if (repository.findByClientId(clientId) == null) {
                var crmClientSecret = pwe.encode("crm");
                repository.save(RegisteredClient.withId(UUID.randomUUID().toString())
                    .clientId(clientId)
                    // you should get this from an environment variable!
                    .clientSecret(crmClientSecret) ①
                    .clientAuthenticationMethod(ClientAuthenticationMethod
.CLIENT_SECRET_BASIC)
                    .authorizationGrantTypes(grantTypes -> grantTypes.addAll(
                            Set.of(AuthorizationGrantType.CLIENT_CREDENTIALS,
AuthorizationGrantType.AUTHORIZATION_CODE,
                                    AuthorizationGrantType.REFRESH_TOKEN,
AuthorizationGrantType.DEVICE_CODE)))
                    .redirectUri("http://127.0.0.1:8080/login/oauth2/code/crm")
                    .scopes(scopes -> scopes.addAll(
                            Set.of("user.read", "user.write", OidcScopes.PROFILE,
OidcScopes.EMAIL, OidcScopes.OPENID)))
                    .build());
            }
        };
    }

}
```

① register the `RegisteredClientRepository`

② this installs a registered client that's more or less equivalent to what we saw earlier in the `application.yml` properties file.

Persisting Authorizations

Remember that bit where you got redirected to the Spring Authorization Server and had to click a checkbox to confirm that the user had `user.read` scope? That fact - the consent of the scope - is stored in memory by default, but it could be stored in a database too. (Big surprise, I know!)

There are two interfaces of note here: `OAuth2AuthorizationService` and `OAuth2AuthorizationConsentService`.

`OAuth2AuthorizationService` handles representations of an authorization - the JWT token, the client, etc. It's the same story as before: there's schema on the classpath (`org/springframework/security/oauth2/server/authorization/oauth2-authorization-schema.sql`) but it doesn't work with PostgresSQL and, importantly, even if it did it would fail on the second run through because Spring Boot would try to define the table twice, in effect. So we'll modify it. Create a file `src/main/resources/sql/schema/oauth2-authorization-schema.sql`. The changes we've made replace all `blob` types with `text`.

```
create table if not exists oauth2_authorization
(
```

```
    id                            varchar(100) NOT NULL,
    registered_client_id          varchar(100) NOT NULL,
    principal_name                varchar(200) NOT NULL,
    authorization_grant_type      varchar(100) NOT NULL,
    authorized_scopes             varchar(1000) DEFAULT NULL,
    attributes                    text          DEFAULT NULL,
    state                         varchar(500)  DEFAULT NULL,
    authorization_code_value      text          DEFAULT NULL,
    authorization_code_issued_at  timestamp     DEFAULT NULL,
    authorization_code_expires_at timestamp     DEFAULT NULL,
    authorization_code_metadata   text          DEFAULT NULL,
    access_token_value            text          DEFAULT NULL,
    access_token_issued_at        timestamp     DEFAULT NULL,
    access_token_expires_at       timestamp     DEFAULT NULL,
    access_token_metadata         text          DEFAULT NULL,
    access_token_type             varchar(100)  DEFAULT NULL,
    access_token_scopes           varchar(1000) DEFAULT NULL,
    oidc_id_token_value           text          DEFAULT NULL,
    oidc_id_token_issued_at       timestamp     DEFAULT NULL,
    oidc_id_token_expires_at      timestamp     DEFAULT NULL,
    oidc_id_token_metadata        text          DEFAULT NULL,
    refresh_token_value           text          DEFAULT NULL,
    refresh_token_issued_at       timestamp     DEFAULT NULL,
    refresh_token_expires_at      timestamp     DEFAULT NULL,
    refresh_token_metadata        text          DEFAULT NULL,
    user_code_value               text          DEFAULT NULL,
    user_code_issued_at           timestamp     DEFAULT NULL,
    user_code_expires_at          timestamp     DEFAULT NULL,
    user_code_metadata            text          DEFAULT NULL,
    device_code_value             text          DEFAULT NULL,
    device_code_issued_at         timestamp     DEFAULT NULL,
    device_code_expires_at        timestamp     DEFAULT NULL,
    device_code_metadata          text          DEFAULT NULL,
    PRIMARY KEY (id)
);
```

`OAuth2AuthorizationConsentService` handles representations of an OAuth 2.0 "consent" to an Authorization request, which holds state related to the set of authorities granted to a client by the resource owner. It's the same story as before: there's schema on the classpath (`org/springframework/security/oauth2/server/authorization/oauth2-authorization-consent-schema.sql`) but it doesn't work with PostgresSQL and, importantly, even if it did it would fail on the second run through because Spring Boot would try to define the table twice, in effect. So we'll modify it. Create a file `src/main/resources/sql/schema/oauth2-authorization-consent-schema.sql`. The changes we've made replace all `blob` types with `text`.

```
create table if not exists oauth2_authorization_consent
(
    registered_client_id varchar(100)  NOT NULL,
    principal_name       varchar(200)  NOT NULL,
```

```
    authorities         varchar(1000) NOT NULL,
    PRIMARY KEY (registered_client_id, principal_name)
);
```

Persisting the HTTP Sessions Themselves

The final piece of the persistence pie is to persist the actual HTTP sessions themselves. Spring Authorization Server assumes a Servlet container is present somewhere. By default, Spring Boot uses Apache Tomcat, though that's very configurable. And Apache Tomcat, in turn, provides session management features consistent with the HTTP Servlet specification. It's even got pluggable HTTP session management, meaning you can plugin other implementations of Apache Tomcat's proprietary abstraction. But this isn't easy, or portable. Indeed, any web server is going to have some rudimentary HTTP session management, but session clustering and replication, consistency checks, etc., are not their _raison d'être: serving HTTP requests is.

Spring Session can help. It wraps the containers' default `HttpSession`. All interactions you have pass through the wrapper, which in turn delegates to any of a number of implementations backed by technology that is far faster and more reliable in the ways of making data consistently available.You can use implementations for, among other things, Hazelcast, Redis, and of course any 'ol SQL `DataSource`. We're choosing to continue to leverage our investment in PostgreSQL, so we'll use the Spring Session JDBC module.

Add the following dependency to the build:

```
    implementation 'org.springframework.session:spring-session-jdbc'
```

There is a property, `spring.session.jdbc.initialize-schema=always`, that once specified will cause Spring Session to install the JDBC schema for you in the database. It worked for me (surprise!), in PostgreSQL. But, I just really want the schema to all be in one place where I can version control it, audit it, etc, so here's the schema. Create a file `src/main/resources/sql/schema/spring-session-jdbc.sql`.

```
-- sessions
create table if not exists spring_session
(
    primary_id            character(36) primary key not null,
    session_id            character(36)             not null,
    creation_time         bigint                    not null,
    last_access_time      bigint                    not null,
    max_inactive_interval integer                   not null,
    expiry_time           bigint                    not null,
    principal_name        character varying(100)
);
create unique index if not exists spring_session_ix1 on spring_session using btree
(session_id);
create index if not exists spring_session_ix2 on spring_session using btree
(expiry_time);
```

```
create index if not exists spring_session_ix3 on spring_session using btree
(principal_name);

-- session attributes
create table if not exists spring_session_attributes
(
    session_primary_id character(36)          not null,
    attribute_name     character varying(200) not null,
    attribute_bytes    bytea                  not null,
    primary key (session_primary_id, attribute_name),
    foreign key (session_primary_id) references spring_session (primary_id)
        match simple on update no action on delete cascade
);
```

We've looked at how to persist almost all aspect of the domain of the Spring Authorization Server with JDBC. All aspects, except *keys*. Keys require a long discussion and so in the next chapter we'll look into that.

Keys

Every time the Spring Authorization Server starts up, the Spring Boot autoconfiguration kicks in generating new keys for our application. It uses the keys to sign the JWT tokens that it vends for our other applications. Other applications - clients, microservices, etc. - retain these tokens, sometimes for hours or days or even weeks! What happens when you restart the Spring Authorization Server, it autocreates new keys, and then a client with a JWT signed with the old keys tries to connect? It fails! Having the Spring Authorization Server generate random tokens on every restart can be quite a boon to getting started, but we should furnish our own, stable key if we want the JWTs to survive restarts. And load balancing, for that matter! After all, each instance of the JVM would, by default, have different keys. We're going to add two new files, a private and a public key, and use that to create a `JWKSource<SecurityContext>`.

Generate a key pair. You'll need `openssl` or `ssh-keygen` installed.

```
①
openssl genpkey -algorithm RSA -out private_key.pem
openssl pkcs8 -topk8 -inform PEM -outform PEM -in private_key.pem -out
private_key_pkcs8.pem -nocrypt
openssl rsa -pubout -in private_key_pkcs8.pem -out public_key.pem
mv private_key.pem app.key
mv public_key.pem app.pub
②
openssl req -new -x509 -key private_key_pkcs8.pem -out cert.pem -days 365
```

① make sure you execute all the following commands in a new, empty directory. It's important that some of the generated files aren't lost in the wildnerness of whatever busy folder you happen to have created them.

② this command exports a self-signed certificate (valid for 365 days) that you could (optionally) use if you wanted to store the file in the Java KeyStore as a PKCS13 file.

You'll get two artifacts, `app.pub` and `app.key`. Copy them both, the private and public key, to the `src/main/resources` folder: the private key should be called `app.key`, and the public key should be called `app.pub`.

I created three (arbitrary) properties in `application.properties` to describe these keys and assign them an ID.

```
jwt.key.id=bootiful-jwt-key-1
jwt.key.private=classpath:app.key
jwt.key.public=classpath:app.pub
```

We'll use the properties when we define the `JwkSource<ServerContext>` bean.

```
package bootiful.authorizationserver.keys;

import com.nimbusds.jose.jwk.source.JWKSource;
import com.nimbusds.jose.proc.SecurityContext;
import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;
import com.nimbusds.jose.jwk.RSAKey;
import com.nimbusds.jose.jwk.JWKSet;
import com.nimbusds.jose.jwk.source.ImmutableJWKSet;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;

@Configuration
class SimpleKeyConfiguration {

    @Bean
    JWKSource<SecurityContext> jwkSource(
            @Value("${jwt.key.id}") String id,
            @Value("${jwt.key.private}") RSAPrivateKey privateKey,
            @Value("${jwt.key.public}") RSAPublicKey publicKey) {
        var rsa = new RSAKey.Builder(publicKey)
                .privateKey(privateKey)
                .keyID(id)
                .build();
        var jwk = new JWKSet(rsa);
        return new ImmutableJWKSet<>(jwk);
    }

}
```

In this example, I've kept the private key (`app.key`) in the source code of this repository, but as you can imagine, this is a **very bad** idea in production. Don't forget my admonition earlier: take care to handle your credentials securely. There are some solid options, including Hashicorp Vault, your favorite hyperscaler cloud's secrets management tools, Lastpass, etc. If you need it available for this

program, it should be passed into the application in such a way that it's very difficult for others to lay hands on it. Perhaps as environment variable in your application's process space? Or as an encrypted file that can only be decrypted with a rotating key? Rotating keys… that brings up one more thing we should cover!

Rotating Keys

In the last example we plugged in a fixed key we generated by hand using `openssl`. It works, for now, but what happens when we need to rotate the keys? Rotating cryptographic keys is a well-established security practice, and it's often recommended to enhance the security posture of systems that use encryption or authentication mechanisms.

Here are a few reasons:

Regularly rotating keys limits exposure and ensures that whatever key an attacker possesses becomes obsolete after a certain time.

Key rotation also mitigates the risk of weak keys or, as sometimes happens, cryptographic erosion. Cryptographic erosion is when you have keys that are susceptible to gains in computing power that make older keys easier to compromise.

Some industry standards and regulations require periodic key rotation. For instance, the Payment Card Industry Data Security Standard (PCI DSS) has requirements around key management practices, which includes periodic key rotation.

So, we agree it's important, but how do we do it? Conceptually, it's easy: we're going to swap out the implementation of the `JWKSource` for an implementation that defers to a repository. In practice, there are a lot more moving parts involved because we have to actually persist the keys somewhere! Remember what we said about the keys in the last section? How should we encrypt the keys, rather than having them laying around on disk? The same applies here.

Also, if we're going to rotate keys, we neeed a way to generate them, and so we'll need to write some code there, too. After all, using `openssl` was easy enough for a one-time thing, but we wouldn't want to shell out for each new key generated. So, we'll need to write some code there, too. And, finally, we'll need to store these keys somewhere. I am using PostgresSQL so we'll use that.

A Brief Look at the state of Cryptography with PostgreSQL

Most databases support column-level encryption of data at rest. That is, nothing is ever written to disk in an unencrypted form. PostgreSQL does not, at least not out of the box. There are plugins, like PGCrypto [https://www.postgresql.org/docs/current/pgcrypto.html], that provide functions that you can use to encrypt text.

You can use PGCrypto pretty easily. It's a trusted module that can be loaded into the database even if you're not a superuser. Log in to your PostgreSQL instance. If you're using the `compose.yml` file, then you can run:

```
PGPASSWORD=postgres psql -U postgres -h localhoslt postgres
```

Once in, you'll need to load the plugin. It's usually bundled with your PostgreSQL distribution.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

You can then confirm that it's worked by using one of the functions provided, like this:

```
SELECT crypt('hello, world', gen_salt('bf'));
```

This example uses the Blowfish algorithm to encrypt some text. This is an awesome option, but it's unique to PostgreSQL.

Some cloud providers have PostgreSQL offerings that support transparent encryption at rest. For instance, AWS RDS for PostgreSQL supports encryption at rest using AWS Key Management Service.

I love PostgreSQL and, if I were building something for production, I'd have no trouble trusting PostgreSQL. But, in the interest of exploring Spring Security, and keeping our code as generic and easily moved from one database to another, we'll use Spring Security's equally rich encryption support to encrypt the private key (and the public one, though we don't really need to) and store it in the database. We'll need a password and a salt fort he encryption, hwoever, and those keys should ultimately not be stored at rest unencrypted. We talked about this! Store *those* keys somewhere like Hashicorp Vault or your hyperscaler's key management solution.

Generating New Keys Programatically

First thing's first: we'll need a way to generate new keys programmatically. Can't rotate 'em if we can't generate 'em! Mercifully, the Java Security APIs are pretty remarkable here.

```
package com.example.authorization_server.keys;

import org.springframework.stereotype.Component;

import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;
import java.time.Instant;

@Component
class Keys {

    RsaKeyPair generateKeyPair(String keyId, Instant created) {
        var keyPair = generateRsaKey();
        var publicKey = (RSAPublicKey) keyPair.getPublic();
        var privateKey = (RSAPrivateKey) keyPair.getPrivate();
        return new RsaKeyPair(keyId, created, publicKey, privateKey);
    }
```

```
    private KeyPair generateRsaKey() {
        try {
            var keyPairGenerator = KeyPairGenerator.getInstance("RSA");
            keyPairGenerator.initialize(2048);
            return keyPairGenerator.generateKeyPair();
        } //
        catch (Exception ex) {
            throw new IllegalStateException(ex);
        }
    }

}
```

Easy! The only externally visible method is `generateKeyPair`, which takes a key ID (a `kid`), and a timestamp, and returns an instance of `RsaKeyPair`, which is our repository's domain type. It's a record to hold both the generated public and private keys.

```
package com.example.authorization_server.keys;

import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;
import java.time.Instant;

①
record RsaKeyPair(String id, Instant created, RSAPublicKey publicKey, RSAPrivateKey
privateKey) {
}
```

① No notes. I just love Java records and want us to take a moment to appreciate them.

We're going to implement a repository to make working with, and persisting, `RsaKeyPair` instances.

```
package com.example.authorization_server.keys;

import java.util.List;

interface RsaKeyPairRepository {

①
    List<RsaKeyPair> findKeyPairs();

②
    void save(RsaKeyPair rsaKeyPair);

}
```

① Find the key pairs and preserve ordering. We want the latest key to be first.

② This method looks so simple. Don't trust it. It's a lie!

Saving the `RsaKeyPair`, in this example, means writing it to our database, which means we'll need a way to serialize the `java.security.*` key objects. It's not as straightforward as you'd think! But I got it working and put that logic in two implementations of Spring Framework's handy `Converter<T>` interface: `RsaPrivateKeyConverter` for private keys, and `RsaPublicKeyConverter` for public keys.

Here's the `RsaPrivateKeyConverter`:

```
package com.example.authorization_server.keys;

import org.springframework.core.serializer.Deserializer;
import org.springframework.core.serializer.Serializer;
import org.springframework.security.crypto.encrypt.TextEncryptor;
import org.springframework.util.FileCopyUtils;

import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.security.KeyFactory;
import java.security.interfaces.RSAPrivateKey;
import java.security.spec.PKCS8EncodedKeySpec;
import java.util.Base64;

class RsaPrivateKeyConverter implements Serializer<RSAPrivateKey>, Deserializer
<RSAPrivateKey> {

    private final TextEncryptor textEncryptor;

    RsaPrivateKeyConverter(TextEncryptor textEncryptor) {
        this.textEncryptor = textEncryptor;
    }

①
    @Override
    public void serialize(RSAPrivateKey object, OutputStream outputStream) throws
IOException {
        var pkcs8EncodedKeySpec = new PKCS8EncodedKeySpec(object.getEncoded());
        var string = "-----BEGIN PRIVATE KEY-----\n"
                + Base64.getMimeEncoder().encodeToString(pkcs8EncodedKeySpec
.getEncoded())
                + "\n-----END PRIVATE KEY-----";
        outputStream.write(this.textEncryptor.encrypt(string).getBytes());
    }

②
    @Override
    public RSAPrivateKey deserialize(InputStream inputStream) {
        try {
            var pem = this.textEncryptor.decrypt(FileCopyUtils.copyToString(new
InputStreamReader(inputStream)));
```

```
            var privateKeyPEM = pem.replace("-----BEGIN PRIVATE KEY-----", "").
replace("-----END PRIVATE KEY-----", "");
            var encoded = Base64.getMimeDecoder().decode(privateKeyPEM);
            var keyFactory = KeyFactory.getInstance("RSA");
            var keySpec = new PKCS8EncodedKeySpec(encoded);
            return (RSAPrivateKey) keyFactory.generatePrivate(keySpec);
        } //
        catch (Throwable throwable) {
            throw new IllegalArgumentException("there's been an exception", throwable
);
        }
    }

}
```

① serialization involves adding a header and footer to the content of the key and then serializing, but not before Base64 encoding the content of the key, and then passing it through a Spring Security `TextEncryptor`

② deserialization is basically the same, in reverse. Decrypt the text by passing it through a Spring Security `TextEncryptor`, then strip out the header and footer, and then use the Base64 decoder to decode the content, and then finally create a key using the Java Security `KeyFactory`.

The code for `RsaPublicKeyConverter`. We won't rehash it since its structure is basically identical to the `RsaPrivateKeyConverter`.

```
package com.example.authorization_server.keys;

import org.springframework.core.serializer.Deserializer;
import org.springframework.core.serializer.Serializer;
import org.springframework.security.crypto.encrypt.TextEncryptor;
import org.springframework.util.FileCopyUtils;

import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.security.KeyFactory;
import java.security.interfaces.RSAPublicKey;
import java.security.spec.X509EncodedKeySpec;
import java.util.Base64;

class RsaPublicKeyConverter implements Serializer<RSAPublicKey>, Deserializer
<RSAPublicKey> {

    private final TextEncryptor textEncryptor;

    RsaPublicKeyConverter(TextEncryptor textEncryptor) {
        this.textEncryptor = textEncryptor;
    }
```

```
    @Override
    public RSAPublicKey deserialize(InputStream inputStream) throws IOException {
        try {
            var pem = textEncryptor.decrypt(FileCopyUtils.copyToString(new
InputStreamReader(inputStream)));
            var publicKeyPEM = pem.replace("-----BEGIN PUBLIC KEY-----", "").replace("
-----END PUBLIC KEY-----", "");
            var encoded = Base64.getMimeDecoder().decode(publicKeyPEM);
            var keyFactory = KeyFactory.getInstance("RSA");
            var keySpec = new X509EncodedKeySpec(encoded);
            return (RSAPublicKey) keyFactory.generatePublic(keySpec);
        } //
        catch (Throwable throwable) {
            throw new IllegalArgumentException("there's been an exception", throwable
);
        }

    }

    @Override
    public void serialize(RSAPublicKey object, OutputStream outputStream) throws
IOException {
        var x509EncodedKeySpec = new X509EncodedKeySpec(object.getEncoded());
        var pem = "-----BEGIN PUBLIC KEY-----\n"
                + Base64.getMimeEncoder().encodeToString(x509EncodedKeySpec.
getEncoded())
                + "\n-----END PUBLIC KEY-----";
        outputStream.write(this.textEncryptor.encrypt(pem).getBytes());
    }

}
```

We'll look at the configuration of the `TextEncryptor` used in both implementations in a bit.

We'll need to register these converters. I suppose I could've aded a `@Component` annotation to each of them, but instead I chose to register them with Java configuration, like this:

```
package com.example.authorization_server.keys;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.encrypt.Encryptors;
import org.springframework.security.crypto.encrypt.TextEncryptor;

@Configuration
class Converters {

    @Bean
    RsaPublicKeyConverter rsaPublicKeyConverter(TextEncryptor textEncryptor) {
```

```
        return new RsaPublicKeyConverter(textEncryptor);
    }

    @Bean
    RsaPrivateKeyConverter rsaPrivateKeyConverter(TextEncryptor textEncryptor) {
        return new RsaPrivateKeyConverter(textEncryptor);
    }

}
```

Let's look at the repository implementation which will use JDBC.

```
package com.example.authorization_server.keys;

import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.stereotype.Component;
import org.springframework.util.Assert;

import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.util.Date;
import java.util.List;

@Component
class JdbcRsaKeyPairRepository implements RsaKeyPairRepository {

    private final JdbcTemplate jdbc;

    private final RsaPublicKeyConverter rsaPublicKeyConverter;

    private final RsaPrivateKeyConverter rsaPrivateKeyConverter;

    private final RowMapper<RsaKeyPair> keyPairRowMapper;

    JdbcRsaKeyPairRepository(RowMapper<RsaKeyPair> keyPairRowMapper,
RsaPublicKeyConverter publicConverter,
            RsaPrivateKeyConverter privateConverter, JdbcTemplate jdbc) {
        this.jdbc = jdbc;
        this.keyPairRowMapper = keyPairRowMapper;
        this.rsaPublicKeyConverter = publicConverter;
        this.rsaPrivateKeyConverter = privateConverter;
    }

①
    @Override
    public List<RsaKeyPair> findKeyPairs() {
        return this.jdbc.query("select * from rsa_key_pairs order by created desc",
this.keyPairRowMapper);
    }
```

```
②
    @Override
    public void save(RsaKeyPair keyPair) {
        var sql = """
                insert into rsa_key_pairs (id, private_key, public_key, created)
                values (?, ?, ?, ?)
                on conflict on constraint rsa_key_pairs_id_created_key
                do nothing
                """;
        try (var privateBaos = new ByteArrayOutputStream(); var publicBaos = new
ByteArrayOutputStream()) {
            this.rsaPrivateKeyConverter.serialize(keyPair.privateKey(), privateBaos);
            this.rsaPublicKeyConverter.serialize(keyPair.publicKey(), publicBaos);
            var updated = this.jdbc.update(sql, keyPair.id(), privateBaos.toString(),
publicBaos.toString(),
                    new Date(keyPair.created().toEpochMilli()));
            Assert.state(updated == 0 || updated == 1, "no more than one record should
have been updated");
        } //
        catch (IOException e) {
            throw new IllegalArgumentException("there's been an exception", e);
        }
    }

}
```

① in order to find the records we'll issue a query and pass in the `RowMapper<RsaKeyPair>`, which is an instance of `RsaKeyPairRowMapper`, which we'll explore shortly.

② writing the record is pretty easy, too. The converters do most of the work. then it's a simple matter of lining up the converted values as arguments for the SQL update.

We saw that the repository implementation uses a Spring JDBC `RowMapper<T>` instance, which looks like this:

```
package com.example.authorization_server.keys;

import org.springframework.core.serializer.Deserializer;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.stereotype.Component;

import java.io.IOException;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Date;

@Component
class RsaKeyPairRowMapper implements RowMapper<RsaKeyPair> {

    private final RsaPrivateKeyConverter rsaPrivateKeyConverter;
```

```
    private final RsaPublicKeyConverter rsaPublicKeyConverter;

    RsaKeyPairRowMapper(RsaPrivateKeyConverter rsaPrivateKeyConverter,
RsaPublicKeyConverter rsaPublicKeyConverter) {
        this.rsaPrivateKeyConverter = rsaPrivateKeyConverter;
        this.rsaPublicKeyConverter = rsaPublicKeyConverter;
    }

    @Override
    public RsaKeyPair mapRow(ResultSet rs, int rowNum) throws SQLException {
        try {

①
            var privateKey = loadKey(rs, "private_key", this.rsaPrivateKeyConverter);
            var publicKey = loadKey(rs, "public_key", this.rsaPublicKeyConverter);

②
            var created = new Date(rs.getDate("created").getTime()).toInstant();
            var id = rs.getString("id");

③
            return new RsaKeyPair(id, created, publicKey, privateKey);
        }
        catch (IOException e) {
            throw new RuntimeException(e);
        }
    }

    private static <T> T loadKey(ResultSet rs, String fn, Deserializer<T> f) throws
SQLException, IOException {
        var privateKeyBytes = rs.getString(fn).getBytes();
        return f.deserializeFromByteArray(privateKeyBytes);
    }

}
```

① again, one of the nice things about this implementation is that most of the heavy lifting is in the converters.

② this stuff is thankfully much easier to deal with

③ I've extracted out the logic of extracting a field from theh `ResultSet`, passing it to a converter, and then returning it to a separate method.

now we've done everything we need to do to support reading and writing key pairs. Let's put it all to good use in an implementation of `JWKSource`, which - when you look at it - is almost anticlimactic. There are two concerns being adressed in this implementation, telling Nimbus, the library whose support for JWT underpins the Spring Authorization Server, how to load a new key given a query (called a `JWKSelector`) and telling Spring Authorization Server that we want to have a chance to post-process, to *customize*, the OAuth 2.0 token attributes. We'll plug that OAuth2TokenCustomizer

functionality in later.

```
package com.example.authorization_server.keys;

import com.nimbusds.jose.KeySourceException;
import com.nimbusds.jose.jwk.JWK;
import com.nimbusds.jose.jwk.JWKSelector;
import com.nimbusds.jose.jwk.RSAKey;
import com.nimbusds.jose.jwk.source.JWKSource;
import com.nimbusds.jose.proc.SecurityContext;
import
org.springframework.security.oauth2.server.authorization.token.JwtEncodingContext;
import
org.springframework.security.oauth2.server.authorization.token.OAuth2TokenCustomizer;
import org.springframework.stereotype.Component;

import java.util.ArrayList;
import java.util.List;

@Component
class RsaKeyPairRepositoryJWKSource implements JWKSource<SecurityContext>,
OAuth2TokenCustomizer<JwtEncodingContext> {

    private final RsaKeyPairRepository keyPairRepository;

    RsaKeyPairRepositoryJWKSource(RsaKeyPairRepository keyPairRepository) {
        this.keyPairRepository = keyPairRepository;
    }

①
    @Override
    public List<JWK> get(JWKSelector jwkSelector, SecurityContext context) throws
KeySourceException {
        var keyPairs = this.keyPairRepository.findKeyPairs();
        var result = new ArrayList<JWK>(keyPairs.size());
        for (var keyPair : keyPairs) {
            var rsaKey = new RSAKey.Builder(keyPair.publicKey()).privateKey(keyPair
.privateKey())
                .keyID(keyPair.id())
                .build();
            if (jwkSelector.getMatcher().matches(rsaKey)) {
                result.add(rsaKey);
            }
        }
        return result;
    }

②
    @Override
    public void customize(JwtEncodingContext context) {
```

```
        var keyPairs = this.keyPairRepository.findKeyPairs();
        var kid = keyPairs.get(0).id();
        context.getJwsHeader().keyId(kid);
    }

}
```

① when asked, we pass thrugh and return the list of `RsaKeyPair` instances from the database, turning them into `RSAKey.Builder` instances.

② when asked, we customize the JWTs that are generated by specifying the key ID, so that it lines up with the keys in the repository.

Let's see how all of this gets plugged into Spring Authorization Server through configuration.

```
package com.example.authorization_server.keys;

import com.nimbusds.jose.jwk.source.JWKSource;
import com.nimbusds.jose.proc.SecurityContext;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.encrypt.Encryptors;
import org.springframework.security.crypto.encrypt.TextEncryptor;
import org.springframework.security.oauth2.core.OAuth2Token;
import org.springframework.security.oauth2.jwt.JwtEncoder;
import org.springframework.security.oauth2.jwt.NimbusJwtEncoder;
import org.springframework.security.oauth2.server.authorization.token.*;

@Configuration
class KeyConfiguration {

①
    @Bean
    TextEncryptor textEncryptor(@Value("${jwk.persistence.password}") String pw,
            @Value("${jwk.persistence.salt}") String salt) {
        return Encryptors.text(pw, salt);
    }

②
    @Bean
    OAuth2TokenGenerator<OAuth2Token> delegatingOAuth2TokenGenerator(JwtEncoder
encoder,
            OAuth2TokenCustomizer<JwtEncodingContext> customizer) {
        var generator = new JwtGenerator(encoder);
        generator.setJwtCustomizer(customizer);
        return new DelegatingOAuth2TokenGenerator(generator, new
OAuth2AccessTokenGenerator(),
                new OAuth2RefreshTokenGenerator());
    }
```

```
③
    @Bean
    NimbusJwtEncoder jwtEncoder(JWKSource<SecurityContext> jwkSource) {
        return new NimbusJwtEncoder(jwkSource);
    }

}
```

① this is the `TextEncryptor` that's doing so much of the work for us in the converters we looked at earier. It's a `TextEncryptor`, as opposed to a `BytesEncryptor`. Ultimately, it's using an algorithm called AES, which is the gold standard in ciphers today, succeeding a long line of other algorithms - like DES and XDES - that go all the way back to the 1970's. It's both space efficient and cryptographically secure.

② We only want to plugin the `OAuth2TokenCustomizer` for the `JwtGeneator`, but need to redeclare the bean that uses it, the `OAuth2TokenGenerator`. most of this is boilerplate that we're copying from the defaults.

③ and finally we want to plugin our new `JWKSource` implementation

In this last example we injected some propertues, `jwk.persistence.password` and `jwk.persistence.salt`, which are credentials that need to be stored in a secure fashion!

The machinery is in place, but something needs to start it! We want the application to startup and automatically register a new `RsaKeyPair` if none exists in the database (otherwise the Spring Authorization Server wouldn't work!), and we want to be able to rotate the keys automatically. So, I've created a new Spring `ApplicationEvent`, called `RsaKeyPairGenerationRequestEvent`. We'll rig up a listener to rotate the keys whenever an instance of that event is published.

Any code anywhere in the Spring Authorization Server could publish that event and trigger a key rotation. You could have a `@Scheduled` method that runs every 24 hours and rotates the keys. You could create a (secured) HTTP endpoint that publishes the event. You could create a (secured) Actuator endpoint that publishes the event. You could write some code to listen to new messages coming in from Kafka and then publish the event in response. The skies the limit! The first we're going to publish the event, howeve? On startup, but only if we discover there are no keys in the repository. Let's see that event wiring.

```
package com.example.authorization_server.keys;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.context.event.ApplicationReadyEvent;
import org.springframework.context.ApplicationEventPublisher;
import org.springframework.context.ApplicationListener;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.time.Instant;

@Configuration
```

```
class LifecycleConfiguration {

①
    @Bean
    ApplicationListener<RsaKeyPairGenerationRequestEvent>
keyPairGenerationRequestListener(Keys keys,
            RsaKeyPairRepository repository, @Value("${jwk.key.id}") String keyId) {
        return event -> repository.save(keys.generateKeyPair(keyId, event.getSource()
));
    }

②
    @Bean
    ApplicationListener<ApplicationReadyEvent> applicationReadyListener
(ApplicationEventPublisher publisher,
            RsaKeyPairRepository repository) {
        return event -> {
            if (repository.findKeyPairs().isEmpty())
                publisher.publishEvent(new RsaKeyPairGenerationRequestEvent(Instant
.now()));
        };
    }

}
```

① this `ApplicationListener` listens for the aforementioned event and writes a new `RsaKeyPair` to the repository, using the injeted `jwk.key.id`, whcih should be specified externally, and should remain constant. that is, after all, the key ID.

② this `ApplicationListener` runs when the service starts and publishes a `RsaKeyPairGenerationRequestEvent`, but only if there are no `RsaKeyPairs` in teh repository already.

At this point you can delete the key files we generated with `openssl` earlier. You can also delete the relevant configration in your `application.properties` or `application.yml`; the application can now create and rotate its own keys.

To Production… and Beyond!

And that's it! Restart the Spring Authorization Server. Now you can run more than one instance of the Spring Authorization Server behind a load balancer, and no matter to which instance a client and its cookies present themselves, the container will resolve the session, authorizations, information about clients from the PostgreSQL database. It'll also sign keys in a consistent fashion. Is now a good time to remind you not to stash sensitive stuff in the HTTP session? You never know when it's going to be serialized to some datastore… = Federated OAuth with the Spring Authorization Server

Protecting a Simple HTTP API

With all that in place, you should be able to start the application. It's got a REST endpoint. You can try it out by hitting the new `/customers` endpoint that we created.

```
curl http://localhost:8081/customers
```

But it fails! Which is good. It fails because it's an authenticated request. We need a valid JWT token. We can get one because, when we registered the client with the Spring Authorization Server, we listed `client_credentials` as an authorized grant type. This in turn allows us to make a request without a user context. And if there's no user, then there's no need to verify the user, and thus no need for a browser or a web page or anything. We only need the client ID and client secret: `crm`/`crm`.

```
curl -X POST \
     -H "Authorization: Basic $(echo -n 'crm:crm' | base64)" \
     -H "Content-Type: application/x-www-form-urlencoded" \
     -d "grant_type=client_credentials&scope=user.read" \
     http://localhost:8080/oauth2/token
```

That should dump a token in a JSON document. On my machine I got this very long JSON document that I've abbreviated for you here.

```
{"access_token":"eyJraWQiOiI4YzQyNGU...Qy1Fg","scope":"user.read","token_type":"Bearer
","expires_in":299}
```

The important bit is the `access_token` attribute. If you have the `jq` command line utility installed, you can extract out the `access_token` like this

```
TOKEN=$( curl -X POST \
     -H "Authorization: Basic $(echo -n 'crm:crm' | base64)" \
     -H "Content-Type: application/x-www-form-urlencoded" \
     -d "grant_type=client_credentials&scope=user.read" \
     http://localhost:8080/oauth2/token | jq -r  .access_token )
```

By the way, you could also issue the same request using Spring's `RestTemplate` or `WebClient` or `RestClient`. Here's the equivalent using ye ole `RestTemplate`:

```
package com.example.client;

import org.springframework.http.HttpEntity;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.util.Assert;
import org.springframework.util.LinkedMultiValueMap;
```

```
import org.springframework.web.client.RestTemplate;
import tools.jackson.databind.JsonNode;

class ClientCredentialsClient {

    String getJwtToken(RestTemplate restTemplate, String clientId, String
clientSecret) {
        var headers = new HttpHeaders();
        headers.setBasicAuth(clientId, clientSecret);
        headers.setContentType(MediaType.APPLICATION_FORM_URLENCODED);

        var body = new LinkedMultiValueMap<String, String>();
        body.add("grant_type", "client_credentials");
        body.add("scope", "user.read");

        var entity = new HttpEntity<>(body, headers);
        var url = "http://localhost:8080/oauth2/token";
        var response = restTemplate.postForEntity(url, entity, JsonNode.class);
        Assert.state(response.getStatusCode().is2xxSuccessful(), "the response needs
to be 200x");
        return response.getBody().get("access_token").asText();
    }

}
```

You can use this token to then issue a request to the HTTP server:

```
curl -H "Authorization: Bearer $TOKEN" http://localhost:8081/customers
```

And there's the data! At long last, reunited with the data. Feels good doesn't it? We've got an HTTP API that is secure and all we had to do was specify the `issuer-uri` int he property file. Our Spring Authorization Server is paying dividends already! We've got honest-to-goodness identity management, security, and more, all for the cost of one lousy little property. And if we want to protect any other microservices, its the same story. One little `issuer-uri` property, and our services will automatically be protected and automatically be able to work with the identities in the centralized Spring Authorization Server.

I love this for us. We did something amazing here. Do you feel the possibilities? We stood up one little Spring Authorization Server and suddenly every microservice in our system can be protected. No need to redundantly duplicate the requests.

We sort of cheated here, though. We got a token using the `client_credentials` authorization grant type. No user context, remember? We *want* users. That's sort of the point of this whole exercise. Somewhere, somehow, we'll need to get a user involved. Once they're involved, they'll have a token that we can use to make requests to this resource server. The thing that originates the token, that forces the redirect to the OAuth IDP where the user will be asked to consent? That's called an OAuth 2 client. An OAuth 2 client is both an OAuth 2 resource server, in that it'll reject invalid requests, *and* it has this extra ability to initiate and handle the OAuth flow - the "OAuth dance" - we looked at

earlier when we discussed Yelp.com's authentication flow. Let's build one. :code: ..

The Gateway Client

In our super secure OAuth onion, this next microservice, the `gateway`, is the outermost layer. It is the first port-of-call for all requests destined for the microservices in our system. When visitors to our site punch in the domain name for our system into a browser, it'll resolve to a loadbalancer serving this gateway service, and it is here where we'll originate an OAuth token. This service is also a gateway, powered by Spring Cloud Gateway. The gateway acts as a proxy, allowing us to forward requests to different hosts and ports, concealing from the user that the responses they're seeing are from different services.

The `gateway` service's job is entirely to handle the OAuth dance - if it detects that the request is not authenticated - and to otherwise proxy requests to two other HTTP endpoints: the `api` we just built, and have running on port `8081`, and the static HTML process running on port `8020`.

Here's the routing table

HTTP origin	HTTP destination
http://gateway:8082/api/customers	http://api:8080/customers
http://gateway:8082/api/me	http://api:8080/me
http://gateway:8082/api/email	http://api:8080/email
http://gateway:8082/index.html	http://static:8020/index.html

The `gateway` also sends along the JWT token to the downstream HTTP endpoints, acting as a token relay. The `static` site won't care (it's just serving up static `.html` and `.css` files, after all), but the `api` is a resource server, and it will care. The `api` will refuse to send back data unless that token is specified.

Let's look at the Java code first.

```
package com.example.gateway_reactive;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.web.server.ServerHttpSecurity;
import org.springframework.security.web.server.SecurityWebFilterChain;

@Configuration
class SecurityConfiguration {

①
    @Bean
    SecurityWebFilterChain securityWebFilterChain(ServerHttpSecurity http) {
        http.authorizeExchange((authorize) -> authorize.anyExchange().authenticated()
)②
            .csrf(ServerHttpSecurity.CsrfSpec::disable)③
            .oauth2Login(Customizer.withDefaults())④
```

```
            .oauth2Client(Customizer.withDefaults());
        return http.build();
    }

}
```

① you could probably get away with not defining this bean at all *if* you were willing to deal with default behavior of CSRF tokens, which I am not. In the interest of simplicity, I'm disabling them, but in so doing I am also disabling all the other things Spring Security assumed I wanted. So we will also go through and re-enable those things.

② We want all HTTP requests to be authenticated…

③ …and to disable the CSRF support…

④ …and to re-enable OAuth 2 OIDC login support and the OAuth 2 client support. The OIDC login functionality is what triggers the OAuth dance we've talked about. The OAuth 2 client support is what tells Spring Security, running in this process, as which OAuth 2 client requests for OIDC login should be done. We'll need to specify the particulars in the property configuration later.

The security configuration is pretty straightforward. We're an OAuth client. We want to prompt users to login with a particular client. Once a user is authenticated, well, there's not much for them to see! We need Spring Cloud Gateway to connect our other HTTP services to the user visiting this `gateway` service.

We'll configure two Spring Cloud Gateway *routes*. Each request has a predicate, optional filter(s), and a destination URI. The predicate defines how requests to the Spring Cloud Gateway service are matched, e.g.: does this request have a particular header or cookie, or a particlar path, or a particular virtual host? You may specify one or more filters that act on the incoming requests, changing it. Finally, the request, after it has passed through any and all filters, is sent to a final destination, which we specify with a URI.

```
package com.example.gateway_reactive;

import org.springframework.cloud.gateway.route.RouteLocator;
import org.springframework.cloud.gateway.route.builder.RouteLocatorBuilder;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
class GatewayConfiguration {

    @Bean
    RouteLocator gateway(RouteLocatorBuilder rlb) {
        var apiPrefix = "/api/";
        return rlb.routes()
①
            .route(rs -> rs.path(apiPrefix + "**")
                .filters(f -> f.tokenRelay().rewritePath(apiPrefix + "(?<segment>.*)",
"/$\\{segment}"))
                .uri("http://localhost:8081"))
```

```
②
            .route(rs -> rs.path("/**").uri("http://localhost:8020"))
            .build();
    }

}
```

① the first route matches all requests to `/api/**`, notes and forwards any OAuth JWTs to the backend service, and changes the path of the request from `gateway:8082/api/foo` to `api:8081/foo`, dropping the `/api/` bit.

② the second route takes every other request and sends it unchecked on to the HTTP endpoint service up the static HTMl 5 and JavaScript assets.

That's just about all the Java code for this service, but its role and importance in the architecture can not be overstated. Let's look at the property file that ties it all together.

```
spring.application.name=gateway-reactive
①
server.port=8082
②
spring.security.oauth2.client.provider.spring.issuer-uri=http://localhost:8080
③
spring.security.oauth2.client.registration.spring.provider=spring
spring.security.oauth2.client.registration.spring.client-id=crm
spring.security.oauth2.client.registration.spring.client-secret=crm
spring.security.oauth2.client.registration.spring.authorization-grant-
type=authorization_code
spring.security.oauth2.client.registration.spring.client-authentication-
method=client_secret_basic
spring.security.oauth2.client.registration.spring.redirect-
uri={baseUrl}/login/oauth2/code/{registrationId}
spring.security.oauth2.client.registration.spring.scope=user.read,openid
```

① this process will run on port `8082`.

② it will use the issuer URI to validate JWTs if it detects them

③ otherwise it will use this configuration to, acting as a client, inintiate an OIDC login to allow you to come up with a valid token.

The last several lines are where we tell the OAuth client what it should ask for from our OAuth IDP (the Spring Authorization Server). You'd write similar configuration for any OAuth IDP, not just the Spring Authorization Server. In this example, the client is configured to identify itself as the `crm` client, with `crm` as the client secret. It's being configured to ask for the `authorization_code` authorization grant. It's being configured to instruct the OAuth IDP to rediect *back* to the gateway, with a special pseudo URL syntax: `{baseUrl}/login/oauth2/code/{registrationId}`. Spring Security will set that endpoint up for us on the gateway, so there's nothing we need to do for that to work. Finally, the OAuth client asks for certain permissions: `user.read` and `openid`.

A User Interface for a Browser User

The OAuth client is where all outside visitors will begin their journey. But obviously, they're not going to be issuing HTTP requests with `curl` , they're going to be expecting some sort of user interface. We looked at how Spring Cloud Gateway requests will route proxied resources: `/api/**` to our backend `api` and everything else to the static HTML5 and JavaScript code. In this case, it's just running on our local machines, but one images this stuff would be deployed to a CDN and made available locally in a real production application.

In order to keep things as simple as possible, I've written a snigle page `.html` application with a smattering - a pinch, a dash, even! - of JavaScript. The concepts will be the same whether you eventually use Vue.js, React, Angular, jQuery, or whatever other thing you decide to use.

Here's the `.html` file. It has a panel to greet the signed in user and another to show a grid of `customer` records.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Spring CRM</title>
    <link rel="text/css" href="app.css"/>
</head>
<body>
<script src="app.js"></script>
<div>welcome, <span style="font-weight: bold" id="me"></span></div>
<div id="customers"></div>
</body>
</html>
```

Both the user name and the `customer` records we'll load by talking to the API from JavaScript.

```
①
const root = '/api'

async function customers() {
    const response = await fetch(root + '/customers')
    return await response.json()
}

async function me() {
    const response = await fetch(root + '/me')
    return await response.json()
}
```

```
②
async function email(customerId) {
    const response = await fetch(root + '/email?customerId=' + customerId, {method:
'POST'})
    const data = await response.json()
    const cssQuerySelector = '#' + divIdFor({id: customerId}) + ' .id'
    document.querySelector(cssQuerySelector).classList.add('fade')
    return data
}

③
function divIdFor(customer) {
    return 'customerDiv' + customer.id
}

④
window.addEventListener('load', async (event) => {
    document.getElementById('me').innerHTML = (await me()).name;
    const customersDiv = document.getElementById('customers')
    const customersResults = await customers();
    customersResults.forEach(customer => {
        const div = document.createElement('div')
        div.innerHTML = `
              <button type="button">email report</button>
              <span class="id"> ${customer.id} </span>
              <span> ${customer.name}</span>
            `
        div.id = divIdFor(customer)
        customersDiv.appendChild(div)
        document
            .querySelector('#' + divIdFor(customer))
            .addEventListener('click', () => email(customer.id))
    });
})
```

① the first two functions of the file do the work of interacting with our API using the non-blocking, but oh-so-verbose, `async`/`await` syntax. (If only JavaScript had Java 21's Project Loom and virtual threads!). NB: we're making these requests relative to the `/api` path. We've written the JavaScript code in such a way that it knows that it's being accessed via an HTTP proxy, our Spring Cloud Gateway gateway.

② this function is a little more interesting in that we're also interacting with the backend API but we're sending `POST` calls, not `GET` calls. And of course we've got a little animation goin' on.

③ there are at least two separate places that I need the `id` that'll have been assigned to a `div` for records for a particular `customer`. So I've extracted it into a sepearate function here.

④ this is the heart of the code: when the page loads, we read the data for the customers and the user's name and then draw the page with some DOM manipulation. This code also wires up an event so that when you click on one of the rendered buttons, it triggers a `POST` which initiates a message sent to RabbitMQ.

I love this code, not because of the JavaScript (bleargh!), but because it's so plain to understand. Remember, this code will run in a browser that has cookies associated with an HTTP session, and that HTTP session in turn is connected with a valid OAuth token. So each time the JavaScript code calls `/api/customers`, it's implicitly sending the associated cookie to the gateway code, which in turn then allows the gateway to find the OAuth session associated with this user and to then forward the token to the backend `api`.

At no point does the JavaScript code ever see your username and password. And, in this case, it doesn't even see your JWT! The code is written in such a way that it doesn't even know OAuth is involved. Now you can develop the application without OAuth, get it working, then add the OAuth resource server and OAuth client support to the backend code and call it done.

Remember, the user visiting the HTML page won't even see any of this markup unless they've authenticated. They'll be redirected away before they ever get a chance. This page is both completely and blissfully unaware of any security, and it doesn't need to be aware of security.

GraphQL

In this section, we'll look at how to secure Spring GraphQL services with OAuth and method security, alike. :code: ..

gRPC

In this section, we'll look at how to secure Spring gRPC applications with Spring Security, OAuth, and more. :code: ..

Secure SOAP-based Web Services

Model Context Protocol

Messaging

This chapter is my favorite one. Not because it's going to be the most useful, but because it's the one that took the longest for me to figure out. You see, I love backoffice code. I love messaging, and integration. I love analytics. I love workflow. I love stream and batch processing. I love workflow engines and business process management. I love grid computing. I love short-lived tasks like CRON jobs. I love all the sorts of stuff that has no business being anywhere near an HTTP request or taking up space on an HTTP webservice. We used to call these sorts of things "backoffice" jobs. And you've probably built some of these in your career, too.

And I didn't know how OAuth fit in this world of backoffice code. I mean, just look at the Spring portfolio! We've got Spring Integration, Spring AMQP, Spring for Apache Kafka, Spring Cloud Data Flow, Spring Cloud Task, Spring Batch, and even Spring Shell. You can build all sorts of cool stuff with those libraries and never see an HTTP header! So how does OAuth fit here? And, more precisely, how does Spring Security's OAuth support plugin? We've already met the usual suspects: `spring-boot-starter-oauth-client`, `spring-boot-starter-resource-server`, etc. But those all assume the presence of an HTTP server and the Spring Security web filter chain.

In this chapter we're going to expand our sample application a bit with some Spring Integration code that will receive a message that the API (the resource server) will send. Each message will contain the JWT token associated with the authenticated user and it'll contain a payload tat the processor will.., you know, process. In this example, we'll imagine that this processor is busy doing the work of sending emails. We're not going to actually write that bit of the code. But it is a good example. After all, sending email can sometimes take a long time and we don't want to keep the API busy doing this when it should be fielding HTTP requests.

When the message arrives at this `processor` module, we'll validate the JWT token by talking to the Spring Authorization Server (our OAuth IDP) through its issuer URI. Does this sort of sound familiar? It should! We're going to basically do the same trick as the resource server did, albeit a bit more granuarly. Well get to see how the resource server support does some of its work. Its good to know this because, outside of an HTTP environment, there's no one-sized fits all approach. It's convenient then that we can easily plug this stuff in ourselves.

Spring Integration

Let's talk again about Spring Integration. we looked at it ever so briefly when we introduced the API, but let's review some basics.

Remember this code from the API we built earlier?

It's a bean of type `IntegrationFlow`, from the Spring Integration project. The bean describes how we handle messages intended for a RabbitMQ broker: messages come, then we turn it into JSON, then we send it over AMQP. Simple.

Spring Integration is an old project, from 2007. The core conceit fo Spring Integration is that as we move forward in time the body of systems and services with which we need to integrate - to maximally retain value - grows, and the protocols and paradigms required for integration also grow. Spring Integration is an enterprise application technnology. It's designed to help glue systems

together, particularly those things that wouldn't otherwise know about and work with each other.

In 2004, Gregor Hohpe and Bobby Woolf wrote the book *Enterprise Integration Patterns* [https://enterpriseintegrationpatterns.com], which gave us the names for the patterns typical of integration solutions. Broadly, the book said, there are four different kinds of integration styles.

- **RPC**: in this style, network services are made to work like a local object. In such a style, a client could invoke a method on a local Java object and have that translated into remote procedure calls on another object on another object, presumably running on another host. This style feels simple but it hides the reality of the network - that it will fail. It makes assumptions that it shouldn't, namely that the service will always be available. If the service - the consumer - is not available then has to retry or abandon the integration. Because of these design tradeoffs, RPC is a poor choice for service integration.
- **Shared Database**: in this style, a client connects the same database (e.g.: Oracle, PostgreSQL, MongoDB, etc), writes data there that another client then reads. This is fragile because a schema change by one client might break another client. This approach completely violates the principles of encapsulation, exposing the peculiarities data storage to consumers, and is therefore a poor choice for service integration. There's a reason nobody ever says "put your best liver forward!" It's nobody's business what your liver looks like! They should be shaking your hand, instead. The same is true in integration: don't share too much.
- **File synchronization**: in this style, a client deposits a file on a filesystem (NFS, FTP, FTPS, SFTP, SMB, etc.) that a client then consumes. This approach works alright but care must be taken so that the consumer of a file doesn't start processing it before the producer has finished writing it. Additionally, this approach lacks any sort of sophistication around message delivery (once and only once, transactions, message rollback) or routing.
- **Messaging**: in this style, integration is done in terms of a messaging system like Apache Kafka or RabbitMQ. This is usually the best approach if you can get access to it. Messaging systems support transactions, they can be made to ensure a message is delivered, that it is delivered at most once, or at least once, etc. It can hanndle routing, allowing you to send the message to different consumers as required. And of course it does not couple producer or consumer; a consumer can send a message to the messaging system, and it'll be recorded there. When the consumer is available and able, it can read and process the message. We can say that this style of integration is the most decoupled: producers and consumers don't need to both be available at the same time; producers and consumers only see and agree upon message payloads, and not the internals of their state management schemes.

Generally, speaking, messaging is the most flexible approach to building and integrating systems. So it is that Spring Integration models everything as `Message` objects that pass through `MessageChannel` objects. A `MessageChannel` is the connective tissue between components that act on `Message` objects, in a sort of pipeline. These components are written in terms of the `Message`s they accept and the `Message`s they produce. They're otherwise usually stateless. In a way, Spring Integration encourages a lot of the same discipline and conventions as any functional programming language might. You're encouraged to write your business logic in terms of granular, composable, and reusable functions.

Where do these messages come from, and where do they go? Well, the real world of course! They have to come from somewhere. It is Spring Integration's job to connect our code to events in the

real world and translate them into `Message` objects: a microwave turned on (MQTT), a new file appeared in an FTP service, a new row appeared in a SQL database, a new email was sent to an inbox, a message arrived on a JMS destination, etc. It does this work with adapters which *adapt* events into `Message` objects. Each `Message` typically contains a payload (the inbound file adapter might contain a `java.io.File` payload, for example) and headers telling us about the payload (the folder in which the file was found, or the timestamp of when the message was produced).

Once we have a `Message` ,we can do all sorts of things to it. We could split it into smaller messages, route it to other handlers, add information to it, filter it, etc.

And then when we're finally done with it, we use an outbound adapter - which does the reverse of an inbound adapter -to send the message onward to some place in the real world.

So, to review the review: events come via inbound adapters, they're turned into `Message` objects with headers and payloads. In this form they're processed, and ultimately sent out via outbound adapters.

Now back to our regularly scheduled programming.

Defining RabbitMQ Infrastructure

First thing's first: we're using RabbitMQ. In RabbitMQ, you send messages to an exchange which then can route it anyway it wants. In our case, it's going to be sent to a single queue, which is where consumers will know to look for it. So, we have an exchange and a queue, and they're bound together through something called a binding. We'll use the Spring AMQP project to make short work of defining these things. If we define them as beans, Spring AMQP will automaticall create the real structures on the RabbitMQ broker.

```
package com.example.messaging;

import org.springframework.amqp.core.*;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import static com.example.messaging.Constants.RABBITMQ_DESTINATION_NAME;

@Configuration
class AmqpConfiguration {

    @Bean
    Queue queue() {
        return QueueBuilder.durable(RABBITMQ_DESTINATION_NAME).build();
    }

    @Bean
    Exchange exchange() {
        return ExchangeBuilder.directExchange(RABBITMQ_DESTINATION_NAME).build();
    }
```

```
    @Bean
    Binding binding() {
        return BindingBuilder.bind(queue()).to(exchange()).with
(RABBITMQ_DESTINATION_NAME).noargs();
    }

}
```

You'll notice that this code uses a constant variable I've defined. There are a few others…

A Few Well Known Constants

There are a few things that I've defined as constants: a header name, a `MessageChannel` bean ID, and the RabbitMQ destination.

```
package com.example.messaging;

public class Constants {

    public final static String REQUESTS_MESSAGE_CHANNEL = "requests";

    public static final String RABBITMQ_DESTINATION_NAME = "emails";

    public static final String AUTHORIZATION_HEADER_NAME = "jwt";

}
```

The Integration

Now we're into the meat of the example, the actual Spring Integration code.

In Spring Integration, an `IntegrationFlow` is the definition of a processing pipeline. You may have more than one `IntegrationFlow` in your application. `IntegrationFlow` objects chain together different components that act on the `Message` objects within the flow. You can route messages from one `IntegrationFlow` to another using `MessageChannel` instances.

Conceptually, all we want to do is take a message from the inbound AMQP (that's the protocol that RabbitMQ speaks) in and then print out the message. One might send an email or do something useful here, but we'll leave that as an exercise for another day… And yet, I've written it a little more obtusely: we have one `IntegrationFlow` that takes the message rom AMQP and then stuffs the message into a `MessageChannel`. It pops out the other side and *then* we print out the message's payload and headers. Why the indirection? Why add the `MessageChannel` into the mix, I hear you query. `MessageChannel` objects can have interceptors that can, in effect, *veto* messages. So we'll configure some interceptors to act on the message, validate the attached JWT token, and if it doesn't match, to reject the message *before* it gets to whatever important business logic we've got downstream of thee `MessageChannel`.

Let's see it all in action.

```
package com.example.messaging;

import org.springframework.amqp.rabbit.connection.ConnectionFactory;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.integration.amqp.dsl.Amqp;
import org.springframework.integration.dsl.DirectChannelSpec;
import org.springframework.integration.dsl.IntegrationFlow;
import org.springframework.integration.dsl.MessageChannels;
import org.springframework.messaging.MessageChannel;
import org.springframework.security.authorization.AuthenticatedAuthorizationManager;
import
org.springframework.security.messaging.access.intercept.AuthorizationChannelIntercepto
r;
import
org.springframework.security.messaging.context.SecurityContextChannelInterceptor;
import
org.springframework.security.oauth2.server.resource.authentication.JwtAuthenticationPr
ovider;

import static com.example.messaging.Constants.AUTHORIZATION_HEADER_NAME;
import static com.example.messaging.Constants.RABBITMQ_DESTINATION_NAME;

@Configuration
class IntegrationConfiguration {

①
    @Bean
    IntegrationFlow inboundAmqpRequestsIntegrationFlow(
            @Qualifier(Constants.REQUESTS_MESSAGE_CHANNEL) MessageChannel requests,
            ConnectionFactory connectionFactory) {
        var inboundAmqpAdapter = Amqp.inboundAdapter(connectionFactory,
RABBITMQ_DESTINATION_NAME);
        return IntegrationFlow.from(inboundAmqpAdapter).channel(requests).get();
    }

②
    @Bean
    IntegrationFlow requestsIntegrationFlow(@Qualifier(Constants
.REQUESTS_MESSAGE_CHANNEL) MessageChannel requests) {

        return IntegrationFlow.from(requests)//
            .handle((payload, headers) -> {
                IO.println("----");
                headers.forEach((key, value) -> IO.println("%s=%s".formatted(key,
value)));
                return null;
            })//
            .get();
```

```
    }

③
    @Bean(Constants.REQUESTS_MESSAGE_CHANNEL)
    DirectChannelSpec requests(JwtAuthenticationProvider jwtAuthenticationProvider) {
④
        var jwtAuthInterceptor = new JwtAuthenticationInterceptor
(AUTHORIZATION_HEADER_NAME, jwtAuthenticationProvider);
⑤
        var securityContextChannelInterceptor = new SecurityContextChannelInterceptor
(AUTHORIZATION_HEADER_NAME);
⑥
        var authorizationChannelInterceptor = new AuthorizationChannelInterceptor(
                AuthenticatedAuthorizationManager.authenticated());
        return MessageChannels.direct()
            .interceptor(jwtAuthInterceptor, securityContextChannelInterceptor,
authorizationChannelInterceptor);
    }

}
```

① the first `IntegrationFlow` defines an AMQP Inbound adapter which listens for new `Messages` on our auto configured RabbitMQ connection. As soon as a message comes in, it gets sent to the next step in the `INtegrationFlow`. IN this case, that next step is to travel through an injected `MessageChannel` to another `IntegrationFlow`.

② the second `IntegrationFlow` takes whatevers been stuffed into the `MessageChannel` and passews it to the next step, which is a simple handler that inspects the message payload and headers.

③ both `IntegrationFlow` objects are connected by the `MessageChannel` whoe bean ID is `Constants.REQUESTS_MESSAGE_CHANNEL`, and whose definition we see here. It's a bit complicated but this is where we do the work of validating the JWT token.

④ most of the important woerk is done in these interceptors. Each interceptr can inspect, transform, or reject the `Message` objects flowign through the `MessageChannel` on which they're configured. I wrote this first interceptor. We'll look at it shortyl, but suffice it to say that it in turn is using the injected `JwtAuthenticationProvider` that is provided by Spring Security and whose configuration we'll examine momentarily to do the work of validating the JWT token against the Spring Authorization Server. If the token is valid, the interceptor creates a new message with a valid Spring Security `Authentication` in the header. If the token is not valid, then the interceptor will create a new message with a null value for the header.

⑤ this next interceptor hoists the `Authentication` from the message header and puts it into the Spring Security `SecurityContextHolder`, which is a well-known `ThreadLocal`-like holder of the current thread's authenticated user. Now, all the other machinery in Spring Security that consults this thread local will work correctly.

⑥ the final interceptor does the actual check, rejecting the message if the current thread doesn't have a valid, authenticated `Authentication`.

The `JwtAuthenticationInterceptor` leaves nothing to the imagination:

```
package com.example.messaging;

import org.springframework.messaging.Message;
import org.springframework.messaging.MessageChannel;
import org.springframework.messaging.support.ChannelInterceptor;
import org.springframework.messaging.support.MessageBuilder;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.AuthorityUtils;
import
org.springframework.security.oauth2.server.resource.authentication.BearerTokenAuthenti
cationToken;
import
org.springframework.security.oauth2.server.resource.authentication.JwtAuthenticationPr
ovider;
import org.springframework.util.Assert;

class JwtAuthenticationInterceptor implements ChannelInterceptor {

①
    private final JwtAuthenticationProvider authenticationProvider;

②
    private final String headerName;

    JwtAuthenticationInterceptor(String headerName, JwtAuthenticationProvider ap) {
        this.headerName = headerName;
        this.authenticationProvider = ap;
    }

    @Override
    public Message<?> preSend(Message<?> message, MessageChannel channel) {
③
        var token = (String) message.getHeaders().get(headerName);
        Assert.hasText(token, "the token must be non-empty!");

④
        var authentication = this.authenticationProvider.authenticate(new
BearerTokenAuthenticationToken(token));

⑤
        if (authentication.isAuthenticated()) {
            var upt = UsernamePasswordAuthenticationToken.authenticated(
authentication.getName(), null,
                    AuthorityUtils.NO_AUTHORITIES);
            return MessageBuilder.fromMessage(message).setHeader(headerName, upt)
.build();
        }

⑥
```

```
        return MessageBuilder.fromMessage(message).setHeader(headerName, null).build(
);
    }

}
```

① We've defined this bean elsewhere and injected it here

② in which header shall we place the authentication once we've validated the token?

③ extract the token from the header

④ use the `AuthenticationProvider` to talk to the Spring Authorization Server.

⑤ if the JWT is in fact valid then we'll create a new `Message`, cloning the headers and payload from the incoming message, but specufying one new header, whose value is an object of type `Authentication`.

⑥ if the JWT is not valid, then create a new `Message` with a null header value. :code: ..

Command Line Interfaces

In this section, we're going to look at how to secure command line interfaces (CLIs) with OAuth and Spring Security. We'll use Spring Shell to build up the CLI and Spring Security's OAuth support, as usual, to do the rest.